XPP Studio

An introduction and tutorial

Peter Ashwin and Bard Ermentrout
with Claude Opus 5.5 (Anthropic)

Version 1.4, November 2026

Abstract

XPP Studio is a browser-based environment for exploring dynamical systems, modelled on Bard Ermentrout's XPPAUT. It reads XPPAUT's `.ode` files and provides phase planes, three-dimensional views, time courses, numerical continuation of equilibria and periodic orbits, one- and two-parameter bifurcation diagrams, maps, stochastic and Markov models, stiff solvers, delay equations, events (global flags), boundary value problems, arrays and Volterra integral equations, and computes the phase and isostable reduction of limit cycles for weakly coupled oscillators. Everything runs locally in a web page: there is nothing to install and no data leaves the browser. This document introduces the interface, describes the model language, and works through a sequence of short tutorials using the examples that come with the program.

Contents

1 Introduction **2**

2 A tour of the interface **3**

3 The model language **6**

4 Tutorial 1: phase-plane analysis **9**

5 Tutorial 2: one-parameter bifurcation diagrams **11**

6 Tutorial 3: two-parameter continuation **15**

7 Tutorial 4: chaos in three dimensions **19**

8 Tutorial 5: maps **20**

9 Tutorial 6: noise and random switching **22**

10 Tutorial 7: stiff systems **22**

11 Tutorial 8: arrays and Volterra equations **23**

12 Tutorial 9: delay equations **25**

13 Tutorial 10: events and global flags **27**

14 Tutorial 11: boundary value problems **29**

15 Tutorial 12: scripts for reproducible figures **30**

16 Tutorial 13: a forced oscillator and Arnold tongues **35**

17 Tutorial 14: homoclinic and heteroclinic orbits **36**

18 Tutorial 15: phase reduction of a periodic orbit **40**

19 Tutorial 16: animating a model **55**

20 Numerical settings **56**

21 Saving and exporting results **59**

22 Differences from XPPAUT **61**

23 Credits and licence **62**

1 Introduction

About this version. Features have been added since many of the screenshots and figures in this document were made, and the earlier pictures have not been remade. The text is current and the pictures may differ from the program in small ways. The new features are described in the later tutorials rather than in the tour of the interface, in particular Tutorials 13 to 15. The most visible change is a sixth view, **Phase reduction** (key 6), which now holds the adjoint and the interaction function of coupled oscillators; the screenshots of the interface show five views, and the Bifurcation panel in them still has the **Adjoint** row, which has since moved to the new view. Tutorial 15 (Section 18) replaces the earlier tutorial on the adjoint, and adds the isostable functions, the second-order theory of weakly coupled oscillators, and an example with exact answers.

New in version 1.4. The button at the top that brings the display up to date is now called **Update**, and the status line is highlighted while anything is being computed. A single **Export...** button offers CSV, SVG or DAT in every view (Section 21). The time series under the phase plane can show any variables and auxiliary functions (**Variables**, Section 2). The **Special points** table is grouped under the values of the second parameter, with **Follow** always the last button of a row. A continuation step **Step ds** that you set yourself is no longer changed by zooming, and two-parameter curves have a step of their own. In the hi-slo-lo diagram the slow points fade out towards the speed threshold. In the Script tab, **Load .ode script** replaces **Run file script**, the list of commands is under **Supported script syntax**, loading a different model asks whether to keep the script, and **Last point → IC** and the **Reset** buttons are recorded.

For more than thirty years XPPAUT [1] has been one of the standard tools for simulating and analysing differential equations in mathematical biology, neuroscience and applied dynamical systems. Its strengths are a compact, readable model language, fast interactive phase-plane analysis, and a tight connection to the continuation package AUTO [2]. XPP Studio keeps that model language and that way of working, and puts them in a modern interface:

- **Same files.** Most XPPAUT `.ode` files load unchanged. Supported features include parameters, functions, fixed quantities, auxiliary variables, maps, Wiener noise, Markov variables, tables, periodic (folded) variables, arrays, Volterra integrals, delays, global flags and boundary conditions (Section 3).
- **Live analysis.** Dragging a parameter slider immediately recomputes trajectories, nullclines and equilibria. For large systems, or when you want to change several things before anything is recomputed, the **Auto-update** box can be unticked (Section 20).
- **Built-in continuation.** Equilibria, periodic orbits and their bifurcations can be continued in one or two parameters without leaving the program. Detected points include folds, Hopf, period-doubling, torus, fold-of-cycles and codimension-two points.
- **Publication output.** Every view can be exported as a vector SVG figure, and its data as CSV or as plain DAT columns, from one **Export...** button.

The program is a single web page. To use it, open

`https://xpp-studio.github.io/xpp-studio/`

in any recent browser. The source code, the example models and this tutorial are in the repository `https://github.com/xpp-studio/xpp-studio`. Its file `index.html` is the whole program, and it also runs when opened directly from disk.

How this document is organised. Section 2 is a tour of the interface and Section 3 summarises the model language. Sections 4–11 are hands-on tutorials, each built on an example from the **Model** menu. Section 15 shows how to record and replay the steps behind a set of figures with a script. Sections 16, 17 and 18 are three further tutorials, on a forced oscillator, on homoclinic and heteroclinic orbits, and on phase reduction and weakly coupled oscillators. Section 20 covers numerical settings and Section 21 exporting results. The document ends with the differences from XPPAUT and with credits.

2 A tour of the interface

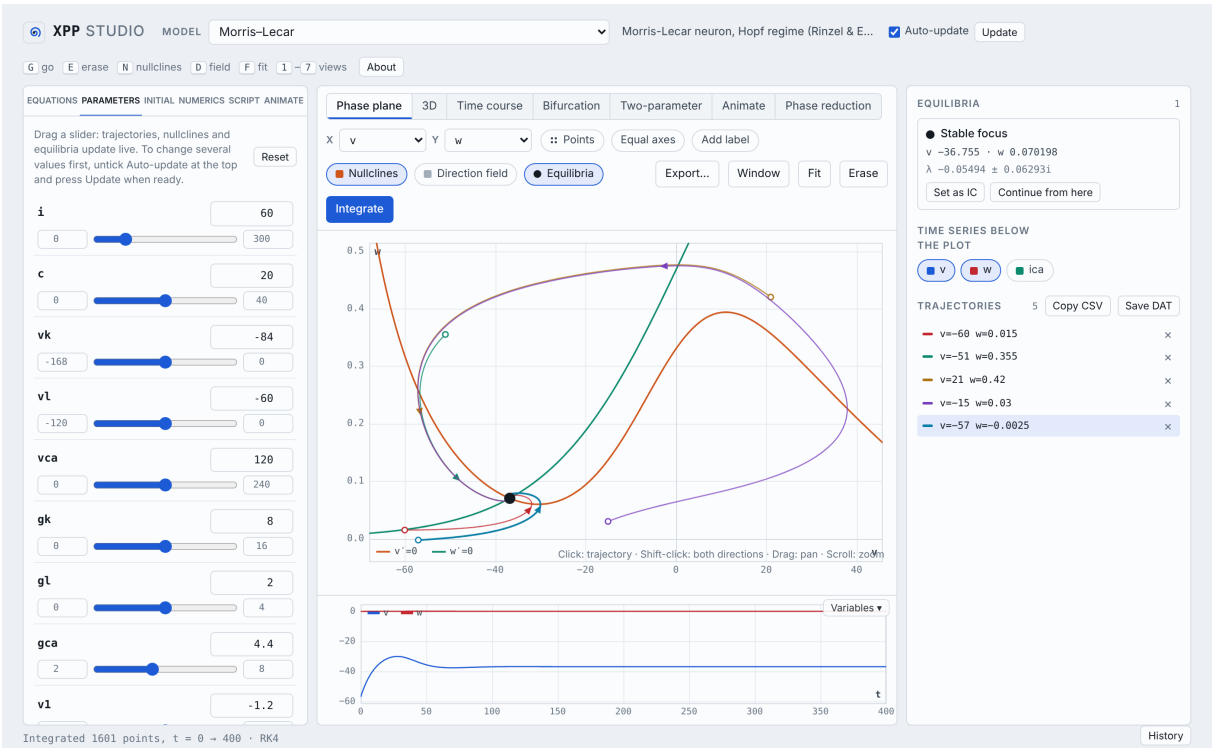


Figure 1: The XPP Studio window with the Morris–Lecar example loaded. Left: tabs for equations, parameters, initial conditions and numerics. Centre: the plot, here the (v, w) phase plane with nullclines, five trajectories and a stable focus, plus a time-series strip underneath, whose **Variables** button chooses what it shows. Right: the inspector, listing equilibria, the variables of the time series, and the trajectories.

The window (Figure 1) has three columns.

Left: the model. Six tabs:

Equations The `.ode` text, which you can edit freely. **Apply** (or Ctrl+Enter) reloads the model, and messages underneath report what was loaded and any errors, with line numbers. **Open .ode** loads a file from disk, **Copy text** copies the current text, **Write state** writes the current parameters, initial conditions and settings into it (Section 21), and **Table files** supplies data files used by `table` declarations. **Reset all**, under the messages, puts XPP Studio back as it is the first time it is opened: the default model (Morris–Lecar), view and settings, with no trajectories, diagrams or labels. It also forgets what the browser has kept between visits (your edited equations, the script, table files and display choices such as **Legend**), so copy anything you want to keep first; it asks before it does anything. To go back to the values in the current file without losing the file, press **Apply** instead.

Parameters One row per parameter, with a value box and a slider whose limits you can edit. Drag a slider to see everything update live (unless **Auto-update** is unticked, see below), and use **Reset** to restore the values in the file. The up and down arrows on a value box, and the arrow keys on a slider, change the value by 1% of the slider's range (10% with Shift).

Initial Initial values of the state variables (and Markov states). **Integrate** runs from here, and **Last point** → **IC** (in blue) copies the end of the current trajectory into the initial conditions; in a script this is the command `last`. **Fill by formula**, at the top, sets a whole array at once, as XPPAUT's **Initialconds** formula does: with `u[0..49]` and `sin(2*pi*[j]/50)` it sets $u_j(0) = \sin(2\pi j/50)$ for $j = 0, \ldots, 49$. The formula may use `[j]`, the parameters as they are now, and `ran(1)`, which gives a new random value each time **Set** is pressed. Unlike editing the file and pressing **Apply**, this keeps the parameters you have changed. The section appears only when the model has an array of variables (three or more state variables with consecutive numbers, such as `u0`, `u1`, `u2`, ...).

Numerics Integration method, total time, transient, step size, tolerance and bound, plus the continuation settings (Section 20).

Script A list of commands that drives the interface, to make a set of figures again exactly, with one click (Section 15).

Animate An animation script for the model, in XPPAUT's animation language, with **Open .ani** to read an `.ani` file and **Put in .ode** to keep the script in the equations (Section 19).

Centre: the plot. Seven views, chosen with the buttons above the plot or with the keys 1–7:

Phase plane Any two variables (state, Markov, Volterra or auxiliary) against each other, with optional nullclines, direction field and equilibria. Nullclines are on when a planar system of differential equations is loaded, and off otherwise: in higher dimensions they are slices through the current values of the other variables, slower to draw and seldom what is wanted. Press N to show them. For a one-dimensional map this view shows a cobweb diagram. **Equal axes** gives both axes the same scale. **Run grid** in the Initial tab starts trajectories from a grid of points, by default covering the window (Section 4). Below the plane a strip shows the time series of the current trajectory. **Variables**, at the top right of the strip, chooses what it shows: any of the variables and auxiliary functions, ticked in a small list (Figure 2). The choice is the same as the chips under *Time series below the plot* in the inspector, and as the variables of the Time course view.

3D Three variables (or time) in a rotatable box: drag to rotate, scroll to zoom, **Spin** to animate. **Equilibria** shows or hides the equilibria; while they are hidden, the box fits the trajectories alone.

Time course Chosen variables against t (or the iteration number n for maps). **Spectrum** shows their power spectra instead, and **Space-time** shows a row of indexed variables as an image (Section 11).

Bifurcation Branches of equilibria and periodic orbits, parameter sweeps, and homoclinic and heteroclinic orbits, against one parameter. The vertical axis is a variable, or the period or frequency of the periodic orbits.

Two-parameter Curves of folds, Hopf points, period doublings and bifurcations of cycles in a parameter plane.

Phase reduction The phase response (adjoint) of periodic orbits, the isostable response and Floquet eigenfunction of their slowest multiplier, and the interaction functions of two weakly

coupled copies, to first and second order (Section 18). Its controls are in the right-hand panel.

Animate The current trajectory played as a moving picture drawn by the animation script, with a time slider and export to GIF or WebM (Section 19).

Across the top, next to the model's title, are three controls that decide when things are computed and which values are used. **Auto-update** is a tick box: while it is ticked, changing a parameter, an initial value or a numerical setting runs the trajectories again at once. When it is unticked, **Update** brings the display up to date on demand. **Sets** appears only when the file defines any, and applies a named collection of parameters, initial values and settings in one step (Section 3). Section 20 describes Auto-update and the handling of long computations.

The status line at the bottom of the window reports what the last action did, and is highlighted, with a pulsing dot, while anything is being computed (a run, a continuation, a sweep, a grid or a script): how many points a trajectory has, why a branch stopped, where a special point was found. **History**, at its right-hand end, lists every message since the page was opened, each with its time, so that a report that has scrolled away can be read again. Progress reports during a long computation are kept as one line, the last one. **Download** saves the list as a text file, **Copy** copies it and **Clear** empties it; in a script, `history save` does the same (Section 15).

In every view you drag to pan and scroll to zoom. **Fit** rescales to the data, and **Window** sets the axis limits by hand (they then stay fixed until you press **Fit**). **Points** draws trajectories as dots instead of lines, which is the default for maps. **Export...** asks what to save: *CSV*, the data behind the view as a table; *SVG*, the view as a vector figure; or *DAT*, the data as plain columns of numbers (Figure 2, Section 21).



Figure 2: Left: **Export...** in the phase plane, with what each format saves in this view. Right: **Variables** on the time series under the phase plane, here showing v and the auxiliary current `ica`.

Labelling a plot. The **Add label** chip above the plot puts a text label on the figure. It appears in the phase plane, the time course (including the spectrum and the space-time plot), the Bifurcation view and the two-parameter plane, but not in the 3D views. Press it, click the plot where the label should go, and type the text in the small box that opens; a new line in the box starts a new line of text. The box also sets the size (small, medium or large) and the colour: the ordinary text colour, the accent, one of the colours used for curves and trajectories, or any

colour from the colour picker. Close the box with **Done**, Ctrl+Enter, Esc or a click elsewhere; a note with no text is discarded.

- **Move, edit, delete.** Drag a note to move it, double-click it to edit it, or click it to select it and press Delete. **Delete** in the editing box removes it too.
- **Notes belong to a plot.** A note is placed in the data coordinates of the plot it was made on, so it stays on its feature when you pan and zoom. It is shown only on that plot: the phase plane of that pair of variables, the diagram for that continuation parameter and vertical axis, or that parameter plane. Switch to other axes and the note is hidden, and it comes back when you return. Loading another model removes all notes.
- **In figures and scripts.** Notes are drawn in SVG exports, in the colours of the light theme, along with the legend. A note you make while recording is written into the script as a `note` line (Section 15). Notes are not saved in the `.ode` file.

Right: the inspector. The inspector's contents depend on the view. In the phase plane it lists the equilibria (type, coordinates and eigenvalues, with buttons to make one the initial condition, start continuation from it, or draw its separatrices), the variables in the time course, and the trajectories. For large systems, and for boundary value problems, equilibria are not searched for automatically; **Find connecting orbit** next to the list searches the visible window when you want them. In the Bifurcation view it holds the continuation controls, the search for homoclinic and heteroclinic orbits, and the table of special points. In the Two-parameter view it holds the two-parameter controls.

Key	Action
G	integrate from the current initial condition ("go")
E	erase all trajectories
N / D	toggle nullclines / direction field
F	fit the axes to the data
1–7	phase plane, 3D, time course, bifurcation, two-parameter, phase reduction, animation
Esc	during continuation: skip to the other direction ("soft interrupt")
Shift+Esc	stop the continuation altogether
click / shift-click	start a trajectory (shift: in both directions of time)

Table 1: Keyboard shortcuts and mouse actions.

3 The model language

A model is a plain-text `.ode` file in XPPAUT's format. Lines starting with `#` are comments, and the first comment line becomes the model's title. Names are case-insensitive, as in XPPAUT: `V` and `v` are the same variable. A file that writes the same name with different capitals gets a warning when it is loaded, since the two were probably meant to differ. The Morris–Lecar example [10, 11] illustrates most of the basic syntax:

```
# Morris-Lecar neuron, Hopf regime (Rinzel & Ermentrout)
dv/dt=(i+gl*(vl-v)+gk*w*(vk-v)+gca*minf(v)*(vca-v))/c
dw/dt=phi*(winf(v)-w)/tauw(v)
minf(v)=.5*(1+tanh((v-v1)/v2))
winf(v)=.5*(1+tanh((v-v3)/v4))
tauw(v)=1/cosh((v-v3)/(2*v4))
aux ica=gca*minf(v)*(v-vca)
param i=60,c=20
param vk=-84,vl=-60,vca=120
```

```
param gk=8,gl=2,gca=4.4
param v1=-1.2,v2=18,v3=2,v4=30,phi=.04
init v=-60,w=0.015
#@range i 0 300
@ total=400,dt=0.25,meth=rk4,xp=v,yp=w
@ xlo=-75,xhi=45,ylo=-0.1,yhi=0.55
done
```

Table 2 summarises the language. The `#@range` line is an XPP Studio extension that sets a slider's range. Because it starts with `#`, XPPAUT reads it as a comment, so files stay portable.

Methods. `meth=` accepts `rk4` (fixed-step Runge–Kutta), `euler`, `dp45` (adaptive Dormand–Prince 4(5)), and the stiff names `stiff`, `gear`, `cvode`, `rosenbrock` and `backeul`. All of the stiff names select an adaptive Rosenbrock 2(3) solver [8]. `discrete` iterates a map. Models with Wiener variables always use Euler–Maruyama, and models with Volterra integrals use a product-trapezoid scheme.

Sets. A *set* is a named collection of parameter values, initial values and numerical settings. It is written as in XPPAUT, one line for each set, with the items separated by commas or semicolons:

```
set fast {a=4, b=2, x=0.5, total=50, dt=0.01}
set slow {a=1, b=0.5, y=0.2, total=120}
```

When a file has sets, a **Sets** menu appears at the top of the window. Choosing a set changes everything it lists at once and then runs *once*, rather than once for each value. With **Auto-update** ticked, the current curve is started again from the new initial values and the other curves are recomputed; with it unticked, the display is marked out of date and **Update** computes it. The menu returns to **Sets** after each choice, so the same set can be applied again after you have edited values by hand, and the status line lists what was changed.

- A set may name parameters, state variables (Markov states too) and the integration settings of the Numerics tab: `total`, `dt`, `meth`, `trans`, `bound`, `tol` and so on. Only what a set mentions changes; everything else keeps its present value, and **Reset** in the Parameters tab still restores the values in the file.
- XPPAUT files often put the plot itself into a set (`xlo`, `xhi`, `ylo`, `yhi`, `xp`, `yp`, `zp`). These are accepted but ignored: the plot stays as you have set it. The status line names them, and any other name it does not recognise.
- Set names are not case-sensitive. In a script, `use fast` applies a set (Section 15).

Try it

Add two lines before `done` in the Morris–Lecar example and press **Apply**:

```
set rest {i=60, total=400}
set firing {i=150, total=200}
```

Choose `firing` from the **Sets** menu: the current curve is recomputed at $i = 150$ and settles onto the limit cycle of Section 4. Choose `rest`, and it returns to the stable focus. The slider for i moves with each choice.

Syntax	Meaning
`x'=...  or  dx/dt=...`	differential equation
`x(t+1)=...`	map (or use `x'=...` with `@ meth=discrete`)
`x(0)=1  or  init x=1,y=0`	initial values
`par a=1, b=2`	parameters (also `param`, `p`)
`set fast {a=4, x=0.5, total=50}`	a named set of parameter values, initial values and settings (Section 3)
`#@coupling v:  v'-v`	the coupling $G(X, X')$ of two copies of an oscillator, for the interaction function H (Section 18); v' is v of the partner, and the overall strength ϵ is not written in the file
`number c=3`	named constant
`!d=a*b`	derived parameter, recomputed when parameters change
`f(x,y)=...`	user function
`q=...`	fixed quantity, evaluated before the equations
`aux s=...`	auxiliary output, plotted like a variable
`wiener w`	white noise for an SDE, e.g. `x'=-x+s*w`
`markov z 2  {0} {kon}  {koff} {0}`	two-state Markov variable; entry (i, j) is the rate from i to j
`table h % 101 0 6.283 sin(t)`	table sampled from an expression
`table h data.tab`	table read from a file (add it with **Table files**)
`b u-1  b u'-2`	boundary conditions: `u` at the left end, `u'` at the right end (also `bndry`)
`#@irange u lo hi`	slider range for the initial value of `u`
`x[1..20]'=-x[j]+x[j-1]`	array: the line is repeated for $j = 1, \ldots, 20$
`%[1..20] ... %`	repeat a block of lines
`int{K(t,t',x)}`	Volterra integral $\int_0^t K(t, t', x(t'))\, dt'$
`int{K#x}`	convolution $\int_0^t K(t - t')\, x(t')\, dt'$
`int[mu]{...}`	weakly singular kernel, extra factor $(t - t')^{-\mu}$
`u(t)=...int{...}`	integral equation (also `volterra u=...`)
`delay(x,tau)`	delayed value $x(t - \tau)$; τ may depend on the state
`x(0)=0.5+0.2*cos(t)`	history of a delay equation for $t < 0$ (its value at $t = 0$ is the initial condition)
`global 1 x-1 {x=0;y=y+1}`	global flag: when `x-1` crosses zero upwards (1), downwards (-1) or either way (0), make the assignments; `arret=1` stops the run
`#@range p lo hi`	slider range for parameter `p`
`special bob=conv(periodic,50,10,w,u0)`	a convolution with the table `w`; `bob(j)` is entry j. Also `mmult`; see Tutorial 8
`#@ani line .5;.5;...`	one line of an animation script (Section 19); XPPAUT reads it as a comment
`sum(0,50)of(shift(u0,i'))`	the sum of `u0`,...,`u50`: i' runs over the integers from the first limit to the second, and `shift(u0,k)` is the variable k places after `u0` (fixed quantities and parameters too)
`@ total, t0, trans, dt, meth, toler, bound`	integration (`t0` is the start time): duration, transient, step, method, tolerance, bound
`@ xp, yp, zp, axes=3, xlo, xhi, ylo, yhi`	initial plot variables and window
`@ fold=th, tor_per=6.2832`	periodic variable `th` with the given period
`@ seed=7`	fix the random seed
`@ vmaxpts=4000`	memory length for Volterra integrals
`@ ds, dsmin, dsmax, nmax, ntst, parmin, parmax`	continuation numerics, as in XPPAUT's AUTO window
Functions: `sin cos tan exp ln log sqrt abs heav sign max min mod atan2 sinh cosh tanh, if(c)then(a)else(b)`, ...	

Table 2: The `.ode` language understood by XPP Studio.

4 Tutorial 1: phase-plane analysis

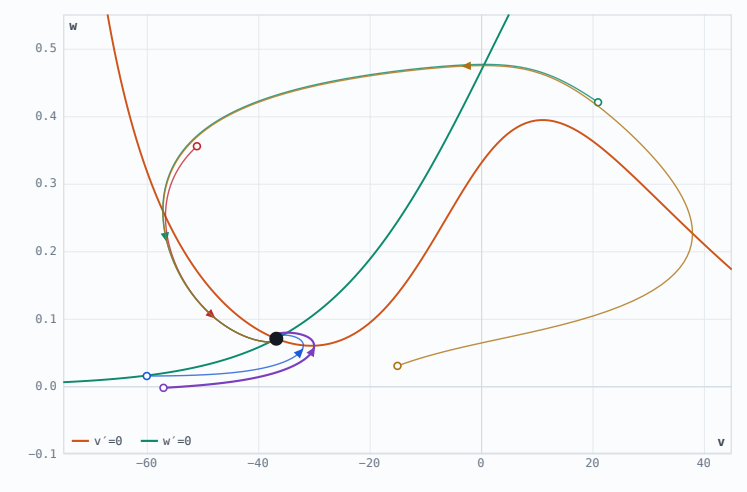


Figure 3: Morris–Lecar phase plane at $i = 60$: the v-nullcline (orange), the w-nullcline (green), a stable focus (filled circle) and several trajectories. Open circles mark starting points.

Choose **Morris–Lecar** from the **Model** menu. The phase plane shows w against the membrane potential v, together with the two nullclines.

Try it

1. Click anywhere in the plane to start a trajectory there. Shift-click integrates in both directions of time. Each new trajectory gets its own colour and appears in the **Trajectories** list, where you can remove it with ×.
2. Press D for the direction field and N to hide or show the nullclines. Hovering over the plot shows the coordinates and the derivatives v', w' at the pointer.
3. The inspector lists one equilibrium, a *stable focus*, with its eigenvalues $\lambda \approx -0.055 \pm 0.063i$. **Set as IC** moves the initial condition onto it.
4. Open the **Parameters** tab and slowly drag the slider for the applied current i. Between about $i = 94$ and $i = 212$ the focus becomes unstable and trajectories settle onto a large limit cycle: repetitive firing.
5. Press 3 for the time course. The chips in the inspector choose which variables are plotted, auxiliary ones included (`ica` here).
6. To label the figure, press **Add label**, click next to the stable focus and type `stable focus`. Drag the label where you want it, and choose a colour in the box if you like. Labels are explained in Section 2.

Equilibria are found by Newton's method from a grid of starting points covering the visible window, so panning or zooming out searches a wider region. For a saddle, the **Separatrices** button integrates its stable and unstable manifolds. Once drawn, they follow the saddle: when you change a parameter, the saddle is found again and its separatrices are recomputed from its new position, so you can watch, for example, a separatrix approach a homoclinic connection as you drag a slider.

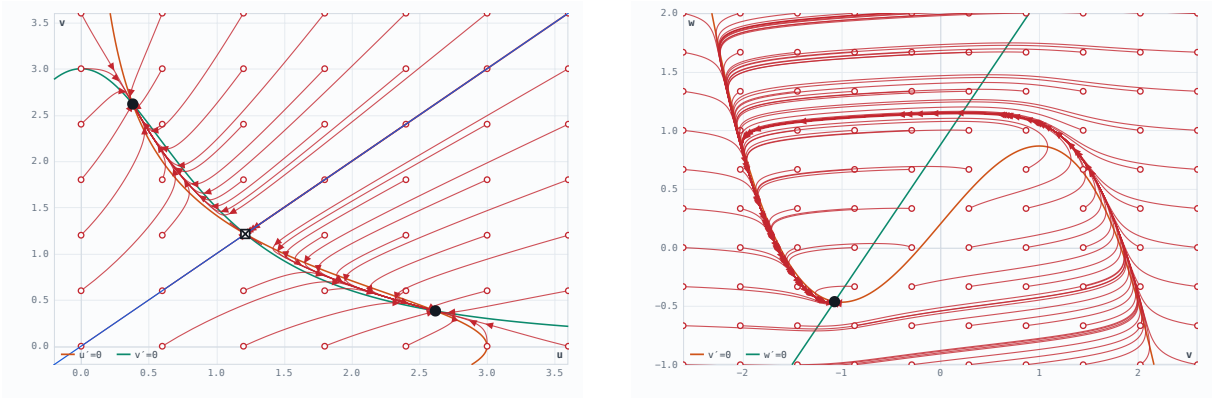


Figure 4: Left: the genetic toggle switch with a 7×7 grid of initial conditions from **Run grid**, u and v each running from 0 to 3.6, and the separatrices of the saddle. Every trajectory ends at one of the two stable nodes, and the stable manifold of the saddle (blue) separates the two basins. Right: **Run grid** with its default settings on the FitzHugh–Nagumo phase plane: trajectories from a 10×10 grid covering the window.

Many trajectories at once: directions and grids

Direction. A run can go forward in time, backward, or in both directions. Choose this in the **Initial** tab, next to **Integrate**, or under **Direction** in the Numerics tab; the two stay in step. *Both* integrates backward and forward from the same point and joins the two halves into one trajectory, with t running from $-T$ to T, so the time course and the arrow on the phase plane show it as a single curve. Trajectories run backward are marked ← in the list and those run both ways ↔. Shift-clicking the plane always runs both ways. Maps, delay and Volterra equations run forward only.

Several initial conditions. XPPAUT's *Initialconds › Range* and *Flow* integrate from many starting values at once. In XPP Studio both are done by the **Several initial conditions** section of the **Initial** tab (Figure 5). Choose one or two variables, each with a start value, an end value and a number of steps, then press **Run grid**; every combination is run, in the chosen direction, while the other variables keep their values from the list above.

By default the grid is 10×10 over the phase-plane window, using the two plotted variables and their axis limits, which gives a quick picture of the whole phase portrait (Figure 4, right). These defaults follow the axes and window as you change them, until you edit a value yourself; **Use window** goes back to them. Set the second variable to *(none)* for a single row of starting points. The runs share one colour and appear in the trajectory list, **Stop** ends a long grid early, and E erases the result. Figure 4 (left) shows a 7×7 grid on the toggle switch.

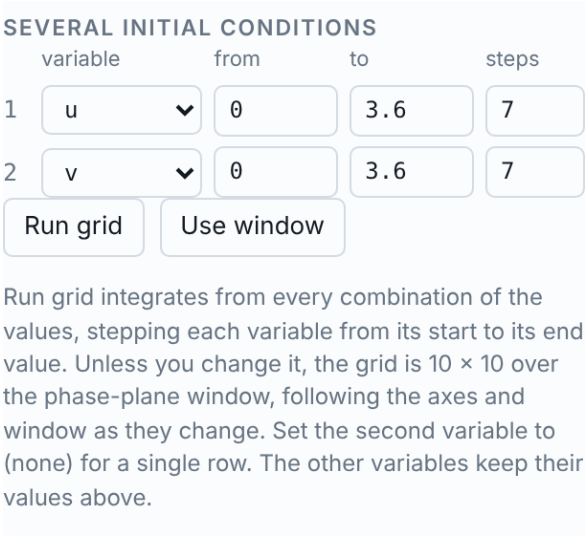


Figure 5: The **Several initial conditions** section of the Initial tab, set for a 7×7 grid in u and v.

5 Tutorial 2: one-parameter bifurcation diagrams

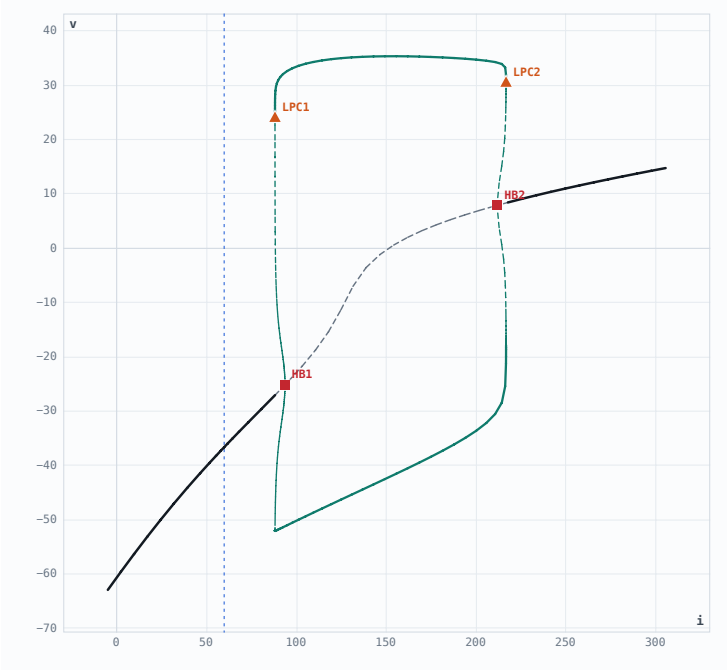


Figure 6: Bifurcation diagram of Morris–Lecar in the current i. Black: equilibria (solid stable, dashed unstable). Green: maximum and minimum of v along the periodic orbits born at the two Hopf points HB1 and HB2. Both Hopf bifurcations are subcritical. The orbit family turns at two folds of cycles, LPC1 and LPC2, giving bistability between rest and firing.

Press 4 for the Bifurcation view. The inspector shows the continuation controls: the **Parameter** to vary, the variable to **Plot**, the parameter **Range**, the initial step **Step ds**, and the equilibrium to **Start** from. A step you type in **Step ds** (or in **Ds** in the Numerics tab) stays as it is when you pan, zoom or change the range; it goes back to the automatic value when you choose another parameter or press **Defaults** in the Numerics tab.

Try it

1. Press **Continue equilibria**. Pseudo-arclength continuation [7] traces the branch in both directions across the range. Stability is read off the eigenvalues, and two Hopf points appear: HB1 at $i \approx 93.68$ (period ≈ 79.2) and HB2 at $i \approx 212.06$. Each Hopf point is tagged *sub* or *super* in the **Special points** table (Figure 10). The tag comes from the sign of the first Lyapunov coefficient ℓ_1, and hovering over it shows the value. Both points here are subcritical ($\ell_1 > 0$), so the orbits born there are unstable.

2. In the **Special points** table, press **Cycles** on the row for HB1. The periodic orbits born there are computed by multiple shooting with an integral phase condition, as in AUTO, and their stability comes from Floquet multipliers. While the computation runs, Esc (or **Skip direction**) abandons the current direction and moves on, and Shift+Esc stops altogether.

3. The family of orbits connects HB1 to HB2 and turns at two folds of cycles: LPC1 at $i \approx 88.29$ and LPC2 at $i \approx 216.90$ (Figure 6). Between LPC1 and HB1 a stable orbit and the stable rest state coexist.

4. Click any point of a branch to jump there. This sets the parameter and loads that equilibrium, or exactly one period of that orbit, as the current state. Clicking a row of the special-points table does the same. Clicking away from every curve moves the parameter to that value (the dashed line) without changing the initial condition, which is a quick way to look at the phase plane at a chosen value.

5. **Limit cycle** starts orbit continuation from the current trajectory instead. Integrate onto a stable cycle first (e.g. at $i = 150$), then press it.

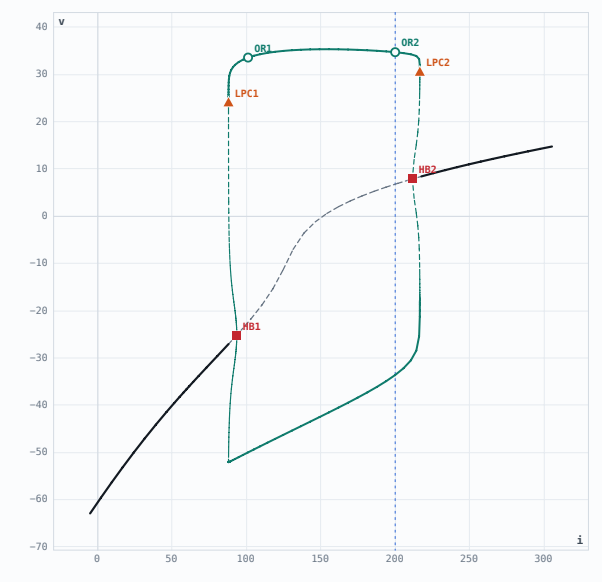
6. **Show** sets what the vertical axis measures for the variable chosen under **Plot**. *hi-lo* draws the maximum and minimum of the variable over each orbit. *L2 norm* draws the norm of the whole solution, $\left(\frac{1}{T}\int_0^T |\mathbf{x}(t)|^2\,dt\right)^{1/2}$ over each orbit, where $|\mathbf{x}|^2$ is the sum of the squares of all the variables, and $|\mathbf{x}|$ at each equilibrium (**Plot** is then greyed out), so a branch of orbits becomes a single curve that leaves the equilibrium branch at the Hopf point (Figure 8). The norms are computed the first time you choose this, by integrating one period of each orbit. *Period* and *frequency* draw the period of the orbits, or its inverse, against the parameter (Figure 9). The period grows from about 79 at HB1 to 135 at LPC1 and is about 42 at HB2. Sweeps show their periods too, where the oscillation is regular enough for one to be estimated. Changing **Show** or **Plot** refits only the vertical axis, to the data in the range of the parameter on screen, so a part of the diagram you have zoomed into stays in view as you switch between hi-lo, L2 norm and period.

7. *hi-slo-lo*, the default for differential equations, is hi-lo with the *slow points* of each periodic orbit added, in amber. One period of each orbit is integrated and its phase-space speed $|F(x)|$ is measured along it, each component scaled by the range of its variable over the orbit so that the units do not matter. A slow point is a local minimum of the speed that is below a threshold, a percentage of the fastest speed on the same orbit; the slowest are kept, at most a set number, and neighbouring minima that are really one place are counted once. They are drawn at the value of the plotted variable, between hi and lo, and joined from orbit to orbit along the branch. Each is shaded by how slow it is: solid where the speed is zero, fading to transparent at the threshold, so the places where the orbits nearly stop stand out from those that only slow down. The line **Slow points** in the panel sets the number (at most 2 by default) and the threshold (5% by default); in a script, `show hi-slo-lo` and `slowpoints max=2 threshold=5%`. A smaller threshold keeps only the places where the orbits nearly stop. An orbit

whose speed hardly varies, such as a small orbit near a Hopf point, has none. The slow points show where a family of orbits starts to linger: near an equilibrium as the orbits approach a homoclinic orbit, or near the ghost of one as they approach a SNIC (Section 17).

8. Click a point on the orbit branch. As well as loading that orbit, this adds it to the **Special points** table as OR1, marked on the diagram with an open circle (Figure 7). Click another point to add OR2, and so on. Each picked orbit has a **Const. T** button, which follows it in two parameters at fixed period (Tutorial 3), and a **Remove** button.

9. **Deleting single curves.** Clicking a curve also selects it: the branch, orbit family, sweep or boundary-value branch is outlined in a blue halo (Figure 8), and the status line names it. **Delete curve** under the continuation buttons, or the `Delete` key, removes that curve and its special points, while the rest of the diagram stays. `Esc` clears the selection. **Clear diagram** still removes everything for the current parameter. The Two-parameter view works the same way, with its own **Delete curve** button.



SPECIAL POINTS 8

PT	I	V	PERIOD	
phi = 0.04				
HB1 sub	93.682	−25.321	79.25	Cycles / Follow
HB2 sub	212.06	7.8003	42.33	Cycles / Follow
LPC1	88.293	23.925	135.4	Follow
LPC2	216.9	30.397	77.93	Follow
OR1	101.44	33.496	83.99	Const. T / Remove
OR2	200.31	34.634	65.68	Const. T / Remove
phi = 0.06				
HB3 sub	97.685	−23.911	65.86	Cycles / Follow
HB4 sub	207.07	7.3407	35.18	Cycles / Follow

Figure 7: Left: two periodic orbits picked on the Morris–Lecar orbit branch, OR1 at $i \approx 100$ ($T \approx 84.0$) and OR2 at $i \approx 200$ ($T \approx 65.7$). Right: the **Special points** table, grouped under the values of ϕ at which the branches were computed, with the picked orbits after the folds, each with **Const. T** and **Remove**. **Follow** is always the last button of a row.

Sweep max/min provides a brute-force alternative. It integrates at a grid of parameter values, carrying the end state forward. The grid has the same step, the range divided by **Sweep samples**, in both directions from the current value, so the whole range is covered evenly wherever the sweep starts. It plots every local maximum and minimum after the transient. This is the right tool for chaotic or stochastic regimes, where continuation does not apply (Tutorial 4).

Sub- or supercritical? At a Hopf point with eigenvalues $\pm i\omega$, XPP Studio computes the first Lyapunov coefficient

$$\ell_1 = \frac{1}{2\omega}\,\mathrm{Re}\Big[\langle p, C(q,q,\bar{q})\rangle - 2\langle p, B(q, A^{-1}B(q,\bar{q}))\rangle + \langle p, B(\bar{q}, (2i\omega I - A)^{-1}B(q,q))\rangle\Big],$$

where A is the Jacobian, $Aq = i\omega q$, $A^{\mathsf{T}}p = -i\omega p$ and $\langle p, q\rangle = 1$. This is the projection formula of [6, Section 5.4, p. 180]. The multilinear forms B and C (the second and third

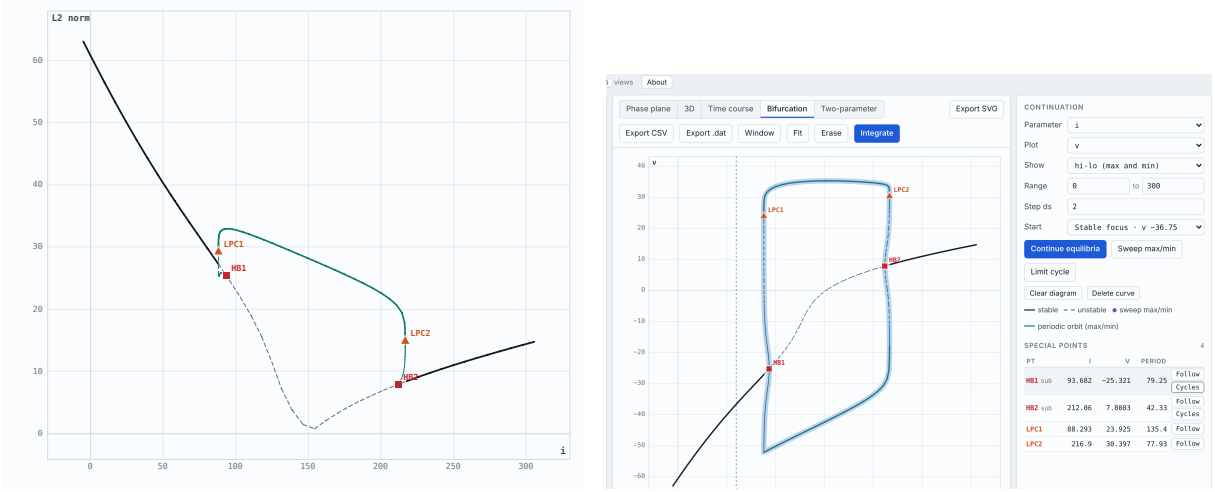


Figure 8: Left: the Morris–Lecar diagram with **Show** set to *L2 norm* (of the whole solution, v and w together). The orbit family is now one curve from HB1, through the folds LPC1 and LPC2, to HB2. Right: the orbit family selected by clicking it, outlined in blue, with **Delete curve** enabled under the continuation buttons.

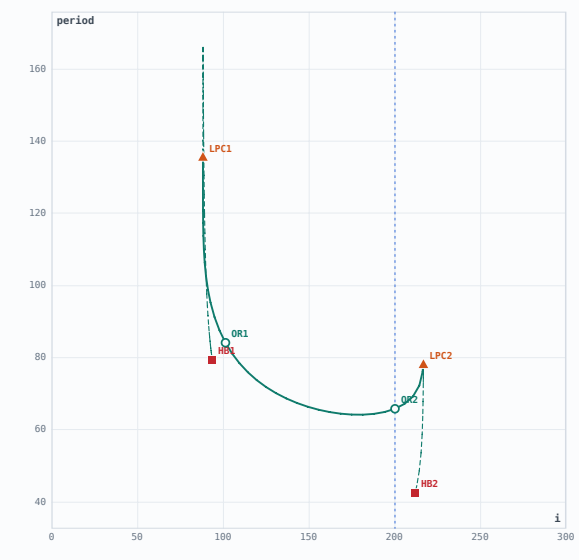


Figure 9: The same diagram with **Plot** set to **period**: the period of the periodic orbits against i, with the special points and the picked orbits.

SPECIAL POINTS 2

PT	I	V	PERIOD	
phi = 0.04				
HB1 sub	93.682	−25.321	79.25	Cycles Follow
HB2 sub	212.06	7.8003	42.33	Cycles Follow

Figure 10: The **Special points** table after continuing equilibria of Morris–Lecar. Both Hopf points are tagged *sub* (subcritical). The points are listed under a heading with the value of the second parameter of the pair (here $\phi = 0.04$); branches computed at several values of it give one group each, in increasing order. **Cycles** continues the periodic orbits born at a point, and **Follow**, always the last button, continues it in two parameters.

derivatives of the vector field) are evaluated by finite differences. If $\ell_1 < 0$ the bifurcation is *supercritical*: a small stable orbit appears on the side where the equilibrium is unstable. If $\ell_1 > 0$ it is *subcritical*: an unstable orbit appears on the side where the equilibrium is stable, and the system usually jumps to a distant attractor.

What is detected. For equilibria of flows: folds (LP), Hopf points (HB, tagged sub- or supercritical) and branch points (BP). For periodic orbits: folds of cycles (LPC), period doublings (PD) and torus (Neimark–Sacker) points (TR). For fixed points of maps: LP, PD and Neimark–Sacker (NS). At a PD of cycles, **Double** switches to the period-doubled branch, and at a branch point **Switch** follows the other branch through it. A branch that comes back to its starting point, such as an isola, is closed into a loop and reported as such.

6 Tutorial 3: two-parameter continuation

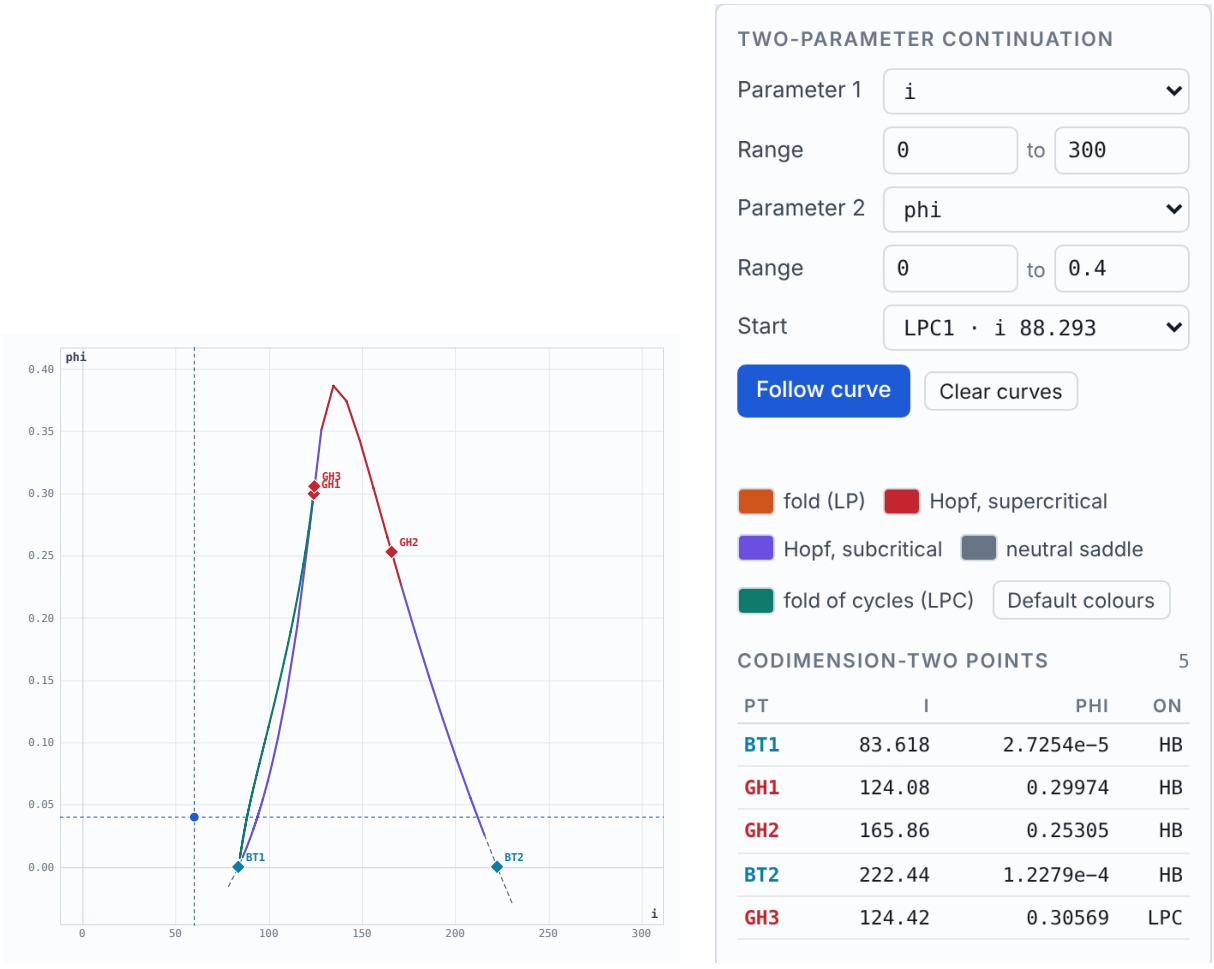


Figure 11: Left: two-parameter diagram for Morris–Lecar in (i, ϕ). A single Hopf curve joins HB1 and HB2. It is purple where the Hopf bifurcation is subcritical and red where it is supercritical, and the colour changes at the generalised Hopf (Bautin) points GH1 and GH2. Dashed ends are neutral saddles beyond the Bogdanov–Takens points BT1 and BT2 near $\phi = 0$. The green fold-of-cycles curve from LPC1 ends at GH3, which lies next to GH1. The blue cross-hairs mark the current parameter values. Right: the Two-parameter panel, with a colour picker for each kind of curve and the list of codimension-two points.

Any special point in the table with a **Follow** button can be continued in a second parameter. The Two-parameter view (5) shows the result, and its inspector panel selects the two parameters

and their ranges. The ranges follow the window: panning or zooming the two-parameter plot, or setting its **Window**, sets them to what is on screen, so a curve that runs off the edge can be followed further by zooming out and pressing **Follow** again.

Try it

1. After Tutorial 2, choose the second parameter (here ϕ, the rate of the recovery variable) in the Two-parameter panel.
2. Set the range of ϕ to 0–0.4. Then press **Follow** on HB1 (in the Bifurcation view, or choose HB1 under **Start** and press **Follow curve**). Hopf curves are computed from a bordered system with the bialternate product [5, 6]. Here a single closed arch joins HB1 to HB2, so following HB2 as well would only retrace it.
3. The first Lyapunov coefficient ℓ_1 is computed at every point of the Hopf curve, and the curve is coloured by its sign: purple for subcritical, red for supercritical (Figure 11). Where ℓ_1 changes sign there is a *generalised Hopf* or *Bautin* point, labelled GH. For Morris–Lecar these are at about $(i, \phi) = (124.1, 0.300)$ and $(165.9, 0.253)$, so the Hopf bifurcations are supercritical only near the top of the arch. Bogdanov–Takens (BT), cusp (CP) and zero-Hopf (ZH) points are flagged in the same way. For maps, strong resonances R1 and R2 and fold–flip points are flagged too.
4. Press **Follow** on LPC1 to continue the fold of cycles. Its curve is computed by multiple shooting and is slower, typically 10–20 seconds. Bautin theory predicts that a curve of folds of cycles is born at every GH point, and here the LPC curve runs up the left side of the arch and ends at GH3 $\approx (124.4, 0.306)$. This is the endpoint of the orbit computation and agrees closely with GH1 from ℓ_1, an independent check of both.
5. **Step ds** in the Two-parameter panel sets the first step along fold, Hopf and period-doubling curves. Left empty, it is 1/100 of the smaller of the two ranges when a curve is started; a value you type stays as it is when you pan or zoom (which changes the ranges).
6. Each kind of curve has a colour swatch in the panel legend: fold, sub- and supercritical Hopf, neutral saddle, fold of cycles, constant period and period doubling. Click a swatch to choose a new colour. The choice applies at once, is used by **Export SVG**, and is remembered in your browser. **Default colours** goes back to the theme colours.
7. Click a point on any curve to move to those parameter values. Then return to the phase plane or the bifurcation diagram to see the dynamics there.
8. **Curves of constant period.** Pick one or more orbits on the bifurcation diagram (Tutorial 2) and press **Const. T** on one of them, or choose it under **Start** as "OR1 (T=...)" and press **Follow curve**. The orbit is continued in both parameters with its period held fixed: the unknowns are the orbit, i and ϕ, and the extra equation is $T = T_0$. In Figure 12 the two gold curves are the parameter values where Morris–Lecar oscillates with period 84.0 and with period 65.7. Each ends where the orbit shrinks onto an equilibrium at a Hopf point whose period $2\pi/\omega$ equals T_0, so constant-period curves end on the Hopf curve. Integrating at points along a curve gives the same period to within the output spacing.

Curves of period doublings of cycles (PDC) are continued the same way, which gives the classical period-doubling tongues of the Rössler system.

Legends, slices and a third parameter.

- **Legend**, above the Bifurcation and Two-parameter plots, adds a key with one line per

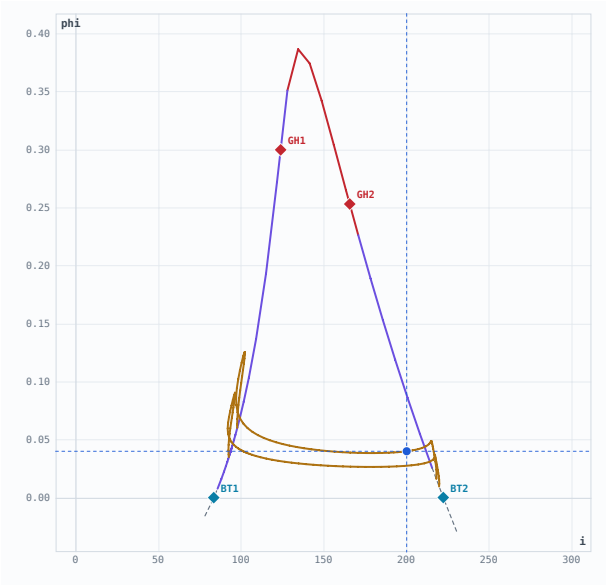


Figure 12: Curves of constant period (gold) for Morris–Lecar in (i, ϕ), traced from the picked orbits OR1 ($T \approx 84.0$) and OR2 ($T \approx 65.7$), together with the Hopf curve, coloured by criticality. Each constant-period curve ends on the Hopf curve, where the frequency of the Hopf point matches the fixed period.

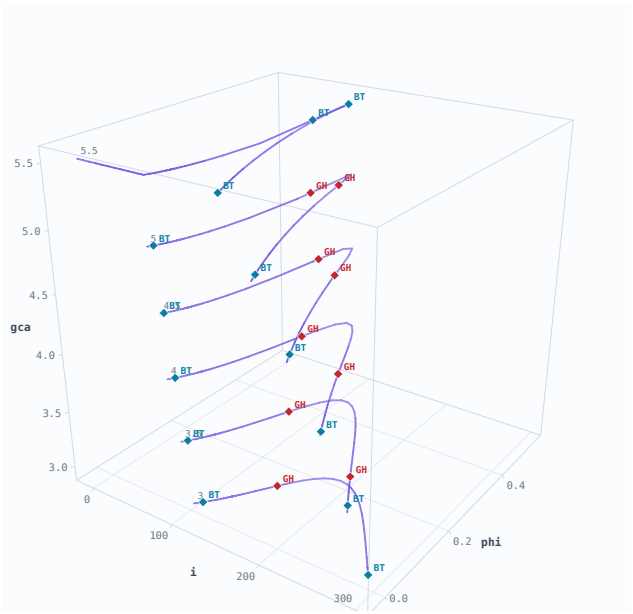


Figure 13: Slices: the Hopf curve of Morris–Lecar in (i, ϕ) followed at six values of g_{Ca} from 3 to 5.5 and drawn in 3D, each curve at the height of its value of g_{Ca}. The generalised Hopf points (GH) and Bogdanov–Takens points (BT) on each slice trace out how they move with the third parameter.

curve: what it is, where it was started from, and any parameter values that differ between curves. The legend is included in **Export SVG**. It starts in the top right corner of the plot; drag it elsewhere, and double-click it to send it back. **Colours**, next to **Legend**, gives every curve its own colour instead of colouring by kind, so that the legend lines can be told apart.

- **Slices in a third parameter**, at the bottom of the Two-parameter panel, repeat the curve chosen under **Start** at several values of a third parameter. Choose **Parameter 3**, its smallest and largest values and the number of slices, and press **Compute slices**. Each slice starts from the special point on the previous one, so the curves are followed continuously as the third parameter changes. This works for fold and Hopf curves, and for folds of cycles that start from a Hopf point of the diagram.
- **Stacked one-parameter diagrams.** The two parameters of the Two-parameter panel make a pair. When curves of the Bifurcation view have been computed at different values of the second parameter of the pair, they are not drawn on top of one another: there is one diagram for each value, stacked like cards with the others greyed out behind, and a row of tabs above them labelled with the second parameter and its value, in increasing order (Figure 14). The diagram on top, outlined in blue, is the one at the current value of the second parameter; the curves of the diagrams behind it show through faintly, all in the same pale grey and without their special points, so that you can see how the diagram changes. Clicking a tab sets the parameter to that value and brings its diagram to the top. While there are several values, the **Special points** table has a column for the second parameter and is in increasing order of it, and clicking a row sets both parameters. **Follow**, **Cycles**, **Double** or **Switch** on a point of a diagram that is not on top first brings that diagram to the top (setting the second parameter to its value) and then acts on the point. Choosing the other parameter of the pair for the Bifurcation view swaps the two, and the curves continued in it are then stacked by the value of the first. **Stack**, above the plot, puts all the curves on one diagram and returns to the stack; it is shown when there is more than one value. Choosing a different pair of parameters returns to one diagram, and the next continuation or sweep stacks again. In a script: `stack on` or `stack off`.
- **3D**, above the Two-parameter plot, draws the curves in a box with the third parameter as the vertical axis, each curve at the value of that parameter it was computed with (Figure 13). Drag to rotate and scroll to zoom, as in the 3D view of trajectories.

Try it

1. After following HB1 above, set **Parameter 3** to g_{Ca} with values 3 to 5.5 and 6 slices, and press **Compute slices**.
2. Turn on **3D** and rotate the box. As g_{Ca} decreases the arch of Hopf points shrinks, and the two GH points move together.

Try it

1. Load **Interacting saddle-nodes**, open the Bifurcation view, set the range of μ to -8 to 4 and choose γ as Parameter 2 in the Two-parameter panel. Press **Continue equilibria** at $\gamma = 0.6$, then set $\gamma = 0$ and press it again, then $\gamma = 0.3$ and again. Three tabs appear above the diagram, and the diagram for $\gamma = 0.3$ is on top (Figure 14). Click the other tabs in turn, and look at the Special points table, now in order of γ. Press **Follow** on the fold at $\gamma = 0.6$: its diagram comes to the top, and the fold curve is traced in (μ, γ).
2. Press **Stack** to see the three branches on one diagram, and again to go back.

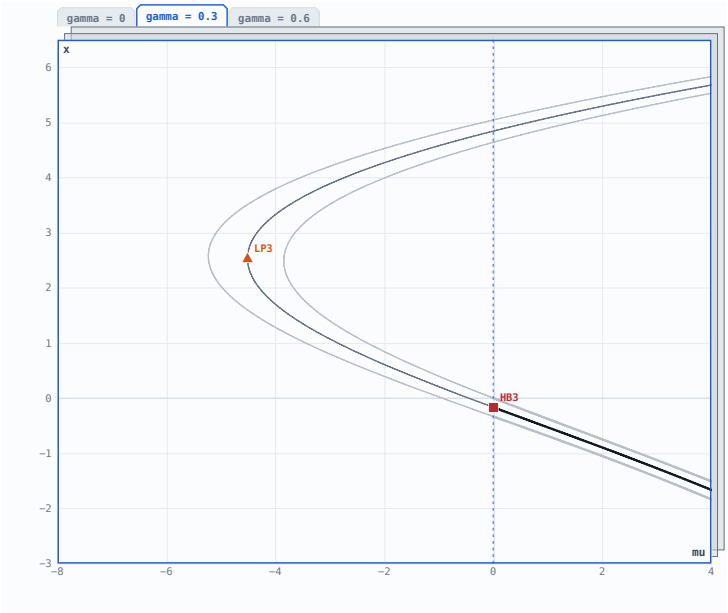


Figure 14: Stacked diagrams: the equilibria of **Interacting saddle-nodes** continued in μ at three values of γ, the second parameter of the pair (μ, γ). Each value has its own diagram and a tab above the stack; the diagram on top is the one at the current value, $\gamma = 0.3$, and the branches at the other two values show faintly in grey on either side. Clicking the tabs shows that the fold LP moves to larger μ as γ decreases.

7 Tutorial 4: chaos in three dimensions

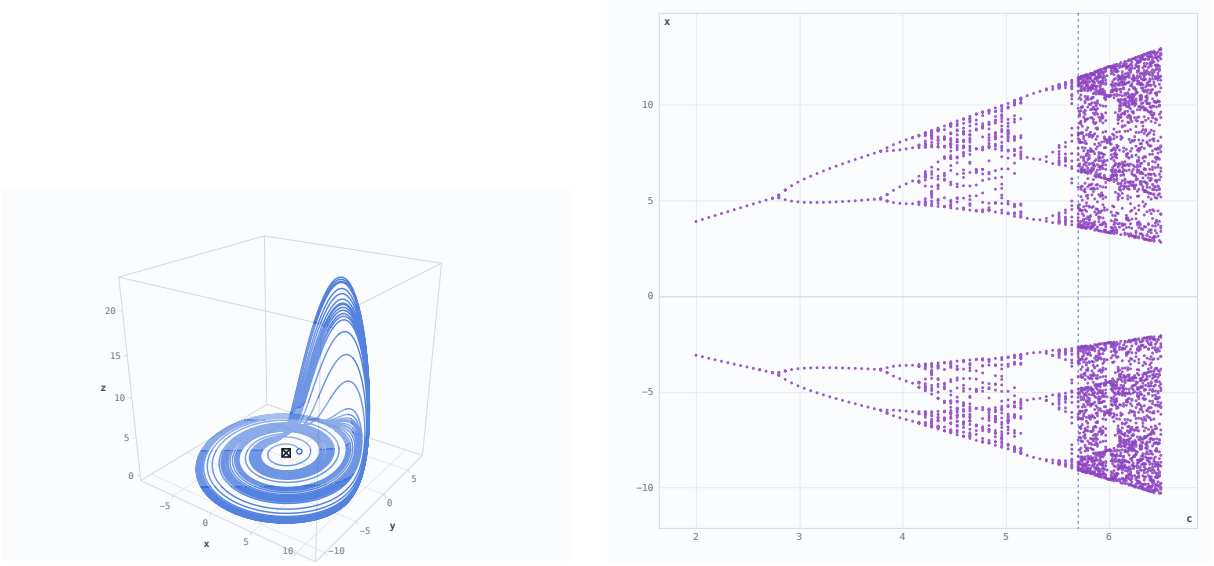


Figure 15: Rössler system at $a = b = 0.2$. Left: the chaotic attractor at $c = 5.7$ in the 3D view. Right: **Sweep max/min** in c, showing the local maxima and minima of x and the period-doubling route to chaos.

Load **Rössler system** [15] and press 2. Drag to rotate the attractor, scroll to zoom, or press **Spin**. The axes are chosen in the toolbar, and any of them can be time.

Try it

1. In the Bifurcation view choose c and press **Sweep max/min** (Figure 15, right). The number of parameter values is set by **Sweep samples** in the Numerics tab.

2. For a continuation version, set $c = 2.5$, integrate onto the stable cycle and press **Limit cycle**. Period doublings are found at $c \approx 2.8324$, 3.8374, 4.1242 and 4.1870. Pressing **Double** at each in turn follows the cascade.

3. **Follow** on a PD point traces its curve in (c, a).

8 Tutorial 5: maps

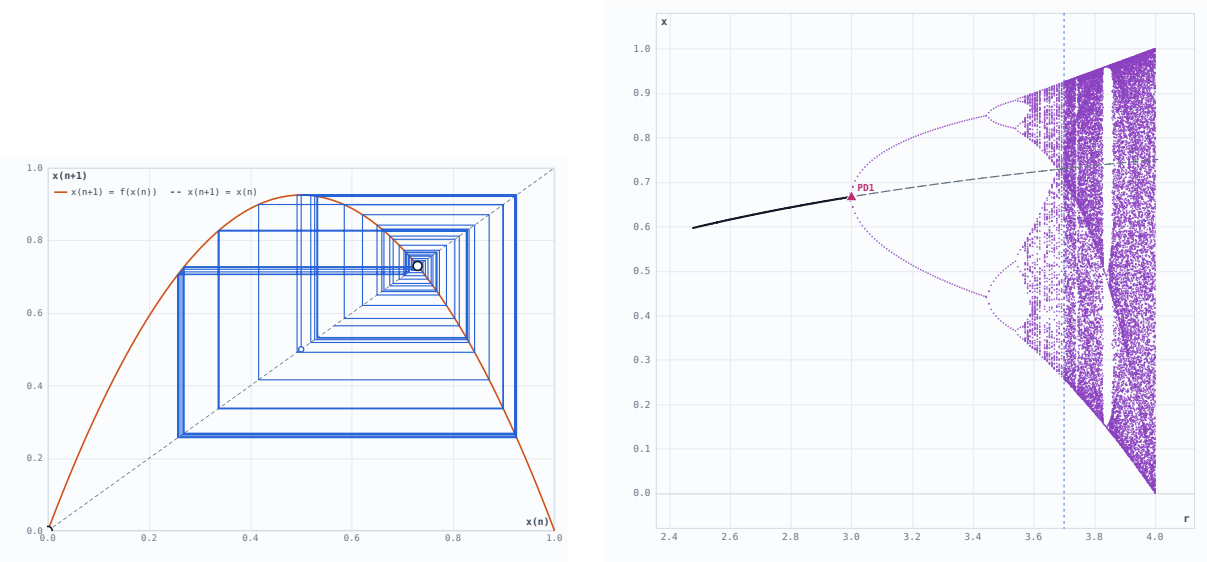


Figure 16: Logistic map $x \mapsto rx(1-x)$. Left: cobweb diagram at $r = 3.7$. Right: orbit diagram from **Sweep max/min** (purple dots), with the continued fixed point and its period doubling PD1 at $r = 3$.

A model is a map when its equations are written `x(t+1)=...`. Trajectories are then drawn as points, and for a one-variable map the phase plane becomes a cobweb diagram (Figure 16). For maps, **Sweep max/min** keeps up to 250 iterates after the transient, which gives the familiar orbit diagram. Continuation finds folds, period doublings and Neimark–Sacker points of fixed points. The **Hénon map** example shows a fold at $a \approx -0.1225$ and a PD at $a \approx 0.3675$, and the **Delayed logistic map** an NS bifurcation at $a = 2$.

Period doublings and orbits of higher period

An orbit of period N of a map f is a fixed point of the N-th iterate f^N, and XPP Studio continues it as one. Its stability comes from the multipliers of f^N, and a period doubling of f^N is where the orbit of period $2N$ is born.

Try it

1. Load **Logistic map**, open the Bifurcation view and press **Continue fixed points**. The fixed point $x = 1 - 1/r$ loses stability at PD1, $r = 3$.

2. Press **Double** on PD1 in the table of special points. The orbits of period 2 are continued, and each is drawn at both of its points, so the branch splits into two. A new period doubling PD2 is found at $r \approx 3.4495$, and the **Period** column of the table gives the period of the orbit it creates, here 4.

3. Press **Double** on PD2, then on PD3 and PD4. The doublings come at $r \approx 3.5441$ and 3.5644, closer and closer together, with ratios of successive gaps approaching

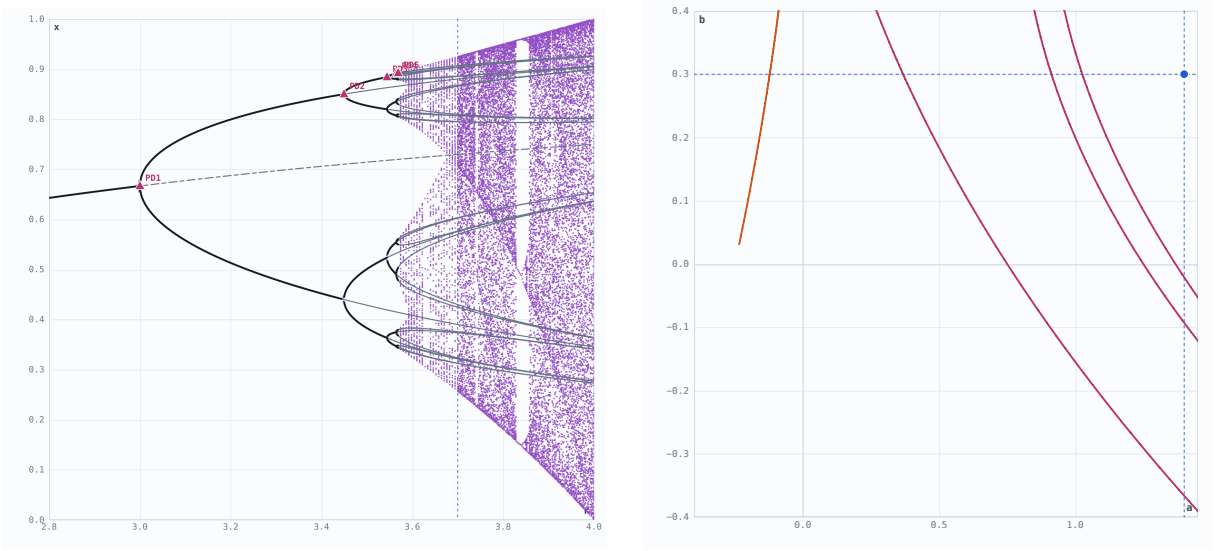


Figure 17: Following period doublings of maps. Left: the logistic map, with the fixed point and the orbits of period 2, 4, 8 and 16 continued by pressing **Double** at PD1 to PD4 in turn, over the orbit diagram. Each orbit is drawn at all of its points; stable parts are solid and black, and each orbit carries on, unstable (grey, dashed), from its next period doubling to $r = 4$. Right: the Hénon map in (a, b), with the fold curve of the fixed point (orange) and the period-doubling curves (crimson) of the fixed point and of the orbits of period 2 and 4, from left to right.

Feigenbaum's constant 4.669 . . . [4]. Press **Orbit diagram** to compare with the attractor (Figure 17, left). Where a continued orbit is drawn dotted, it is momentarily an orbit of lower period.

4. Orbits that are not born in a period doubling can be followed directly. Set **Period** to 3 in the Continuation panel, set $r = 3.84$ and $x = 0.15$, and press **Continue fixed points**. The period-3 window opens at a fold, LP1 at $r = 1 + \sqrt{8} \approx 3.8284$, and its stable orbit doubles at $r \approx 3.8415$. A starting point is found by Newton's method from the initial condition, which may land on an orbit of lower period (a fixed point, say). Tick **minimal period only**, under **Period**, to look only for orbits whose least period is the one asked for: XPP Studio then also tries the points of the orbit through the initial condition and a grid over the phase-plane window, and draws the branch only where its orbits keep that least period (the script command is `period 3 minimal`).

5. **Follow** on a period doubling traces its curve in two parameters, for any period. In the **Hénon map** example, continue the fixed point in a, press **Double** on PD1 and PD2, set the second parameter to b, and follow PD1, PD2 and PD3 (Figure 17, right). For the fixed point the curve is $a = \frac{3}{4}(1 - b)^2$.

6. **Chirikov–Taylor map (area-preserving twist map)** is a map of the torus, $p \mapsto p + \frac{K}{2\pi} \sin q$, $q \mapsto q + p$ with the new p, both taken modulo 2π (`@ fold=`). Press G, then click in the phase plane to start more orbits. With the kick written as $K/2\pi$, $K = 6$ is close to the value at which the last invariant circles running round the torus break up: chains of islands and chaotic layers lie side by side. Smaller K (try 3) gives more invariant circles, larger K (try 12) mostly chaos.

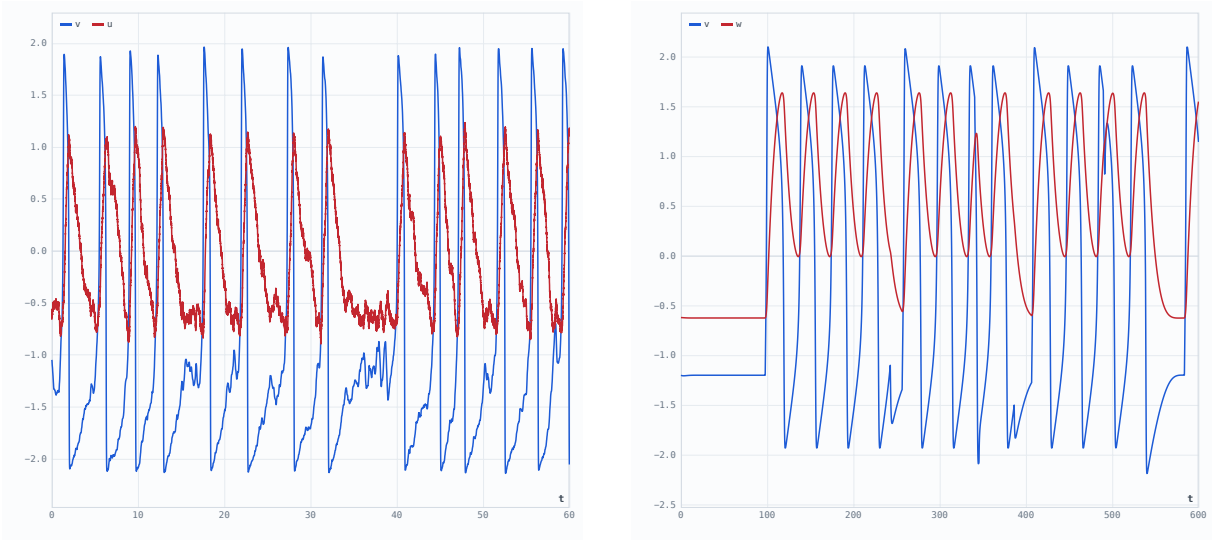


Figure 18: Left: excitable FitzHugh–Nagumo driven by white noise; noise triggers irregular spikes. Right: FitzHugh–Nagumo whose input is a two-state Markov variable, so oscillation switches on and off at random.

9 Tutorial 6: noise and random switching

Wiener noise. Declaring `wiener w` makes `w` a white-noise source. The equations are integrated by Euler–Maruyama with step `dt`, and over each step `w` is $N(0,1)/\sqrt{dt}$, exactly as in XPPAUT. In **Noisy FitzHugh–Nagumo** the rest state is stable, and noise of amplitude `sig` kicks the system over threshold (Figure 18, left). Drag `sig` to change the firing rate.

Markov variables. **Telegraph-driven FitzHugh–Nagumo** declares

```
markov s 2
{0} {kon}
{koff} {0}
```

a variable that jumps $0 \to 1$ at rate `kon` and $1 \to 0$ at rate `koff`. Rates may depend on the state variables. The input `i0+di*s` switches the cell between rest and oscillation (Figure 18, right).

Seeds. Each stochastic run uses a new random seed, which is reported in the status line. Enter a number in **Noise seed** (Numerics tab), or write `@ seed=7`, to repeat the same noise path, for instance to compare parameter values. Equilibria, nullclines and continuation always use the noise-free system, with Markov states held at the values in the Initial tab.

10 Tutorial 7: stiff systems

The **Stiff van der Pol** example has $\mu = 1000$: relaxation oscillations whose slow and fast time scales differ by a factor of about 10^6. With `@ meth=stiff` the Rosenbrock 2(3) solver integrates a period ($T \approx 1614.4$) in a few thousand steps, where an explicit method needs hundreds of thousands. When the stiff method is selected, periodic-orbit continuation also uses it for its shooting segments, so the period can be continued down in μ.

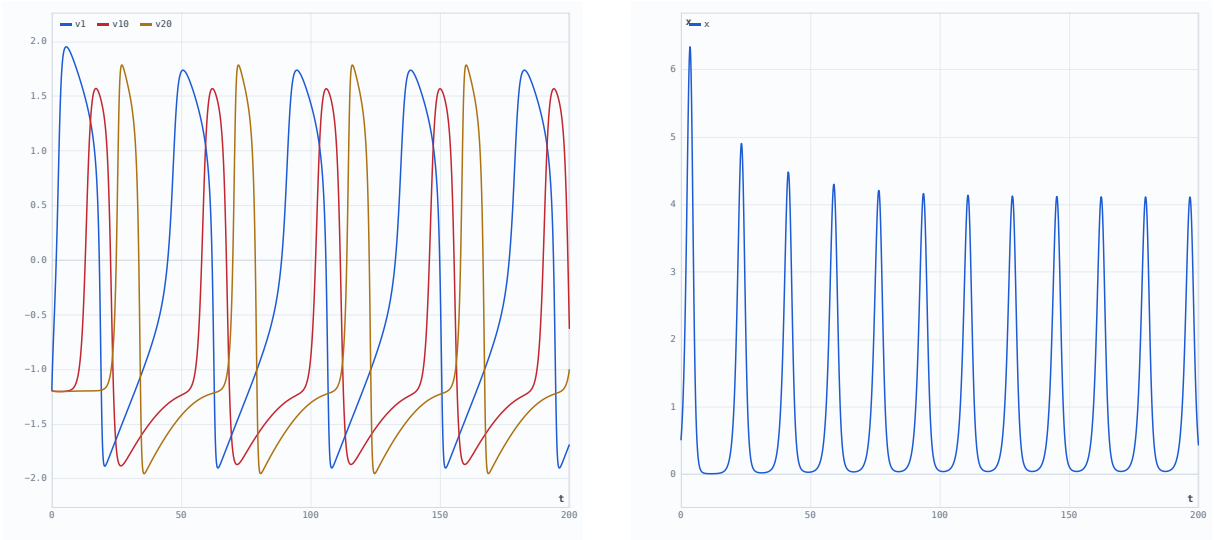


Figure 19: Left: a chain of 20 FitzHugh–Nagumo cells written with arrays. Spikes start at cell 1 and arrive later at cells 10 and 20. Right: logistic growth with a gamma-distributed delay, written as a Volterra convolution. At $r\tau = 2.5$ the equilibrium is unstable and the population oscillates.

11 Tutorial 8: arrays and Volterra equations

Arrays. XPPAUT's arrays are a text macro. The **Chain of 20 FitzHugh–Nagumo cells** example is written as

```
%[1..20]
v[j]'=v[j]-v[j]^3/3-w[j]+i1*heav(1.5-[j])+d*(v[j+1]-2*v[j]+v[j-1])
%
w[1..20]'=eps*(v[j]+a-b*w[j])
v0=v1
v21=v20
init v[1..20]=-1.2
init w[1..20]=-0.62
```

The block between `%[1..20]` and `%` is repeated for $j = 1, \ldots, 20$. Every `[expr]` is replaced by its integer value, so `v[j-1]` becomes `v3` when $j = 4$, and `[j]` on its own becomes the number j. A single line containing `[1..20]` is repeated the same way. The load message reports how many lines the file expanded to. The expanded model is an ordinary 40-dimensional system, so its equilibria, eigenvalues (computed by the QR algorithm) and continuation all work as usual. In Figure 19 (left) the time course shows v_1, v_{10} and v_{20}; choose them with the chips in the inspector.

Sums and convolutions. `sum(lo,hi)of(expr)` adds `expr` for $i' = \texttt{lo}, \ldots, \texttt{hi}$, with i' written `i'` in the expression, and `shift(u0,k)` is the variable k places after `u0`. So `s=sum(0,49)of(shift(u0,i'))` adds $u_0, \ldots, u_{49}$. Coupling every cell to its neighbours this way costs one term for each pair of cells at every step. The `special` command does the same sum in one step:

```
@ autoeval=1
table w % 21 -10 10 exp(-abs(t)/sig)/(2*sig)
special bob=conv(periodic,50,10,w,u0)
par g=3,sig=4
u[0..49]'=-u[j]+g*tanh(bob([j]))
```

Here `bob(j)` $= \sum_{i=-M}^{M} w(i)\, u_{j+i}$ for $j = 0, \ldots, N-1$, with $N = 50$ and $M = 10$ above, and $w(i)$ is the entry of the table `w` in position $i + M$, so the table needs at least $2M + 1$ points (its range

does not matter). Outside $0, \ldots, N-1$, `bob(j)` is 0. The type, which can be shortened to its first letter, says what happens beyond the ends: `periodic` wraps around, `zero` (or `0`) uses 0, and `even` reflects about the first and last cells, so that $u_{-1} = u_1$ and $u_N = u_{N-2}$. The command `special bob=mmult(N,M,w,u0)` is a matrix product, `bob(j)` $= \sum_{i=0}^{N-1} w(i + Nj)\, u_i$, with the table `w` holding the M rows of the matrix one after another. The variables $u_0, \ldots$ may be state variables or fixed quantities, so a neural field can convolve its firing rates `r[0..49]=f(u[j])`. Special functions are computed after all the fixed quantities, so a fixed quantity that calls `bob` sees the values from the previous step, and a warning says so. With the default `autoeval=1` a table is recomputed when a parameter changes; with `@ autoeval=0` it keeps the values it had when the file was loaded, as a random matrix should. Load **Ring of 50 rate units with a convolution** and move the sliders for `g` and `sig`. In **Wilson–Cowan wave front** a stimulus at the left end of a line of 100 cells recruits its neighbours, and the front crosses the line in about 15 time units; the example uses an `even` convolution and carries its own animation (Section 19). In a test with 256 cells and two populations, one evaluation of the equations took 0.45 ms with `special conv` and 11 ms with the same sum written out by `sum` and `shift`.

Space-time plots. For a chain or lattice it is often clearer to see all the cells at once, as in XPPAUT's array plot. In the Time course view, **Space-time** draws a row of indexed variables as an image, with time across and the cell index up (Figure 20). Each point is coloured by its value, and a colour bar shows the range. A *row* is any set of at least six variables with a common name and consecutive numbers, such as $v_1, \ldots, v_{20}$; when a model has several rows, **show** chooses between them. **colour min** and **max** fix the colour range, and **Auto** returns to the range of the run. In the chain the travelling pulses appear as stripes, and their slope is the propagation speed.

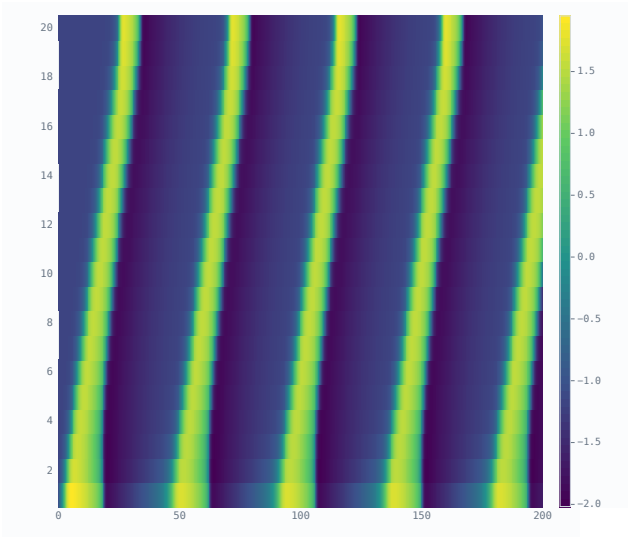


Figure 20: Space-time plot of $v_1, \ldots, v_{20}$ for the chain of FitzHugh–Nagumo cells. Each pulse starts at cell 1 and crosses the chain in about 20 time units.

Spectra. **Spectrum** in the Time course view replaces the time series by the power spectrum of each chosen variable, as amplitude against frequency (cycles per unit of t). A sinusoid of amplitude A gives a peak of height about A. **Log** uses a logarithmic amplitude axis, and **Smooth** averages the spectra of overlapping segments of the run (Welch's method), which helps with noisy or chaotic runs at the cost of coarser frequency resolution.

Volterra integrals. XPP Studio supports XPPAUT's integral terms:

- `int{K(t,t',x)}` $= \int_0^t K(t, t', x(t'))\, dt'$, where the variables inside are evaluated at t';

- `int{K#x}` $= \int_0^t K(t-t')\,x(t')\,dt'$, a convolution with the kernel written as a function of t;
- `int[mu]{...}` adds the weakly singular factor $(t-t')^{-\mu}$, $0 \le \mu < 1$;
- `u(t)=...` or `volterra u=...` defines a variable by an integral equation.

The **Logistic growth with distributed delay** example is

```
x'=r*x*(1-int{t*exp(-t/tau)/tau^2#x})
par r=1,tau=2.5
@ total=200,dt=0.05,vmaxpts=1000
```

The kernel $K(s) = s\,e^{-s/\tau}/\tau^2$ has mean delay 2τ. Linearising about $x = 1$ gives $\lambda(1+\lambda\tau)^2 + r = 0$, so the equilibrium loses stability in a Hopf bifurcation at $r\tau = 2$. Sweeping τ with **Sweep max/min** shows the oscillation appear.

Integro-differential equations are solved by a fixed-step method: the trapezoid rule for the differential part and product-trapezoid quadrature [9] for the integrals. The quadrature weights are exact for the $(t-t')^{-\mu}$ factor, and the implicit terms are found by fixed-point iteration. The error falls as dt^2, or as $dt^{2-\mu}$ with a singular kernel. The method reproduces $u(t) = e^t$ for $u = 1 + \int_0^t u$, and the solution $e^{\pi t}\,\mathrm{erfc}(\sqrt{\pi t})$ of the Abel equation `u(t)=1-int[.5]{1#u}`. It also agrees with the equivalent three-variable ODE for the gamma kernel. Only the last `vmaxpts` steps are remembered (default 4000). Because the state of such a system is its whole history, equilibria, nullclines and continuation are switched off for these models. Trajectories and sweeps work normally.

Periodic variables and tables. Two further XPPAUT features are worth knowing:

- `@ fold=th` keeps an angle in $[0, 2\pi)$ (or $[0,$ `tor_per`$)$). Plots break the trajectory where it wraps, and continuation measures distances modulo the period; see the **Driven pendulum** example.
- Tables (**Phase oscillators with a tabulated H**) are interpolated linearly. They can be sampled from an expression or read from a data file.

12 Tutorial 9: delay equations

A delay-differential equation uses past values of its variables. In XPP Studio, as in XPPAUT, `delay(x,tau)` stands for $x(t-\tau)$, where `x` must be a variable with an equation and `tau` is any expression; it may depend on the state. Load **Delayed logistic equation**, Hutchinson's model [13]:

```
x'=r*x*(1-delay(x,tau))
aux xlag=delay(x,tau)
par r=1.8,tau=1
x(0)=0.5+0.2*cos(3*t)
```

History. A delay equation needs its state on a whole interval $[-\tau, 0]$, not only at $t = 0$. Writing `x(0)` as a formula in `t` gives that history, and its value at $t = 0$ becomes the initial condition. Without a formula, the history is held constant at the initial value. The formulas also appear in the **Initial** tab under **History for t < 0** (Figure 22), where they can be changed or cleared. Clicking the phase plane changes only the values at $t = 0$. The auxiliary variable `xlag` makes $x(t-\tau)$ available for plotting, so the phase plane in Figure 21 shows $x(t-\tau)$ against $x(t)$, a common way to draw the attractor of a scalar delay equation.

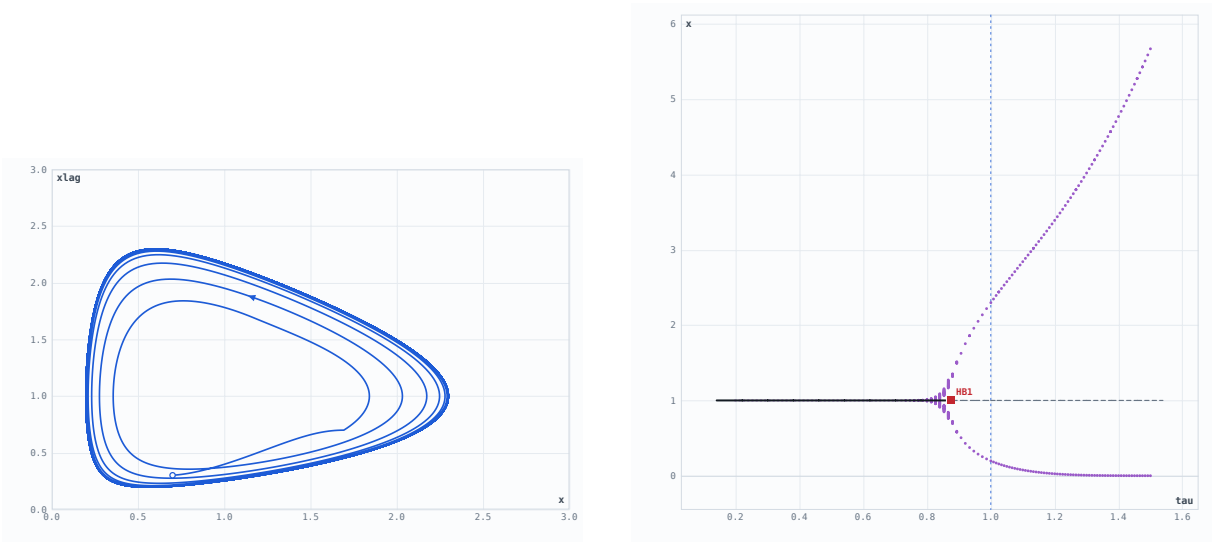


Figure 21: Hutchinson's delayed logistic equation with $r = 1.8$. Left: $x(t-\tau)$ against $x(t)$ for $\tau = 1$, starting from the history $0.5 + 0.2\cos 3t$ and settling onto a periodic orbit. Right: bifurcation diagram in τ. The equilibrium $x = 1$ (black) loses stability at the Hopf point HB1, $\tau \approx 0.874$, close to the exact value $\pi/(2r) = 0.873$. The purple dots, from **Sweep max/min**, show the oscillation that grows from it.

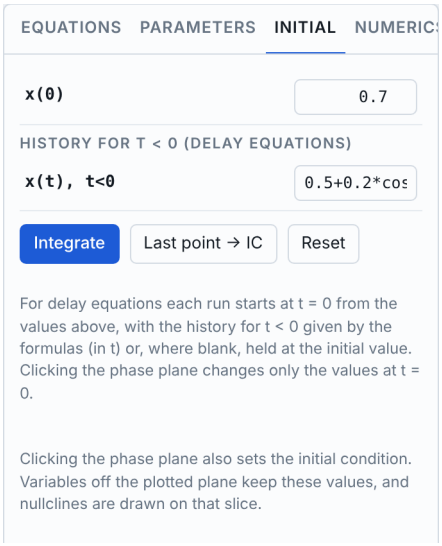


Figure 22: The Initial tab for a delay equation: the value at $t = 0$ and, below it, the history for $t < 0$ as a formula in t.

Integration. Delay equations run forward from $t = 0$ with fixed-step fourth-order Runge–Kutta (adaptive methods fall back to it). Every step is stored, and delayed values are found by cubic Hermite interpolation of the stored solution, or from the history when $t - \tau < 0$. For $x' = -x(t-1)$ with $x = 1$ on $[-1, 0]$, whose exact solution is piecewise polynomial, the computed $x(2)$ and $x(3)$ agree with the exact values to rounding error.

Stability and continuation. Equilibria of a delay equation are those of the system with the delays removed, but their stability is not. It is set by the roots of the characteristic equation

$$\det\Big(\lambda I - A_0 - \sum_k B_k\, e^{-\lambda \tau_k}\Big) = 0,$$

where A_0 and B_k are the derivatives with respect to the current and the delayed values. There are infinitely many roots, but only finitely many lie to the right of any vertical line. XPP Studio

approximates the rightmost ones by Chebyshev collocation of the solution operator [12] and then polishes them with Newton's method. The equilibrium list then reports, for example, *Unstable, 2 roots with Re* $\lambda > 0$ and lists the rightmost roots. **Continue equilibria** uses the same roots, so Hopf points are those of the delay equation.

Try it

1. Load **Delayed logistic equation** and press G. With $r\tau = 1.8 > \pi/2$ the solution settles onto a periodic orbit (Figure 21, left). Change the history formula in the Initial tab and integrate again.
2. The inspector lists two equilibria. $x = 0$ has the single root $\lambda = r$. $x = 1$ has a complex pair $\lambda \approx 0.097 \pm 1.63i$ in the right half-plane.
3. In the Bifurcation view choose `tau`, a range of 0.2 to 1.5 and **Step ds** 0.02, then press **Continue equilibria**. A Hopf point HB1 is found at $\tau \approx 0.874$, against $\pi/(2r) = 0.8727$. **Sweep max/min** adds the oscillation amplitude (Figure 21, right).
4. Load **Mackey–Glass equation** [14] (Figure 23). With $\tau = 17$ the solution is chaotic; the file uses `@ trans=300` to drop the transient. Continuation in τ finds Hopf points at $\tau \approx 4.71$ and 20.98. Linearising by hand gives $\lambda = -0.1 - 0.4\,e^{-\lambda\tau}$, with Hopf points at $\tau = 4.708$ and 20.93.

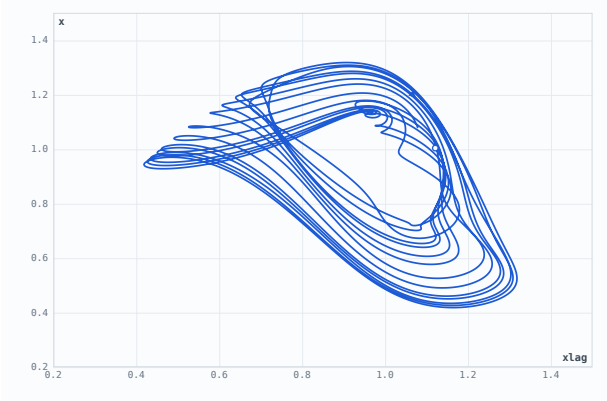


Figure 23: The chaotic Mackey–Glass attractor at $\tau = 17$, drawn as $x(t)$ against $x(t-\tau)$ after a transient of 300.

Periodic-orbit and two-parameter continuation, the direction field and the derivative readout are switched off for delay equations, because the state at time t is a whole function rather than a point.

13 Tutorial 10: events and global flags

Many models change discontinuously when a threshold is reached: a neuron fires and resets, a ball bounces, a switch flips. XPPAUT describes such events with *global flags*,

```
global sign condition {name1=expr1;name2=expr2;...}
```

The assignments are made when `condition` crosses zero from negative to positive (`sign`= 1), from positive to negative (−1), or in either direction (0). All the right-hand sides are evaluated first, then all the assignments are made together. The names may be state or Markov variables, and `arret=1` stops the integration.

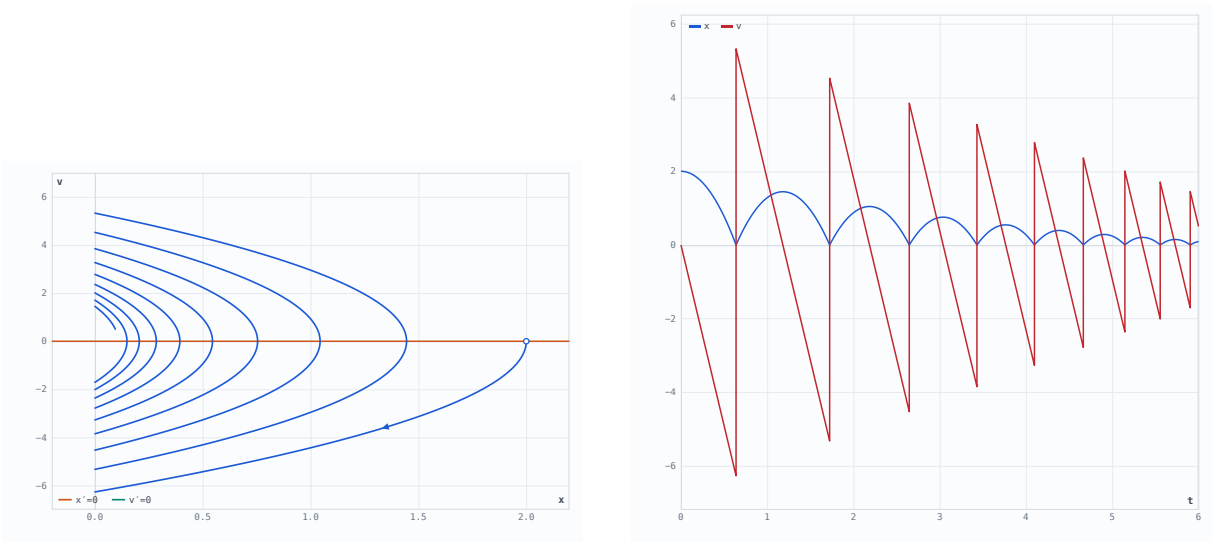


Figure 24: The bouncing ball. Left: velocity against height. Each parabola is one flight, and the line breaks at each event instead of joining the points before and after it. Right: height x (blue) and velocity v (red) against time, with the jump in v at each bounce.

How events are located. After every step XPP Studio checks each condition. If one has crossed zero in its direction, the crossing time is found on the cubic Hermite interpolant of that step, by regula falsi on the condition. The point just before the event and the point just after the assignments are both stored, at the same time, and integration continues from there. This works with every integrator, including the adaptive and stiff ones, and with maps, where the assignments are applied to the new iterate.

Try it

1. Load **Bouncing ball**:

```
x'=v
v'=-g
global -1 x {x=0;v=-e*v}
```

   Press G. The status line reports the number of events. The first two bounces occur at $t = \sqrt{2x_0/g} = 0.6386$ and $t = 0.6386\,(1 + 2e) = 1.7242$, and the computed times agree to six decimal places. Each flight is a parabola in the phase plane (Figure 24). Lower the restitution e with its slider and watch the bounces die away faster.

2. Load **Integrate-and-fire neuron**:

```
v'=(-v+i-w)/tau
w'=-w/tauw
global 1 v-1 {v=0;w=w+b}
```

   Each time v reaches threshold it is reset and the adaptation variable w is kicked up, so the firing rate slows until it settles (Figure 25). With $b = 0$ the interspike interval is $\tau \ln(i/(i-1))$, and the computed value agrees to six or more digits.

3. Add `global 1 t-50 {arret=1}` to a model to stop every run at $t = 50$.

Equilibria, nullclines and equilibrium continuation ignore flags. Periodic-orbit continuation is switched off for models with flags; use **Sweep max/min** for the long-term behaviour instead.

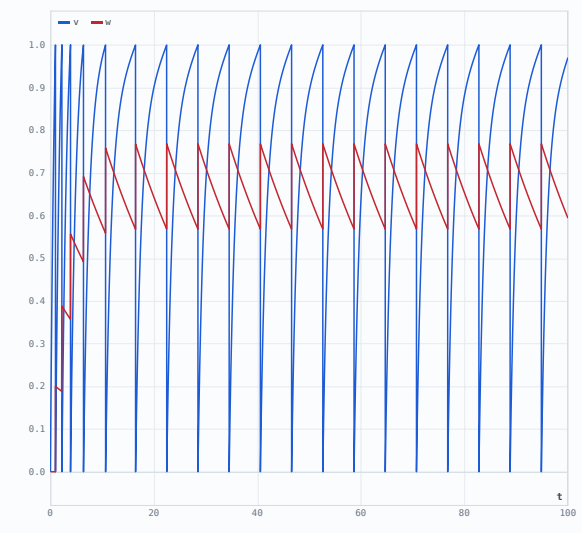


Figure 25: Integrate-and-fire neuron with adaptation: v (blue) is reset to 0 at each threshold crossing, while the adaptation current w (red) builds up and lengthens the interspike interval.

14 Tutorial 11: boundary value problems

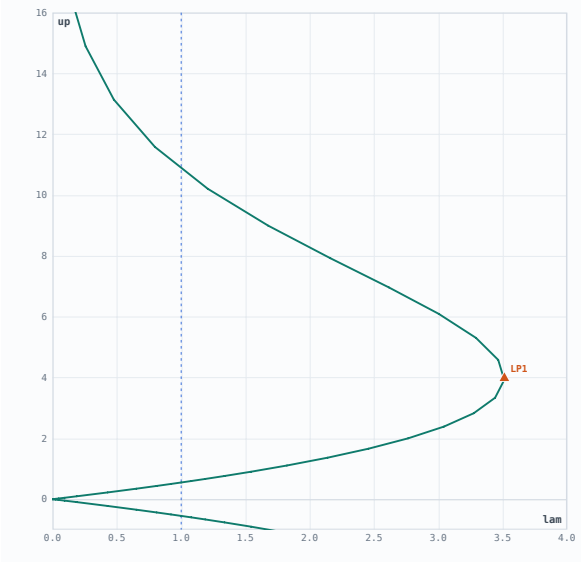


Figure 26: Bratu's problem $u'' = -\lambda e^u$, $u(0) = u(1) = 0$, continued in λ with **Continue BVP**. The vertical axis is the slope $u'(0)$. The branch turns round the fold LP1 at $\lambda \approx 3.5138$ onto the upper solutions.

XPP Studio also solves two-point boundary value problems by shooting, using XPPAUT's syntax. A line `b` (or `bndry`) followed by an expression gives a condition that the expression vanishes. In it, a plain name `u` is the value at the left end and `u'` the value at the right end. The interval runs from `t0` (`@ t0=`, default 0) to `t0` plus the total time. An unknown parameter, such as an eigenvalue, is written as a variable with zero derivative, `lam'=0`, so that the numbers of unknowns and conditions match. `#@irange u lo hi` gives the initial value of `u` a slider in the Initial tab.

Try it

1. Load **Bratu boundary value problem**:

```
u'=up
up'=-lam*exp(u)
b u
b u'
```

   Turn on **Solve BVP** above the plot. Newton's method adjusts the initial values until both conditions hold, and the status line reports the largest residual. Moving the λ slider solves the problem again at each new value. A boundary value problem has no equilibria to look for, so the phase-plane inspector has no **Find equilibria** button and says where to find its solutions instead.

2. In the Bifurcation view set the range of λ to 0–4, choose **Plot** `up`, and press **Continue BVP**. Pseudo-arclength continuation in the initial values and λ follows the branch round the fold at $\lambda \approx 3.5138$ (Figure 26). Clicking the branch loads that solution.

3. The examples **Eigenvalue boundary value problem** and **Rotating spiral wave** have a free parameter written as a variable with zero derivative. The spiral-wave example also starts at $t_0 = 0.01$ to avoid a singular end point.

15 Tutorial 12: scripts for reproducible figures

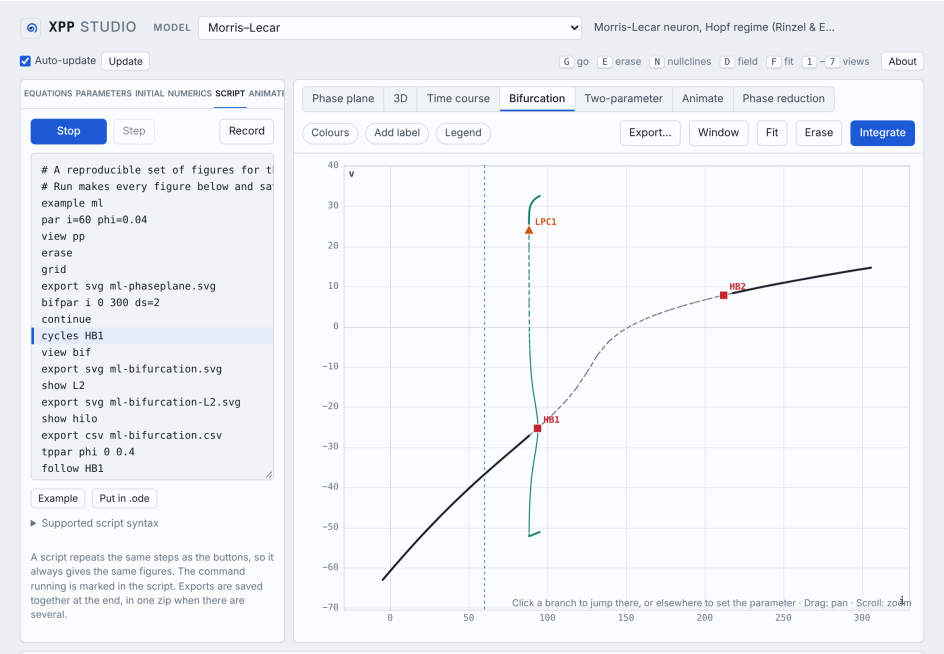


Figure 27: The Script tab while the example script runs. The command being carried out, `cycles HB1`, is marked by a blue bar, and **Run** has become **Stop**; the diagram fills in as the orbits are continued. Errors and the files saved are reported in a log under the script, and **Supported script syntax** below it lists every command.

Each figure in a paper usually comes from a sequence of clicks: load a model, set some parameters, continue a branch, start orbits from a Hopf point, change what is plotted, export. The **Script** tab writes that sequence down as plain text, one command per line, and replays it. The commands call the same functions as the buttons, and each waits for any continuation or

grid of trajectories to finish before the next starts. The same script therefore gives the same figures every time, whatever state the window was left in.

Try it

1. Open the **Script** tab and press **Example**. This script is loaded:

```
example ml
par i=60 phi=0.04
view pp
erase
grid
export svg ml-phaseplane.svg
bifpar i 0 300 ds=2
continue
cycles HB1
view bif
export svg ml-bifurcation.svg
show L2
export svg ml-bifurcation-L2.svg
show hilo
export csv ml-bifurcation.csv
tppar phi 0 0.4
follow HB1
pick 1 i=150
follow OR1
view tp
fit
export svg ml-twopar.svg
```

2. Press **Run** (or Ctrl+Enter in the script). The Morris–Lecar model is loaded, a 10×10 grid of trajectories is drawn, the equilibria are continued in i, the periodic orbits are followed from HB1, the Hopf curve and a curve of constant period are followed in (i, ϕ), and five files are saved together in one zip (Figure 28). It takes about 15 seconds.
3. Change `phi=0.04` to `phi=0.06` and run it again: all five files are remade for the new value.
4. While a script runs, the command being carried out is marked by a blue bar in the script, and **Run** becomes **Stop**.
5. **Step** (or Ctrl+Shift+Enter in the script) runs one command and stops. The next command is marked with a dashed bar, and the files the command exports are saved straight away. If a command fails, correct the line and press **Step** again: the script can be edited between steps, and stepping carries on from the same line. **Run to end** runs the rest, and **Restart** goes back to the first line.

The commands. **Supported script syntax**, under the script, lists them all with a line on each. Names of variables, parameters and special points are those shown in the interface, and `#` starts a comment.

Command	Effect
`example ml`	load a built-in example (by its id, or part of its name)
`apply`	load the model in the Equations tab
`par i=60 phi=0.04`	set parameters
`init v=-20 w=0.1`	set initial values

Command	Effect
`set name=value`	either of the two, whichever the name is
`use fast`	apply a set defined in the equations: its parameters, initial values and settings (Section 3). The plot is left as it is
`sets`	list the sets, with their contents, in the log
`num total=400 dt=0.05 trans=100`	numerical settings; also `meth`, `dir` (forward, backward or both), `tol`, `bound`, `seed`, continuation settings `ds`, `nmax`, `ntst`, ..., and the continuation tolerances `eqtol`, `potol`, `tptol`, `restol`, `itmax`, `tgrow` (Section 20); `auto` empties a box
`view pp|3d|tc|bif|tp|ph`	choose the plot (`ph`: phase reduction)
`axes v w`	phase-plane axes (three names for 3D)
`window -80 50 -0.1 0.6`	axis limits x_{lo}, x_{hi}, y_{lo}, y_{hi}
`fit`	fit the axes to the data
`nullclines on`, `field off`	also `equilibria`, `points`, `equal`
`tcvars v w`	variables in the time course
`spectrum on`, `spacetime off`	also `logscale`, `smooth`, `legend`, `stack`, `tp3d`, `solvebvp`
`rotate 30 20 [1.5]`	3D view: azimuth, elevation (degrees) and zoom
`note 12 0.3 [colour=t2] text`	label at (x, y) of the current plot; `\n` starts a new line. The colour is `accent`, `t1`–`t8` or `#rrggbb`
`notes clear`	remove the notes of the current plot
`erase`, `go [both]`	erase; integrate from the initial values
`last`	initial values from the end of the active trajectory (**Last point** → **IC**)
`reset par`, `reset init`	parameters, or initial values, back to the file's values (the **Reset** buttons)
`traj v=-40 w=0.2 [backward]`	a trajectory from a given point
`grid`, `grid v -60 40 5 w 0 0.5 5`	grid of initial conditions (default 10×10 over the window)
`separatrices [k]`	separatrices of the kth saddle
`findeq`	search the window for equilibria
`bifpar i 0 300 ds=2`	continuation parameter, range and step
`continue [eq k]`	continue equilibria
`cycles HB1`, `double PD1`	orbits from a Hopf point; period-doubled orbits (flows and maps)
`switch BP1`	the other branch through a branch point
`period 3`	maps: continue period-3 orbits (also `bifpar ... period=3`)
`limitcycle`, `sweep`	continue the orbit reached by the trajectory; sweep the range
`adjoint here [align=VAR]`	the adjoint, the isostable functions and H of the limit cycle of the active trajectory at the present parameters, found again after each **Update** that changes the cycle while the Phase reduction view is open; switches to the Phase reduction view; `adjoint follow=off` stops the following (Section 18)
`hfunction show=odd`	the interaction function H of the `#@coupling` lines, recomputed from the stored orbits; `show=all|H|even|odd`, or `off` (Section 18)
`hsecond eps=0.1 [show=odd]`	the second-order interaction function H_2: $H + \epsilon H_2$ and its parts (`show=all|H|even|odd`, or `rate` for ϕ'/ϵ), and γ; with `eps=0`, H_2 alone (Section 18)
`phasefn orbit|Z|I|rho|B|H|H2`	which function the Phase reduction view draws

Command	Effect
`adjoint [K] n=8 range=100,200`	the adjoint (phase response) $Z(t)$ at n orbits spread evenly in the parameter along orbit family K (default: the latest continuation, both directions), within the range; also `at=v1,v2,...`, `align=VAR|none`, `res=`; also computes the isostable functions when they exist (Section 18)
`plot v`, `show L2`	variable, and hi-lo, L2 norm (of the whole solution), period or frequency
`pick 1 i=150`	pick an orbit of family 1 (it becomes OR1, OR2, ...)
`tppar phi 0 0.4 [ds=0.01], follow HB1`	second parameter (and the step of two-parameter curves); follow a special point (OR1: constant period)
`slices HB1 gca 3 5.5 6`	follow HB1 at 6 values of a third parameter
`connect 2`, `connect 3 1 [backward]`, `connect end`	find a homoclinic orbit of equilibrium 2, or a heteroclinic orbit from 3 to 1; `end` is the saddle found at the end of the last branch of orbits (Section 17). `follow HOM1` follows it
`clear bif`, `clear tp`	clear a diagram
`export svg|csv|dat [file]`	save the current view or its data, as **Export...** does
`writestate`, `echo`, `wait ms`	write the state into the file; note; pause
`history save [file]`, `history clear`	save the status-line history as a text file (with the other exports), or empty it

If a command fails (an unknown name, a Hopf point that is not there, a continuation that does not start), the script stops, the line is marked in red, and the log under the script gives the reason. Files exported before then are still saved.

Sets in scripts. Several sets make a short script. This one runs the two sets of the previous section one after the other, with a pause so that each can be seen:

```
erase
use firing
go
wait 2000
use rest
go
```

Each `use` changes all the values of its set together, and `go` starts from them. **Step** runs the script one line at a time, which makes `wait` unnecessary when you are watching. (`use` is not the same as the script command `set name=value`, which sets a single parameter or initial value.) A script always carries out its commands at once, whether or not **Auto-update** is ticked.

Recording. You need not type a script. Press **Record**, then work as usual: choosing a model, moving sliders, changing settings, integrating, continuing, following curves and exporting are written into the script as you go. A slider dragged across many values gives a single `par` line with its final value. Actions are recorded however you start them, from a button, a row of the special-points table, the equilibrium list or a key, and from any view, so **Follow** or **Cycles** in the table gives the same lines as the buttons in the inspector. What you do with the mouse is recorded too:

- clicking the phase plane gives a `traj` line with the starting point (and `both` for a shift-click);
- panning and zooming give a `window` line once the gesture ends, and rotating or zooming the 3D view a `rotate` line, so the figure is framed as you left it;
- clicking a diagram to load a point (an equilibrium, a point on a boundary value branch or a two-parameter curve) gives a `set` line with the parameter values and all the initial values it loads, and clicking an orbit branch gives `pick`;

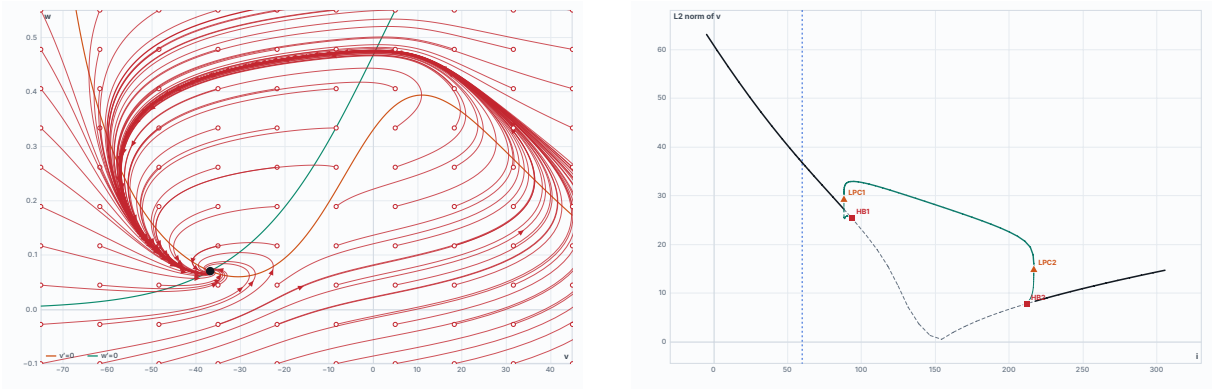


Figure 28: Two of the five figures the example script saves: the phase plane with a 10×10 grid of trajectories, and the bifurcation diagram in i plotted as the L^2 norm of v, with the branch of periodic orbits from HB1 to HB2 and its folds LPC1 and LPC2.

- **Find connecting orbit** for a homoclinic or heteroclinic orbit gives a `connect` line, and **Follow** on HOM1 or HET1 a `follow` line;
- placing a label with **Add label** gives a `note` line with its position, colour and text. Moving or editing the label afterwards is not recorded, and neither is its size;
- switching views, by button or key, gives a `view` line, and the chips above the plot (spectrum, space-time, legend, 3D, and the functions of the Phase reduction view) and the variables chosen for the time course are recorded as well.
- choosing a set from the **Sets** menu gives one `use` line, not a line for each value in it;

Last point → IC gives a `last` line, and the **Reset** buttons of the Parameters and Initial tabs give `reset par` and `reset init`, so a script can carry on from where a trajectory ended or start again from the file's values. Repeated changes of the same thing (dragging a slider, panning in several strokes) give a single line with the final value. Press **Record** again to stop, then edit the result as you like.

If the script box already holds commands when you press **Record**, you are asked whether to clear them or to keep them and start a new block, marked by a dated comment line. A script belongs to a model. When you load a different model while the box holds a script, you are asked whether to keep it or clear it (and, if the new file carries a script of its own, whether to use that one instead); **Restore old script** brings back a script that was cleared.

Scripts inside `.ode` files. **Put in .ode** copies the script into the equations as lines starting `#@do`, just before `done`:

```
#@do par i=60
#@do bifpar i 0 300
#@do continue
#@do cycles HB1
#@do view bif
#@do export svg bifurcation.svg
done
```

XPPAUT reads these as comments. When the equations hold `#@do` lines, a **Load .ode script** button appears in the Script tab. It puts them into the script box, after an `apply` line that reloads the model, ready to **Run** or **Step**, so a single `.ode` file can carry both a model and the figures made from it.

16 Tutorial 13: a forced oscillator and Arnold tongues

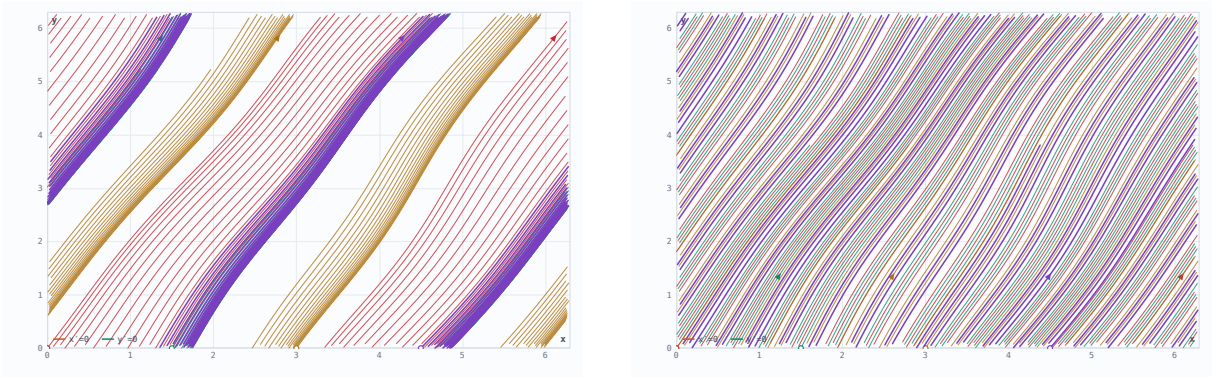


Figure 29: The forced phase oscillator on the torus, x against y (both reduced modulo 2π, so the lines break where they wrap). Left: $\omega = 2$, inside the 1:2 tongue; the trajectories from several starts all close in on the same band, the locked orbit. Right: $\omega = 2.3$, outside the tongue; the lines do not settle onto a closed orbit.

In a periodically forced oscillator, or in a phase model, the state lives on a torus. The **Forced phase oscillator** example has two phases, x for the oscillator and $y = \omega t$ for the forcing:

```
p(x)=-sin(x)/(1+a*cos(x))
x'=1+b*p(x)*cos(y)
y'=om
par om=2,b=.25,a=.25
@ fold=x,fold=y,tor_per=6.283185
```

Both variables are declared periodic with `@ fold=`. Plots break where they wrap, and the closing condition of a periodic orbit is taken modulo the period. A locked solution closes in that sense: $x(T) = x(0) + 2\pi n$ and $y(T) = y(0) + 2\pi m$ with $T = 2\pi m/\omega$. Such orbits are followed by the same continuation as limit cycles. The set of parameters for which a locked solution exists is an *Arnold tongue*, and its two edges are folds of cycles.

Try it

1. Load **Forced phase oscillator** and press G with $\omega = 2$. The trajectory settles onto a locked orbit in which x goes once round while the forcing goes twice (Figure 29, left). Change ω to 2.3 and integrate again to see that it is no longer locked.

2. Return to $\omega = 2$, integrate, and open the Bifurcation view. Set the range of `om` to 1.2–3 and press **Limit cycle**. The locked orbit exists only for ω between 1.9784 and 2.0416: there are two folds of cycles, LPC1 and LPC2, the edges of the 1:2 tongue. The stable and the unstable locked orbit meet at the two folds, so the branch is a closed loop. The continuation stops when it has gone once round, and the status line says that the branch closed up on itself.

3. Open the Two-parameter view, set the range of `b` to 0–0.5, and press **Follow** on LPC1 and on LPC2 in the Special points table. The two curves are the edges of the tongue in (ω, b), and they meet at $b = 0$, $\omega = 2$.

4. To add the 1:1 tongue, press **Clear diagram** in the Bifurcation view. This erases the one-parameter branch and leaves the two-parameter curves. Set $\omega = 1$, integrate, set the range of `om` to 0.5–3, continue in `om` with **Limit cycle** (the folds are at 1.1233 and 0.8684), and follow both folds. Turn on **Legend** and **Colours** (Figure 30).

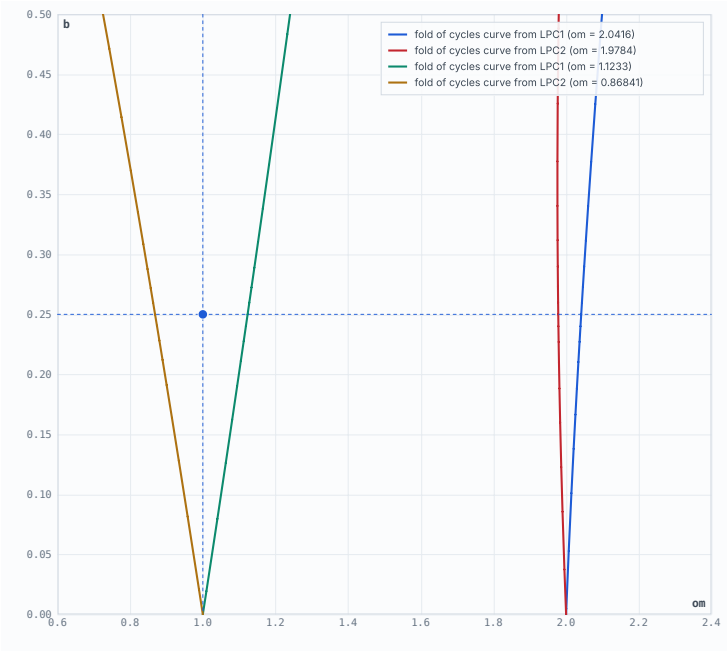


Figure 30: The 1:1 and 1:2 Arnold tongues of the forced phase oscillator in (ω, b), as curves of folds of cycles. Each tongue has two edges, and the legend says which fold and which value of ω each curve was started from. The dashed lines mark the current parameter values.

The tongue is the region between the two edges, widening as the forcing b grows. Narrower tongues, such as 1:3, are harder to reach: at $\omega = 3$ the default run does not settle onto a locked orbit, so you need a longer integration or a starting point on the orbit. One-parameter diagrams add up in the same way: continuing the orbit in ω at three values of b keeps three branches on the same axes, and **Legend** tells them apart by the value of b.

17 Tutorial 14: homoclinic and heteroclinic orbits

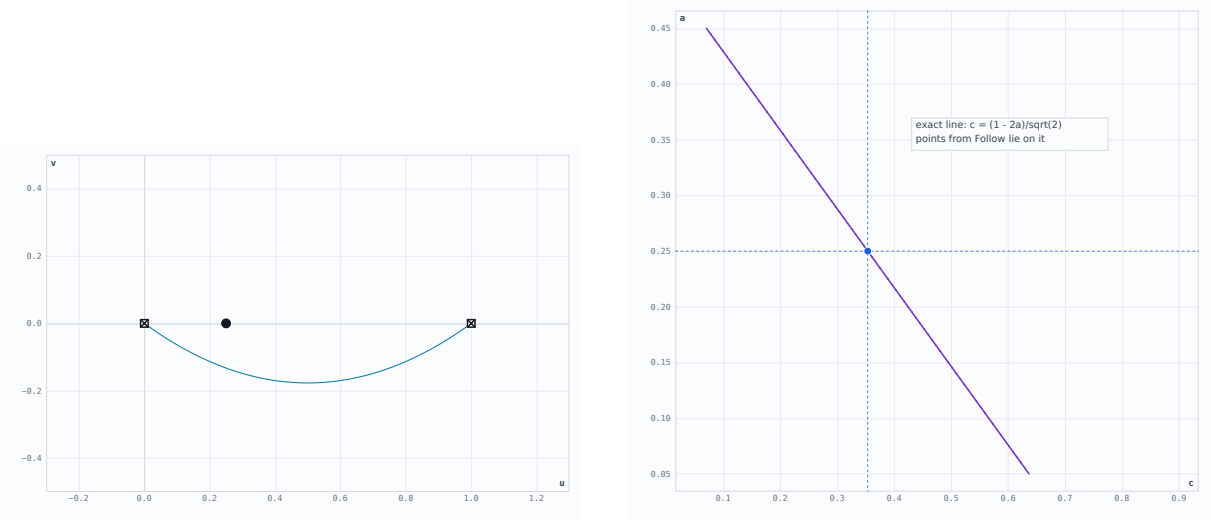


Figure 31: The Nagumo front. Left: the heteroclinic orbit in the (u, v) plane, from the saddle at $u = 1$ to the saddle at $u = 0$ (squares), with the focus at $u = a$ (dot). Right: the speed c at which it exists, followed in a from 0.05 to 0.45. The points lie on the exact line $c = (1 - 2a)/\sqrt{2}$ to about 10^{-9}.

A *homoclinic orbit* leaves a saddle and comes back to it; a *heteroclinic orbit* joins two saddles.

Both exist only for special values of a parameter: they are codimension one, so a one-parameter diagram shows them as single points where a family of orbits ends. They matter for waves. Written in a frame moving with speed c, a travelling front of a reaction–diffusion equation is a heteroclinic orbit of an ordinary differential equation between two rest states, and a pulse is a homoclinic orbit. The speed c is the parameter that has to be tuned until the connection exists.

How the orbit is found. The orbit starts a small distance from the first saddle, along the eigenvector of its single unstable eigenvalue, and is integrated until it comes closest to the second. There the program measures on which side of the second saddle's stable manifold it passes, using the unstable eigenvector of the second saddle. The measure changes sign at a connection. XPP Studio scans the range of the continuation parameter for a sign change, narrows it down by false position (the Illinois variant, which needs far fewer integrations than bisection), and accepts the result only if the orbit really goes on to the target. **Follow** then steps a second parameter and solves again for the first near the previous value, in both directions.

The controls are in the section **Homoclinic and heteroclinic orbits** of the Bifurcation panel, under the continuation buttons. The menus **From** and **To** list the equilibria found in the phase plane at the current parameters, and any saddle found at the end of a branch of periodic orbits (below). **Find connecting orbit** searches the range of the continuation parameter, reporting its progress in the status line. While it searches, the button becomes **Stop**: press it, or Esc, to end a search that is taking too long. **Follow** can be ended the same way: Esc moves on to the other direction, and **Stop** in the Two-parameter view (or Shift+Esc) ends it, keeping the points found so far. The orbit found is drawn in the phase plane and becomes a point of the diagram, HOM1, HET1, ... in the table of special points. On the diagram it is marked by diamonds at the maximum and minimum of the plotted variable along the orbit, at its L^2 norm in the L2 view (for a homoclinic orbit, the norm of the saddle, the limit of the norms of periodic orbits that approach it), at zero frequency, and by a dashed line in the period view, where the period is infinite. Clicking its row moves to it and draws it again. **Follow** in its row continues it in Parameter 2 of the Two-parameter view, over the range set there; the curve is drawn in the Two-parameter view in its own colour, and HOM1 and HET1 also appear under **Start** there.

Try it

The Nagumo front.

1. Load **Nagumo front (heteroclinic orbit)**. The equations are $u' = v$, $v' = -cv - u(1-u)(u-a)$, with the two saddles at $u = 0$ and $u = 1$ and a focus at $u = a$.
2. Open the Bifurcation view; the continuation parameter is c. Under **From** choose the saddle at $u = 1$, under **To** the saddle at $u = 0$.
3. Press **Find connecting orbit**. At $a = 0.25$ the speed is $c = 0.353553392$, the exact value $(1-2a)/\sqrt{2}$ to the digits shown. The phase plane shows the orbit (Figure 31, left), and HET1 is added to the special points.
4. In the Two-parameter view, Parameter 2 is a. Press **Follow** on HET1 in the special points table (or choose HET1 under **Start** and press **Follow curve**). In a second or two the curve of speeds over $0.05 \le a \le 0.45$ appears (Figure 31, right), and a note placed on it (Section 2) can carry the formula.

Try it

A homoclinic orbit of Morris–Lecar, found from the periodic orbits. The Hopf-regime example has no saddle, so a homoclinic orbit cannot exist there. The example

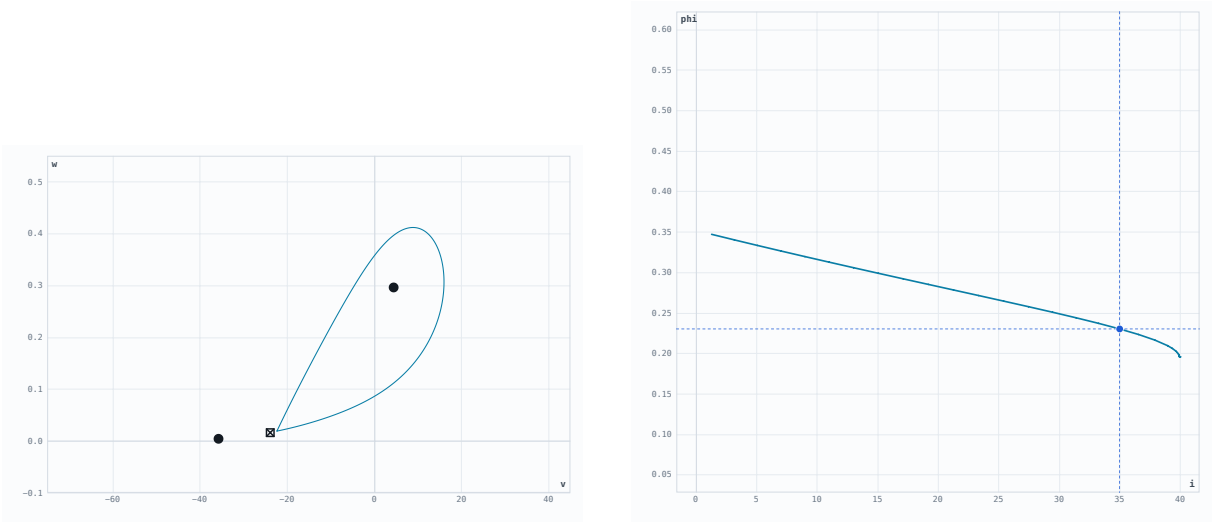


Figure 32: A homoclinic orbit of the Morris–Lecar model with $v_3 = 12$, $v_4 = 17.4$, $g_{Ca} = 4$. Left: at $\phi = 0.23$ the orbit leaves the saddle (square), goes round the focus (upper dot) and returns to the saddle at $i = 35.0067343$. The lower dot is the stable node. Right: the homoclinic curve in (i, ϕ), followed in ϕ. It ends where the saddle merges with the node, near $i = 40$, and where it reaches the left edge of the range.

Morris–Lecar homoclinic orbit (class 1 parameters) has three equilibria at $i = 37$: a stable node, a saddle and a stable focus.

1. Load it and open the Bifurcation view, with the parameter i over its range 0 to 40. Choose the stable focus under **Start** and press **Continue equilibria**: the branch has a subcritical Hopf point HB1 at $i = 36.316$.

2. Press **Cycles** on HB1. The unstable orbits grow, turn at the fold LPC1 ($i = 40.593$) and come back, and their period grows without bound as i approaches 35.0068: the branch stops when the period reaches 40 times its starting value (**Period growth stop**, Section 20).

3. Whenever a branch of periodic orbits ends with its period still growing, XPP Studio looks at its last orbit. Where the orbit moves most slowly it passes close to an equilibrium; Newton's method from that point finds it, and if it is a saddle with one real unstable direction (or one real stable direction, with time running backwards) it is loaded under **From** and **To**, as *end of HB1 orbits: saddle at ...*, the **Find connecting orbit** button turns blue, and the status line says so. Here it is the saddle at $v = -22.3$.

4. Press **Find connecting orbit**. A narrow window round the end of the branch is searched first, and in a second or two the status line reports a homoclinic orbit at $i = 35.0067343$. HOM1 appears at the end of the branch of orbits (Figure 33), and the orbit in the phase plane (Figure 32, left).

5. In the Two-parameter view set Parameter 2 to ϕ (range 0.05 to 0.6), and press **Follow** on HOM1. The curve in the (i, ϕ) plane (Figure 32, right) takes under a minute. It falls from $\phi \approx 0.35$ at small i to $\phi \approx 0.2$ near $i = 40$, where the saddle and the node merge and the connection is lost.

The saddle can also be chosen by hand: it is in the **From** and **To** menus as one of the equilibria of the phase plane.

Homoclinic orbit or SNIC? *hi-slo-lo* under **Show** (Tutorial 2) draws where the orbits of a family move slowly: on the Morris–Lecar example above, a line of slow points appears

near the end of the HB1 branch and runs into the saddle at $v = -22.3$, and in the class 1 case below it runs into the fold. The period of a family of orbits also grows without bound at a *saddle-node on an invariant circle* (SNIC), where a fold of the equilibria appears on the orbit; this is how class 1 neurons start to fire. The period then grows like $|p - p_*|^{-1/2}$, against $-\log|p - p_*|$ at a homoclinic orbit. XPP Studio tells the two apart from where the last orbit moves most slowly. If that is next to a fold LP of an equilibrium branch at the end of the orbits, or there is no equilibrium there at all (the slow "ghost" of a saddle-node just past the fold), or the equilibrium there has an eigenvalue near zero, the status line reports a probable SNIC, and the fold is marked SNIC in the table and on the diagram; nothing is loaded into the homoclinic search. For example, in **Morris–Lecar homoclinic orbit** set $\phi = 0.067$ and $i = 35$, set the range of i to 30–60, and continue the equilibria from the stable node: there is a fold LP1 at $i = 39.963$. Then set $i = 45$, integrate, and press **Limit cycle**. The orbits followed down in i end at LP1, with the period beyond 4000, and LP1 is marked SNIC. Without the branch of equilibria the end is still recognised as a probable SNIC, from the ghost of the saddle-node. The example **Interacting saddle-nodes** (Ashwin, Postlethwaite and Sieber, *Nonlinearity*, 2026) is a planar system in which two saddle-node bifurcations interact, and is a good place to explore how folds, Hopf points and the orbits between them depend on two parameters (see also Figure 14).

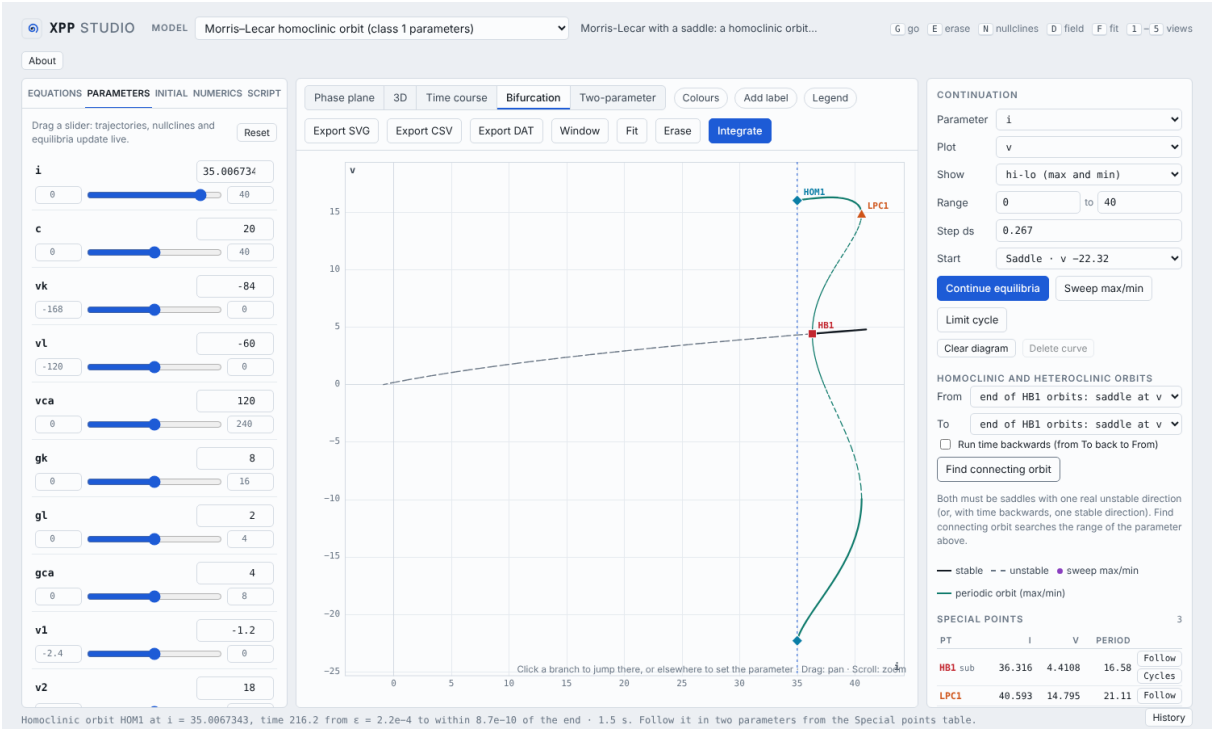


Figure 33: The Bifurcation view after **Cycles** on HB1 and **Find connecting orbit** for the Morris–Lecar homoclinic orbit. The branch of periodic orbits from HB1 turns at LPC1 and ends, with its period growing, at the homoclinic orbit HOM1, marked by two diamonds at the maximum and minimum of v. The section **Homoclinic and heteroclinic orbits** in the right-hand panel holds the saddle found at the end of the branch, under **From** and **To**.

When to run time backwards. The method needs the starting saddle to have exactly one real unstable direction, and the target saddle too, with all the other directions stable. If instead the starting saddle has a one-dimensional *stable* manifold and a bigger unstable one, as can happen at the rest state of a pulse, tick **Run time backwards**. The orbit is then followed from

To back to **From**, which turns the one-dimensional stable manifold into the starting direction.

Limits.

- Both equilibria must be saddles with one real unstable direction (counted backwards in time if **Run time backwards** is ticked). Saddles with complex unstable eigenvalues, or with a two-dimensional unstable manifold at both ends, are not handled.
- **Follow** steps Parameter 2 and solves for Parameter 1, so a curve that turns back in Parameter 2 is lost at the turn. The status line says whether the curve ended at the edge of a range or because the connection was lost.
- The method has been tested on planar problems, where the answers above are exact or checked. For systems with three or more variables, check an orbit by integrating it, and do not rely on a failure to find one as proof that none exists: the range of Parameter 1 and the choice of equilibria matter.
- The orbit is not tested for stability, and codimension-two points are not located on the curve. Orbits found are not saved in the `.ode` file, but a script can find them again with `connect` and follow them with `follow` (Section 15).
- The saddle at the end of a branch is only looked for when the period is still rising over the last few orbits and has at least doubled along the branch. If the branch stopped for another reason first (the point limit, say), raise the limit and continue again, or choose the saddle by hand.

18 Tutorial 15: phase reduction of a periodic orbit

A limit cycle can be described by a single *phase* θ that advances at a constant rate. Weakly coupled oscillators are then studied through how a small push changes the phase: the *adjoint* $Z(t)$, or phase response curve, and from it the *interaction function* H that decides whether two coupled oscillators synchronise [16, 17]. This is the first-order theory. It misses everything that depends on how strongly the oscillators are coupled, because the shape of H cannot change with the coupling strength. The second-order theory of Wilson and Ermentrout [21] adds the decaying *amplitude* of the orbit as a second coordinate (an *isostable* coordinate [19]), and finds how locked states appear, disappear and change stability as the coupling grows.

XPP Studio computes all of this in one place, the **Phase reduction** view (the sixth button above the plot, or the key 6). Everything starts from the **Adjoint** button in its panel, which takes a periodic orbit, or several, and computes the adjoint Z; the slowest Floquet multiplier ν_1 and, when it allows, the isostable response I, the Floquet eigenfunction ρ and a correction B to the phase response; and, if the model has coupling lines, H and the second-order functions g_2, g_3 and the coefficient γ. The chips above the plot choose which of these is drawn.

Adjoint turns blue when there is something new for it: a continued family of periodic orbits for the current parameter, or a trajectory that has settled onto a limit cycle. Its menu chooses between the two, *Current periodic branch* and *Current periodic orbit*, and an entry with nothing to work on is greyed out; until you choose by hand it follows what there is, the branch if there is one and otherwise the orbit. Once **Adjoint** has been pressed it goes back to normal until there is a new family or a new cycle.

The adjoint Z

Let $X' = F(X)$ have a periodic orbit $U(t)$ of period T and write $A(t) = D_X F(U(t))$. The adjoint is the T-periodic solution of

$$Z' = -A(t)^{\mathsf{T}} Z, \qquad Z(t)^{\mathsf{T}} U'(t) = 1. \tag{1}$$

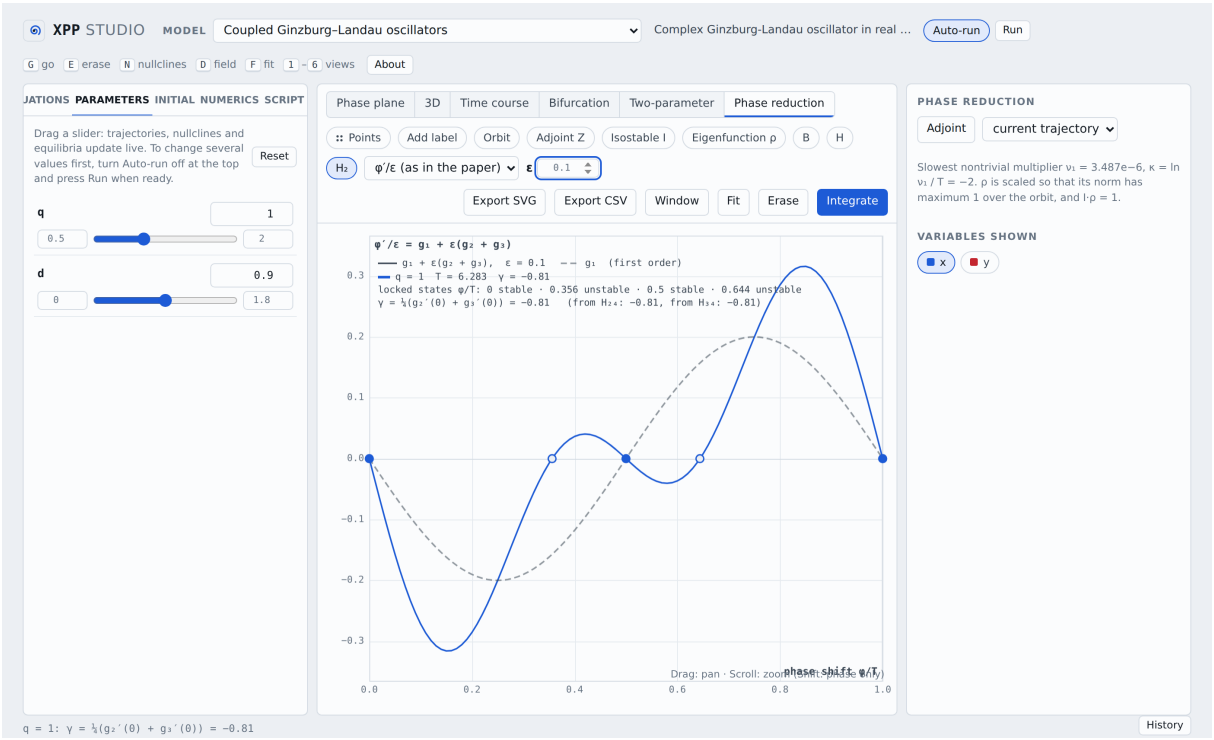


Figure 34: The Phase reduction view, here with the coupled Ginzburg–Landau oscillators of Section 18. The chips above the plot choose what is drawn: the orbit itself, the adjoint Z, the isostable response I, the Floquet eigenfunction ρ, the correction B, the interaction function H and its second-order version H_2, with the coupling strength ϵ beside the last. On the right are the **Adjoint** button, the choice of where the orbits come from, and a note on the slowest Floquet multiplier ν_1.

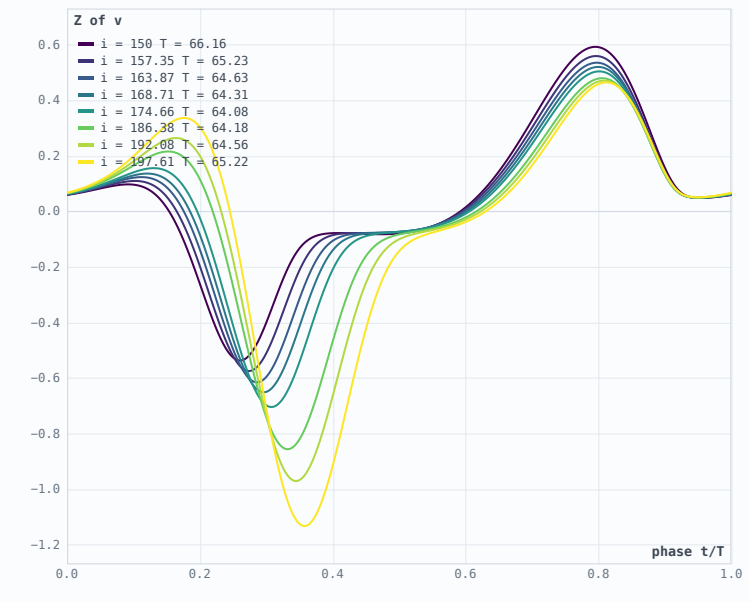


Figure 35: The adjoint $Z_v(t)$ of the Morris–Lecar limit cycle at eight values of the applied current i, from 150 (dark) to 198 (light), against the phase t/T with phase 0 at the maximum of v. As i increases the negative lobe deepens and moves later in the cycle. Made by the script on page 42.

A small kick δ given to the oscillator at time t advances it, once it has settled back onto the orbit, by $Z(t)^{\mathsf{T}}\delta$ units of time. The normalisation $Z^{\mathsf{T}}U' = 1$ holds for all t once it holds at $t = 0$, and XPP Studio reports how well it holds as a check.

How it is computed. For each orbit the variational equation is integrated once around the cycle, which gives the monodromy matrix $\Phi(T, 0)$. Since $Z(t)^{\mathsf{T}}\Phi(t, 0)$ does not change with t, $Z(0)$ is the left eigenvector of $\Phi(T, 0)$ for the multiplier 1, scaled so that $Z(0)^{\mathsf{T}}U'(0) = 1$. Z is then carried back around the orbit with the transposes of the same steps, which makes it exactly periodic. The orbit comes from the shooting points of the continuation, and is polished with a few Newton steps so that it closes on the fine grid (800 points per period, more if the check below asks for it). Because the eigenvector is found directly, nothing has to converge over many periods, so the method works for unstable orbits as well as stable ones, and in the middle of a family as well as at its ends.

Accuracy. The status line gives the largest error found, the largest deviation of $Z^{\mathsf{T}}U'$ from 1 over the orbit and of Z from periodicity. For the Stuart–Landau oscillator, where $Z = (-\sin\omega t, \cos\omega t)/(\omega\sqrt{\mu})$ is known exactly, the error is below 10^{-8}; for Morris–Lecar it is about 10^{-7} or less. It is larger for orbits very close to a fold of cycles, where a second multiplier is almost 1 and Z is sensitive to any error, and the status line then says so. As a check against the definition, kicks of 0.5 mV and smaller applied to the Morris–Lecar orbit shift the phase by $Z^{\mathsf{T}}\delta$ to within a relative error that goes to zero as the kick is made smaller.

Try it

1. Load **Morris–Lecar**, set $i = 150$ and press G so that the trajectory settles onto the limit cycle.
2. Open the Bifurcation view. Choose i as the parameter, a range of 90 to 220 and a step of 2, and press **Limit cycle**. The family of orbits is continued in both directions.
3. Open the Phase reduction view. Its panel has the **Adjoint** button, a menu that says *Current periodic branch*, a number of orbits and a from–to range. Enter 8 orbits, from 150 to 200, and press **Adjoint**. The **Adjoint Z** chip is on, showing Z of v, the variable that phase 0 is placed on; the variable chips on the right add others.
4. Each curve is one orbit, coloured by the value of i (the key at the top lists the values and periods; unstable orbits are marked). Orbits are chosen evenly in i along the family, and phase 0 is placed at the maximum of v so that the curves line up (Figure 35).
5. Select w as well: it is drawn with dashed lines. Z of a variable that varies little is large (here Z_w is about 200 times Z_v, because w varies by 0.37 and v by 80), so you will want **Window**. **Export CSV** writes the phase and Z for every variable of every orbit, with the periods in a last comment line, and **Export SVG** writes the figure.

The same figure is made by a script:

```
example ml
par i=150
go
bifpar i 90 220 ds=2
limitcycle
adjoint n=8 range=150,200 align=v
export svg ml-adjoint.svg
export csv ml-adjoint.csv
```

Choosing the orbits. **Adjoint** works on the family selected in the diagram (click it first), and otherwise on the latest continuation, with its two directions taken together. The **from**–**to** range keeps only orbits with the parameter in that range, and leaving it empty uses the whole family apart from the orbits next to a Hopf point (see the notes at the end). In a script, `adjoint 2` takes the second family listed for the parameter, and `at=100,150,200` takes the orbits nearest those parameter values instead of evenly spaced ones. A family that folds back is followed along its length, so the orbits on both sides of a fold are included.

Where phase 0 is. By default phase 0 is placed at the maximum of the variable on the horizontal axis of the phase plane, at the moment **Adjoint** is pressed, so that curves from different orbits, and the orbit itself, line up. The maximum is that of the smooth curve through the computed points, not of the nearest grid point, because at the flat top of a smooth peak a grid point can be several steps from the true maximum; phase 0 is therefore exact to the accuracy of the orbit, which matters when you compare with a formula. Changing the axis afterwards does not realign curves already computed; press **Adjoint** again. In a script, `align=w` uses the maximum of w instead, and `align=none` keeps the phase reference of the continuation, which is set by its phase condition and varies smoothly along the family.

The orbit itself. The **Orbit** chip draws $U(t)$ of the chosen variables against the same phase axis as everything else, one curve for each orbit. It is there so that a feature of Z can be read against the waveform: the voltage of the spike and the point at which the response changes sign, for example. With the **Orbit** chip on, **Export CSV** writes one file with the orbit together with Z and, when they exist, I, ρ and B, for every variable and every orbit, against the same phase; the period, ν_1 and κ of each orbit, the parameter values and the coupling lines are in comment lines at the end. (The other chips write only their own function.)

Beyond the phase: ν_1, I, ρ and B

The monodromy matrix has the multiplier 1, which belongs to the phase, and the other multipliers ν_j, which say how fast a displacement from the orbit dies away (or grows, if it is unstable). Let ν_1 be the *slowest nontrivial* one, meaning the one with the smallest $|\ln|\nu||$; for a stable orbit this is the multiplier of largest modulus below 1. Write $\kappa = \ln \nu_1 / T$. Near the orbit a state can be written

$$X = U(\theta) + \psi\, \rho(\theta), \qquad \theta' = 1, \quad \psi' = \kappa\psi, \tag{2}$$

where ψ is the *isostable* amplitude: the distance from the orbit along the slow direction, which decays like $e^{\kappa t}$. Three more periodic functions go with it:

$$\rho' = (A(t) - \kappa)\rho, \tag{3}$$

$$I' = \kappa I - A(t)^{\mathsf{T}} I, \qquad I^{\mathsf{T}}\rho = 1, \tag{4}$$

$$B' = -\big(A(t)^{\mathsf{T}} + \kappa\big)B - \big(D^2F(U)[\rho]\big)^{\mathsf{T}} Z. \tag{5}$$

ρ is the Floquet eigenfunction for ν_1. I is the *isostable response*: a kick δ at time t changes ψ by $I(t)^{\mathsf{T}}\delta$, just as it changes the phase by $Z(t)^{\mathsf{T}}\delta$. B is the first correction to the phase response off the orbit: the gradient of the phase at the point $U(\theta) + \psi\rho(\theta)$ is $Z(\theta) + \psi B(\theta) + O(\psi^2)$. In its equation $D^2F(U)[\rho]$ is the derivative of $A(t) = DF(U(t))$ in the direction ρ. Besides these, $I^{\mathsf{T}}U' = 0$ and $Z^{\mathsf{T}}\rho = 0$ hold all along the orbit. Wilson and Ermentrout [21] call the corresponding functions Z, I, B and C; the last, the correction to I, is not needed for the second-order coupling functions and is not computed.

How they are computed. $\rho(0)$ and $I(0)$ are the right and left eigenvectors of the same monodromy matrix for ν_1. I is carried back around the orbit with the transposes of the integration steps, as Z is, and ρ forward with the steps themselves; at every step the components along U' (in ρ) and along Z (in I), which are exactly zero, are removed. B is the periodic solution of an affine recursion over the same steps, which exists and is unique unless some multiplier equals $1/\nu_1$. $D^2F[\rho]$ is taken by central differences of the Jacobian along ρ. As with Z, XPP Studio reports the errors found: $I^{\mathsf{T}}\rho$ against 1, $Z^{\mathsf{T}}\rho$ and $I^{\mathsf{T}}U'$ against 0, and the closure of ρ, I and B after one period. For the coupled Ginzburg–Landau oscillators of Section 18, where all four functions are known in closed form, the computed ones agree to about 10^{-9} (10^{-8} for B).

Conventions. ρ is scaled so that its Euclidean norm has maximum 1 over the orbit, and signed so that it points outward from the centre of the orbit on average; I is then fixed by $I^{\mathsf{T}}\rho = 1$. The amplitude ψ is measured in these units, so rescaling ρ by a factor c divides ψ and I by c and multiplies B by c; the products $\psi\rho$, $I^{\mathsf{T}}\delta$ and ψB do not change. A function quoted in a paper with another scaling therefore differs from the one drawn here by such a factor (Section 18 gives an example). **Export CSV** says which convention was used.

When are I, ρ and B not drawn? They belong to one *real, positive* slowest multiplier. When that is not so, XPP Studio skips them, says why in the note on the right and in the key, and still computes Z, the orbit and H.

- **A complex pair.** If the slowest multipliers are a complex-conjugate pair, two isostable coordinates are needed, which XPP Studio does not compute (see [20] for the theory).
- **A negative multiplier.** Then ρ and I change sign over one period and have period $2T$; this happens next to a period doubling. They are not computed.
- **A multiplier near 1.** Within about 10^{-6} of 1 the orbit is at a bifurcation, and the functions are meaningless. The same holds if two real multipliers are equal.
- **A multiplier of zero.** If $|\nu_1| < 10^{-10}$ the amplitude dies away so fast that there is nothing to resolve. This is common for strongly attracting relaxation oscillations (we found about 10^{-17} for a FitzHugh–Nagumo oscillator with a small time-scale ratio), and the examples here, with ν_1 between 3×10^{-6} and 0.07, are all well inside the range that works.
- **An unstable orbit** ($\nu_1 > 1$). The functions are computed, but an isostable expansion about an unstable orbit is only formal; H_2 says so.

Along a family the choice is made orbit by orbit: the functions are drawn for the orbits that have them, and the key says how many do not and why.

Try it

1. Load **Morris–Lecar**, set $i = 150$ and press G.
2. Open the Phase reduction view, set the menu to *Current periodic orbit* (the number of orbits and the range disappear) and press **Adjoint**. The note on the right gives $\nu_1 = 7.6 \times 10^{-5}$ and $\kappa = \ln \nu_1 / T$, and the chips **Isostable I**, **Eigenfunction** ρ and **B** appear next to **Adjoint Z**.
3. Press the chips in turn; they are a choice of one. Choose v only among the variables on the right to get Figure 36. The key under the title of each curve gives the value of ν_1.
4. Choose v and w together: they are drawn solid and dashed. ρ is scaled by the Euclidean norm of both components, so the two curves are directly comparable, unlike those of Z.

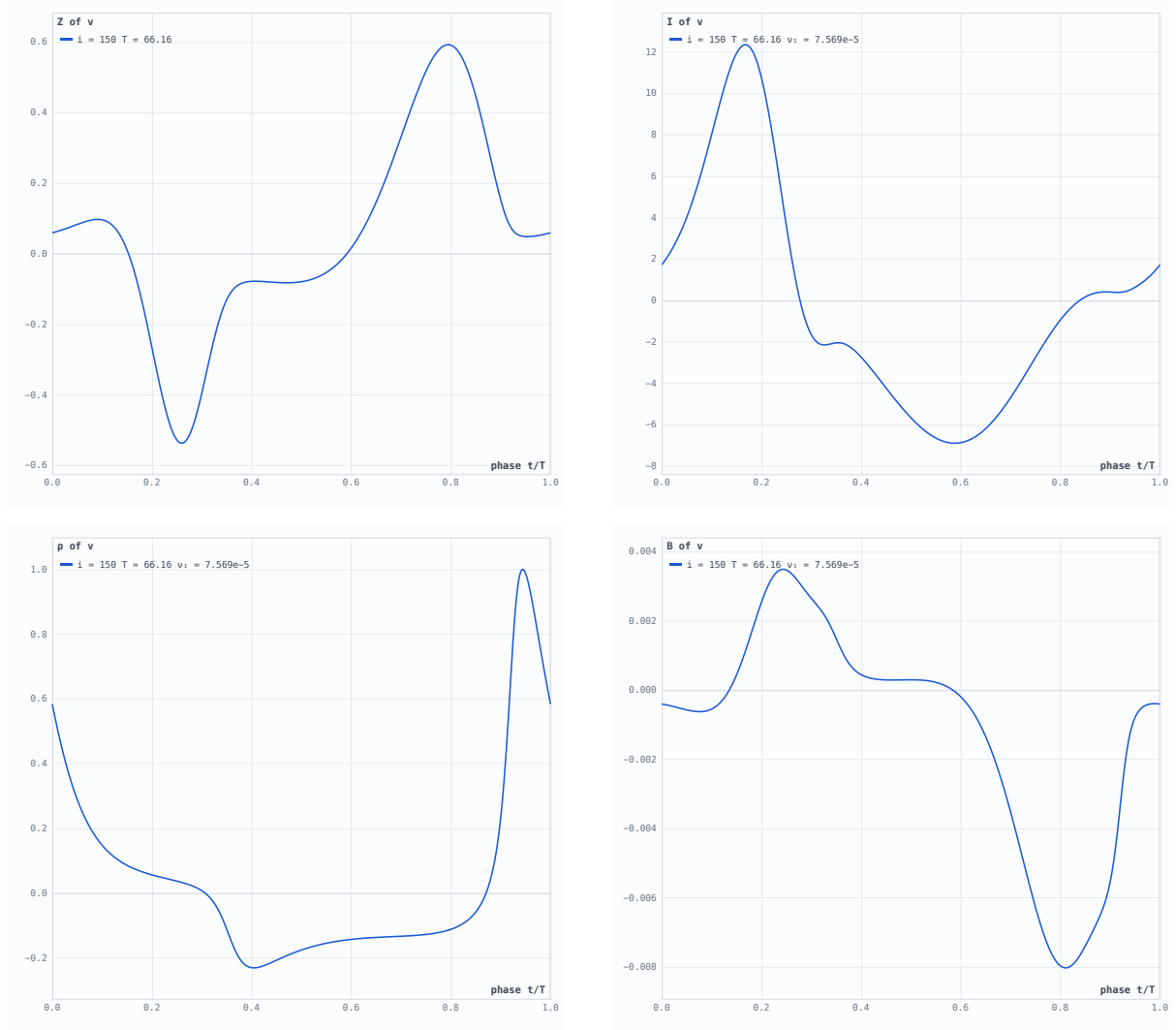


Figure 36: The v components of Z (top left), I (top right), ρ (bottom left) and B (bottom right) for the Morris–Lecar limit cycle at $i = 150$, against the phase t/T with phase 0 at the maximum of v. The period is $T = 66.16$ and the slowest multiplier is $\nu_1 = 7.6 \times 10^{-5}$. Made by the script on page 45.

The same figures are made by the script:

```
example ml
par i=150
go
adjoint here align=v
tcvars v
phasefn Z
export svg ml-iso-Z.svg
phasefn I
export svg ml-iso-I.svg
phasefn rho
export svg ml-iso-rho.svg
phasefn B
export svg ml-iso-B.svg
```

Coupled oscillators: the interaction function H

Two identical oscillators with weak coupling, $X_1' = F(X_1) + \epsilon G(X_1, X_2)$ and $X_2' = F(X_2) + \epsilon G(X_2, X_1)$, reduce to phase equations $\theta_1' = 1 + \epsilon H(\theta_2 - \theta_1)$, $\theta_2' = 1 + \epsilon H(\theta_1 - \theta_2)$ with the interaction function

$$H(\phi) = \frac{1}{T}\int_0^T Z(t)^{\mathsf{T}}\, G(U(t), U(t+\phi))\, dt. \tag{6}$$

The phase difference $\phi = \theta_2 - \theta_1$ then obeys $\phi' = \epsilon\,[H(-\phi) - H(\phi)] = -2\epsilon H_{\mathrm{odd}}(\phi)$, where $H_{\mathrm{odd}}(\phi) = \frac{1}{2}[H(\phi) - H(-\phi)]$ is the odd part. A zero of H_{odd} is a phase-locked state; it is stable (for $\epsilon > 0$) where H_{odd} crosses zero upwards. The synchronous state $\phi = 0$ and the anti-phase state $\phi = T/2$ are always zeros, by symmetry.

The form of the coupling. The coupling is written in the equations, one line for each variable that receives input:

```
#@coupling v: v'-v
#@coupling ca: dcs*(ca'-ca)
```

The expression is that component of $G(X, X')$, where X is the cell itself and a primed name (`v'`) is the same variable of the partner; a variable with no line gets no coupling. XPPAUT reads these lines as comments. Three points matter.

- ϵ **is not in the line.** The lines give G and nothing else. The overall strength ϵ in $\epsilon G(X, X')$ is not part of the model: in H it is understood to multiply the result, and in the second-order functions below it is the number in the field next to the H_2 chip. A factor inside the expression, such as `dcs` above, or a synaptic conductance, is a part of G and not of ϵ. Parameters like these may appear in the line, and H follows them as they are changed.
- **Pairs only.** The theory used is that of two identical cells with symmetric coupling. Networks, and unequal or one-way weights, are not treated.
- G **must be smooth.** The second-order functions need the derivatives of G with respect to both of its arguments, which are taken numerically. A coupling that is not differentiable (an absolute value, say) will give poor second-order results, though H itself is unaffected.

To check a result by direct simulation, write two copies of the model with the coupling terms added explicitly, $X_1' = F(X_1) + \epsilon G(X_1, X_2)$ and $X_2' = F(X_2) + \epsilon G(X_2, X_1)$.

When the model has coupling lines, an **H** chip appears next to the others. It shows H against the phase shift ϕ/T; a menu beside it chooses H, its even part $\frac{1}{2}[H(\phi) + H(-\phi)]$, its odd part, or all three (solid, dashed and dotted). For a single orbit the zeros of the odd part are marked, filled where stable and open where unstable, and listed under the curves. The curves are coloured by the parameter, as for Z. **Export CSV** writes all three parts for every orbit, together with a second file of Fourier coefficients (Section 18), and `hfunction` in a script does the same job as the chip.

Because H only needs the stored U and Z, it is recomputed whenever a parameter changes, which takes a few milliseconds for a family of orbits. You can therefore drag the slider for a coupling strength or a synaptic reversal potential and watch H change, with the axes only growing while you drag (press **Fit** to rescale). For each orbit H is computed with the continuation parameter at the value of that orbit and every other parameter at its present value. XPP Studio does not check whether a change has altered the cycle itself: if it has (the applied current, say), press **Adjoint** again. Moving the slider of the continuation parameter leaves H as it is.

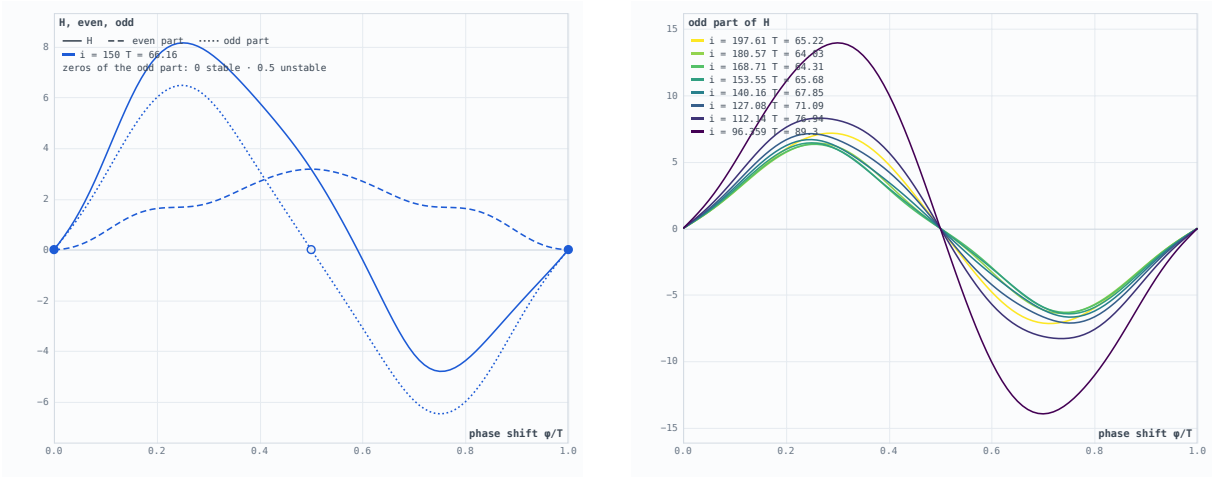


Figure 37: The interaction function of two Morris–Lecar cells coupled by `#@coupling v: v'-v`. Left: H (solid), its even part (dashed) and its odd part (dotted) at $i = 150$. The odd part has zeros at $\phi/T = 0$ (filled: stable) and $\frac{1}{2}$ (open: unstable). Right: the odd part for eight orbits from $i = 197.6$ (light) to 96.4 (dark).

Try it

1. In the Morris–Lecar example add the line `#@coupling v: v'-v` before `done` and press **Apply**. This is diffusive (electrical) coupling through the voltage.
2. Continue the family as before (set $i = 150$, press G, choose i with range 90 to 220 and step 2, press **Limit cycle**).
3. In the Phase reduction view enter 1 orbit, from 149 to 151, and press **Adjoint**; then press the **H** chip and choose *H, even, odd*. This is the left panel of Figure 37. The status line and the text under the curves give the phase-locked states.
4. Enter 8 orbits, from 95 to 200 and press **Adjoint** again, then the **H** chip. For several orbits the menu starts at the odd part (right panel).

For every orbit examined, from $i = 89$ to 198, the odd part has only the two zeros $\phi/T = 0$ and $\frac{1}{2}$: diffusive coupling through v makes synchrony stable and anti-phase unstable. Its size increases as i falls and the period lengthens, from about 6 at $i = 150$ to about 14 at $i = 96$. The same figure from a script, with the coupling line in the equations:

```
go
bifpar i 90 220 ds=2
limitcycle
adjoint n=8 range=95,200 align=v
hfunction show=odd
export svg ml-h-family.svg
```

Second order: H_2 and γ

To first order the locking equation $\phi' = \epsilon\, g_1(\phi)$, with $g_1(\phi) = H(-\phi) - H(\phi)$, has a right-hand side whose *shape* does not depend on ϵ. So it can never show a locked state losing stability, or a new one appearing, as the coupling is made stronger, although both happen in real oscillators. The next order does [21]. With the amplitude of each cell $\psi_i = \epsilon r_i$ eliminated, the phase difference obeys

$$\phi' = \epsilon\,[g_1(\phi) + \epsilon\,(g_2(\phi) + g_3(\phi))], \tag{7}$$

with $g_2(\phi) = A(-\phi) - A(\phi)$ and $g_3(\phi) = B_f(-\phi) - B_f(\phi)$, where

$$A(\phi) = \langle f_\phi(t)\, h_2(t, t+\phi)\rangle, \qquad B_f(\phi) = \langle f_{-\phi}(t+\phi)\, h_3(t, t+\phi)\rangle, \tag{8}$$

$$h_2(t,s) = Z(t)^{\mathsf{T}}\,[G_1(U(t),U(s))\,\rho(t)] + B(t)^{\mathsf{T}} G(U(t),U(s)), \tag{9}$$

$$h_3(t,s) = Z(t)^{\mathsf{T}}\,[G_2(U(t),U(s))\,\rho(s)], \tag{10}$$

G_1 and G_2 are the derivatives of G with respect to its first and second arguments, $\langle\cdot\rangle$ is the average over one period, and f_ϕ is the periodic solution of $f' = \kappa f + I(t)^{\mathsf{T}} G(U(t), U(t+\phi))$. XPP Studio finds f_ϕ exactly, term by term in a Fourier series, so there is no integral to truncate. The functions g_2 and g_3 are odd and periodic, so synchrony and anti-phase remain locked states at every ϵ, but their stability can change, and new states can appear in pairs.

The H_2 chip. The functions A and B_f are the second-order counterparts of H. With $H_2(\phi) = A(\phi) + B_f(\phi)$, each cell obeys $\theta_i' = 1 + \epsilon\,[H + \epsilon H_2](\theta_j - \theta_i)$, and the phase difference obeys

$$\phi' = -2\epsilon\,\mathrm{odd}\big[H + \epsilon H_2\big](\phi), \tag{11}$$

so that $g_1 = -2H_{\mathrm{odd}}$ and $g_2 + g_3 = -2\,\mathrm{odd}[H_2]$. The $\mathbf{H}_2$ chip, which appears next to **H** when I, ρ and B are available, therefore works like the H chip, with a field for ϵ and the same menu for the part that is drawn: the whole function, its even part, its odd part, or all three. Both cells use the same G as the H chip.

- With $\epsilon = 0$ it shows H_2 itself (or its even or odd part): the shape of the second-order correction.
- With $\epsilon \neq 0$ it shows $H + \epsilon H_2$. When one part is drawn, the same part of the first-order H is drawn dashed in grey for comparison. The zeros of the odd part are the locked states, marked filled where stable and open where unstable, and listed under the curves. As for H, a zero is stable where the odd part crosses zero upwards if $\epsilon > 0$ (downwards if $\epsilon < 0$; negative ϵ is allowed). The even part does not enter the equation for the phase difference, but it gives the common rate of advance of a locked pair.
- The menu has one more entry, ϕ'/ϵ, which draws $g_1 + \epsilon(g_2 + g_3) = -2\,\mathrm{odd}[H + \epsilon H_2]$, the rate of change of the phase difference as plotted in Figure 2 of [21], with g_1 dashed (at $\epsilon = 0$ it draws $g_2 + g_3$). A stable state is then a zero where the curve crosses downwards.
- Changing ϵ recombines functions that are already computed, so it is instant.

The CSV file from **Export...** has g_1, g_2, g_3, $g_2 + g_3$, the composite for the present ϵ, the two parts $H_{24}(\phi,\phi)$ and $H_{34}(\phi,0)$ of H_2, and H_2 and $H + \epsilon H_2$; these are the functions themselves, without ϵ, except for the two composite columns, which use the value in the field. A second file holds their Fourier coefficients. In a script, `hsecond eps=0.1 show=odd` and `phasefn H2` do the same; `show` takes `all`, `H`, `even`, `odd` or `rate`.

The coefficient γ. The legend gives

$$\gamma = \tfrac{1}{4}\,[g_2'(0) + g_3'(0)] \tag{12}$$

for each orbit, and for a single orbit also the two numbers $-A'(0)$ and $-B_f'(0)$ from which it is made. For diffusive coupling $G(X, X') = D(X' - X)$ these are equal, and γ is the coefficient of the fourth-derivative term in the Kuramoto–Sivashinsky equation $\phi_t = \epsilon(\beta\phi_x^2 + \alpha\phi_{xx} + \gamma\phi_{xxxx})$ that describes a continuum of such oscillators [21, 17]. It is the term that matters when the diffusion coefficient $\alpha = H'(0)$ is small. The program reports both numbers, so that for other couplings you can see how far they differ: the equality is a property of diffusive coupling, and is a useful check on a model's coupling lines.

Fourier coefficients, for networks. To put these functions into a model of many oscillators it is easier to have them as formulas than as tables. CSV from **Export...** in the H or H_2 mode therefore writes a second file, with the same name and `-fourier` added, that holds the first 20 Fourier coefficients of the functions, for each orbit:

$$F(\phi) = a_0 + \sum_{k=1}^{20} \Big[a_k \cos \frac{2\pi k \phi}{T} + b_k \sin \frac{2\pi k \phi}{T} \Big], \tag{13}$$

with ϕ the time shift and T the period (the periods are in the comment lines; in the file the rows are $k = 0, \ldots, 20$ and the columns are `F_a` and `F_b` for each function F). In the H mode the file has H; in the H_2 mode it has H, $H_{24}(\phi,\phi)$, $H_{34}(\phi,0)$, H_2 and g_2+g_3. Since $g_2+g_3 = -2\,\mathrm{odd}[H_2]$ is odd, its cosine coefficients are zero and $b_k(g_2+g_3) = -2\,b_k(H_2)$. Like the functions, the coefficients do not contain ϵ.

Twenty harmonics are plenty for a smooth cycle. For the Hodgkin–Huxley example the amplitude $\sqrt{a_k^2+b_k^2}$ of H falls from about 2 at $k=1$ to 4×10^{-5} at $k=10$ and 3×10^{-6} at $k=20$; for the Ginzburg–Landau oscillators only $k \le 2$ are present, and the computed coefficients agree with the exact ones to 10^{-9}. The coefficients are found from 100 samples of each function over a period, so they should not be used much beyond $k=20$. A short script for a network of N cells then needs only the coefficients: at first order, $\theta_i' = 1 + \epsilon \sum_j a_{ij} H(\theta_j - \theta_i)$ with H the series above.

The second order is not complete for a network. The functions $H_{24}(\phi,\phi)$ and $H_{34}(\phi,0)$ are the terms of the network equation of [21] that involve one neighbour: for weights a_{ij} they enter as $\epsilon^2 \sum_j [a_{ij}^2 H_{24}(\phi_{ij},\phi_{ij}) + a_{ij}a_{ji}H_{34}(\phi_{ij},0)]$, with $\phi_{ij} = \theta_j - \theta_i$, which is why they are written out separately. For symmetric weights equal to 1 the sum is H_2. The terms that involve two different neighbours of the same cell need the full two-argument functions $H_{24}(\eta,\xi)$ and $H_{34}(\eta,\xi)$, which XPP Studio does not compute. They can be computed from the file written with the **Orbit** chip, together with the coupling function G and its derivatives, which you supply: $H_{24}(\eta,\xi) = \langle f_\xi(t)\, h_2(t, t+\eta)\rangle$ and $H_{34}(\eta,\xi) = \langle f_{\xi-\eta}(t+\eta)\, h_3(t,t+\eta)\rangle$, with h_2, h_3 and f_ϕ as defined above. The two functions in the exported table are the cases $\xi=\eta$ and $\xi=0$, and serve to check such a calculation. So $H + \epsilon H_2$ is exact for a pair, and a useful but incomplete second-order correction for a network.

How far to trust it. The expansion is accurate to $O(\epsilon^2)$ in the coupling strength, and it assumes that the other multipliers are small compared with ν_1, so that one isostable coordinate is enough. For a model with two variables this is automatic. Otherwise the legend gives the ratio of the next multiplier to ν_1 whenever it exceeds $\frac{1}{4}$, and also warns when $|\kappa|T < 0.3$, when the slowest multiplier is close to 1 and the expansion breaks down as the orbit nears a bifurcation, and when the orbit is unstable. The computation costs about five times as much as H (it evaluates G and its derivatives at every point of the orbit for each of 100 phase shifts), so XPP Studio computes it only while the H_2 chip is on, and again when a parameter is changed while it is on.

An example with exact answers: coupled Ginzburg–Landau oscillators

The complex Ginzburg–Landau oscillator, in real coordinates and with a pair of cells $j = 1, 2$, $k = 3-j$, is [21]

$$x_j' = x_j(1 - x_j^2 - y_j^2) - q(x_j^2 + y_j^2)\, y_j + \epsilon[x_k - x_j - d\,(y_k - y_j)], \tag{14}$$

$$y_j' = y_j(1 - x_j^2 - y_j^2) + q(x_j^2 + y_j^2)\, x_j + \epsilon[y_k - y_j + d\,(x_k - x_j)]. \tag{15}$$

It is the model **Coupled Ginzburg–Landau oscillators** in the Model menu. Its equations are

```
x'=x*(1-x^2-y^2)-q*(x^2+y^2)*y
y'=y*(1-x^2-y^2)+q*(x^2+y^2)*x
```

```
par q=1,d=0.9
init x=1,y=0
#@coupling x: (x'-x)-d*(y'-y)
#@coupling y: (y'-y)+d*(x'-x)
@ total=40,dt=0.01,meth=rk4,xp=x,yp=y
done
```

The first two lines are one cell. The two coupling lines are $G(X, X')$, the part of the equations multiplied by ϵ, and the strength ϵ itself is not in the file (as explained above). The other parameters, q and d, are part of G or of the cell, and can be moved with their sliders.

The exact answers. Uncoupled, a cell has the limit cycle $U = (\cos qt, \sin qt)$ of period $T = 2\pi/q$ and a single nontrivial multiplier, so that $\kappa = -2$ and $\nu_1 = e^{-4\pi/q}$. Write $C = \cos qt$ and $S = \sin qt$. In the scaling of [21],

$$Z = (C - S/q,\ S + C/q), \quad \rho = (qS + C,\ S - qC), \quad I = (C,\ S), \quad B = \frac{1+q^2}{q}(S,\ -C). \tag{16}$$

XPP Studio scales ρ so that its norm has maximum 1, which divides ρ by $\sqrt{1+q^2}$; I is then multiplied by $\sqrt{1+q^2}$ and B divided by it (the rule in the conventions above), while Z is unchanged. With phase 0 at the maximum of x, which is where the trajectory starts, these are exactly the curves drawn, with C and S evaluated at t/T in place of $qt/2\pi$. The locking equation is

$$\phi' = -\epsilon \frac{2}{q} \sin q\phi \left[\, 1 - dq + \epsilon d^2(1+q^2) \cos q\phi \,\right]. \tag{17}$$

So $g_1(\phi) = -\frac{2}{q}(1 - dq) \sin q\phi$ and $g_2(\phi) + g_3(\phi) = -\frac{d^2(1+q^2)}{q} \sin 2q\phi$, and $\gamma = -\frac{1}{2}d^2(1+q^2)$. To first order synchrony is stable if and only if $1 - dq > 0$, and anti-phase if and only if $1 - dq < 0$ (for $\epsilon > 0$). The exact critical couplings, at which synchrony and anti-phase change stability [18], are

$$\epsilon_s = \frac{dq - 1}{d^2 + 1}, \qquad \epsilon_a = \frac{1 - dq}{d^2 - 2dq + 3}, \tag{18}$$

and the second-order theory replaces them by

$$\epsilon_{s,\mathrm{approx}} = \frac{dq - 1}{d^2(1+q^2)}, \qquad \epsilon_{a,\mathrm{approx}} = \frac{1 - dq}{d^2(1+q^2)}. \tag{19}$$

The two agree near $d = 1/q$, where both are zero, and the approximate curves are tangent to the exact ones there.

Try it

1. Choose **Coupled Ginzburg–Landau oscillators** in the Model menu and press G. The trajectory starts on the cycle, so it is a circle.
2. Open the Phase reduction view, set the menu to *Current periodic orbit* and press **Adjoint**. The note on the right gives $\nu_1 = 3.487 \times 10^{-6}$, which is $e^{-4\pi}$, and $\kappa = -2$.
3. Press the **Adjoint Z**, **Isostable I**, **Eigenfunction** ρ and **B** chips and compare with the formulas above for $q = 1$. At phase 0 the values are $Z = (1, 1)$, $I = (\sqrt{2}, 0)$, $\rho = (1, -1)/\sqrt{2}$ and $B = (0, -\sqrt{2})$, to about 10^{-8}. Moving the pointer over the plot reads off the values.
4. Press the **H** chip. Its odd part is $0.1 \sin q\phi$ (so $g_1 = -0.2 \sin q\phi$): the zero at $\phi = 0$ is stable and the one at $T/2$ is unstable, as the first-order test $1 - dq = 0.1 > 0$ says.
5. Press the **H**$_2$ chip, with $\epsilon = 0$, and choose *odd part* in the menu: the curve is

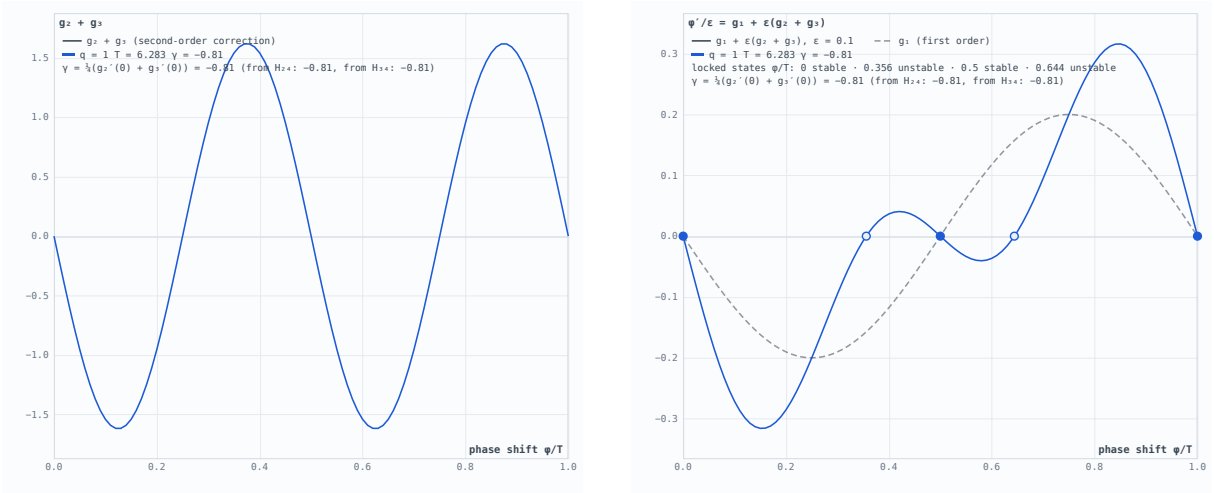


Figure 38: The second-order functions of two coupled Ginzburg–Landau oscillators with $q = 1$, $d = 0.9$. Both panels use the ϕ'/ϵ entry of the menu. Left: the correction $g_2 + g_3$ ($\epsilon = 0$), which is $-1.62 \sin 2\phi$ exactly. Right: the composite $g_1 + \epsilon(g_2 + g_3)$ at $\epsilon = 0.1$ (solid) and the first-order g_1 (dashed). At first order only synchrony is stable. At second order anti-phase is stable as well, with unstable states at $\phi/T = 0.356$ and 0.644 between them (filled circles: stable, open: unstable). Made by the script on page 51.

$0.81 \sin 2q\phi = -\frac{1}{2}(g_2 + g_3)$. Choose ϕ'/ϵ instead and it is $g_2 + g_3 = -1.62 \sin 2q\phi$ (left panel of Figure 38). The legend gives $\gamma = -0.81$, equal to $-\frac{1}{2}d^2(1+q^2)$, from both H_{24} and H_{34}. The even part of H_2 is not zero (it contains a constant shift of the frequency), which shows the difference from the odd parts above.

6. Enter $\epsilon = 0.05$ with the odd part chosen: it crosses zero upwards at 0 and downwards at $\frac{1}{2}$, so synchrony is stable and anti-phase unstable, as at first order, and there are no other zeros. Enter 0.07: anti-phase has become stable, and two unstable states have appeared at $\phi/T \approx 0.42$ and 0.58. At $\epsilon = 0.1$ they are at 0.356 and 0.644 (right panel of Figure 38, drawn with the ϕ'/ϵ entry, in which stable states are where the curve crosses downwards). The threshold at which anti-phase becomes stable is $\epsilon_{a,\text{approx}} = 0.1/1.62 = 0.062$; the exact value is $\epsilon_a = 0.050$, and the difference is the part of the answer that this order of the theory cannot give.

7. Move the slider for d above $1/q = 1$, say to 1.1. Now $1 - dq < 0$: at weak coupling synchrony is unstable and anti-phase stable. Raise ϵ in the H_2 field: for $\epsilon = 0.03$ there are no other zeros, and at $\epsilon = 0.05$ synchrony has become stable, with two unstable states close to it at $\phi/T = 0.095$ and 0.905. This is the exchange of stability at $\epsilon_{s,\text{approx}} = 0.041$ (exact $\epsilon_s = 0.045$). Figure 1 of [21] shows these curves in the (d, ϵ) plane.

The figures are made by the script:

```
example cgl
go
adjoint here
hsecond eps=0 show=rate
export svg cgl-g23.svg
hsecond eps=0.1 show=rate
export svg cgl-comp.svg
```

A single orbit, in one press of Update

To study one set of parameter values you do not need a family. In the Phase reduction view set the menu to *Current periodic orbit* (the number of orbits and the range then disappear), integrate a trajectory until it has settled onto its limit cycle, and press **Adjoint**; in a script, `adjoint here`. XPP Studio finds the periodic orbit at the end of the active trajectory, as the first step of **Limit cycle** does, but at the present parameters and without continuing it, and computes everything above for it.

The choice then stays on, and XPP Studio follows the trajectory *while the Phase reduction view is open*. After a **Update** (or an automatic run) that has changed the cycle, it finds the orbit again and recomputes Z, I, ρ, B, H and H_2, so that setting parameters and pressing **Update** gives everything at once, and the chip that you have chosen stays chosen. In the other views nothing is recomputed: the results are only marked as out of date, and are brought up to date once when you return to the Phase reduction view, so you can go on working in the phase plane without paying for them. To stop following altogether, untick **Follow the trajectory** in the panel (in a script, `adjoint follow=off`, or `adjoint off`, which also hides the curves); the results then stay as they are until you press **Adjoint** again. It decides whether the cycle has changed by sampling the right-hand side along the stored orbit: a change of a coupling parameter alone leaves the orbit as it is and only updates H and H_2. When a slider is dragged the update waits until you pause, and with Auto-update unticked it is done by **Update**. Pressing **Adjoint** along a family replaces it. The trajectory must have settled onto a *stable* limit cycle by the end of its total time, as for **Limit cycle**, and if it has not the status line says so and the previous result is kept. An unstable orbit needs continuation from a Hopf point.

The Hodgkin–Huxley model of the squid axon, with coupling through the voltage, is in the Model menu; its equations are

```
dv/dt=(i-gna*m^3*h*(v-ena)-gk*n^4*(v-ek)-gl*(v-el))/cm
dm/dt=am(v)*(1-m)-bm(v)*m
dh/dt=ah(v)*(1-h)-bh(v)*h
dn/dt=an(v)*(1-n)-bn(v)*n
am(v)=if(abs(v+40)<1e-7)then(1)else(0.1*(v+40)/(1-exp(-(v+40)/10)))
bm(v)=4*exp(-(v+65)/18)
ah(v)=0.07*exp(-(v+65)/20)
bh(v)=1/(1+exp(-(v+35)/10))
an(v)=if(abs(v+55)<1e-7)then(0.1)else(0.01*(v+55)/(1-exp(-(v+55)/10)))
bn(v)=0.125*exp(-(v+65)/80)
par i=10,gna=120,gk=36,gl=0.3,ena=50,ek=-77,el=-54.4,cm=1,g=1
init v=-65,m=0.05,h=0.6,n=0.32
#@coupling v: g*(v'-v)
@ total=300,dt=0.02,meth=rk4,xp=v,yp=n
done
```

Try it

1. Choose **Hodgkin–Huxley (squid axon)** in the Model menu, which is the model above, and press G. The trajectory settles onto the spiking cycle.
2. In the Phase reduction view choose *Current periodic orbit* in the menu and press **Adjoint**. It takes about half a second. Press the **H** chip and choose *H, even, odd* (Figure 39).
3. The period is 14.64 ms. The odd part has four zeros, so diffusive coupling through v leaves both synchrony ($\phi = 0$) and anti-phase ($\phi = T/2$) stable, with unstable states at $0.38T$ and $0.62T$ between them.
4. The note on the right gives $\nu_1 = 0.074$ ($\kappa = -0.178$ per ms). The other multipliers are smaller than ν_1 by a factor of order 10^{-11}, so one isostable coordinate is exactly what

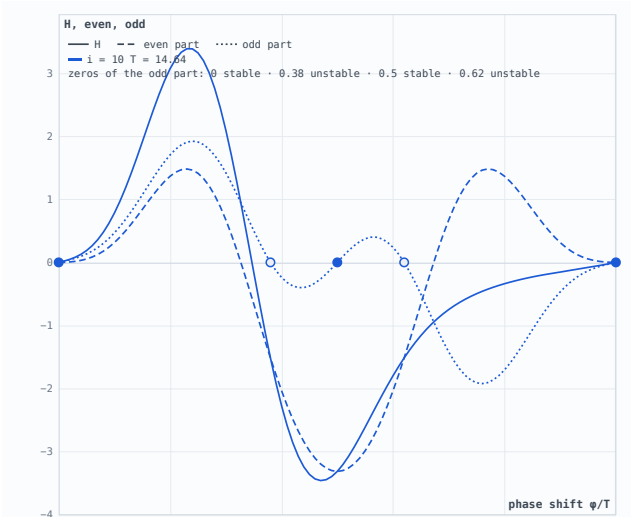


Figure 39: The interaction function of two Hodgkin–Huxley neurons coupled through the voltage, H (solid), its even part (dashed) and odd part (dotted), at $i = 10$ (period 14.64 ms). The odd part has stable zeros (filled) at $\phi/T = 0$ and $\frac{1}{2}$ and unstable ones (open) at 0.38 and 0.62.

is needed. Press **H**$_2$: with $\epsilon = 0$ the correction $g_2 + g_3$ is small, and $\gamma = 0.024$. With $\epsilon = 0.05$ the unstable states have moved only from $0.380T$ and $0.620T$ to $0.384T$ and $0.616T$, and nothing has changed qualitatively. For this model, at these couplings, first order is a good guide.

5. Open the Parameters tab and move the slider for g: H follows at once and changes only in size, and the orbit is not computed again. Move the slider for i to 15 and pause: after a moment the period becomes 12.72 ms and the curves are recomputed for the new cycle.

6. Untick **Auto-update**, change i and press **Update** once: the trajectory, the orbit and all the functions are brought up to date by that one press.

Checks

First order. For the Stuart–Landau oscillator with `#@coupling x: x'-x` and `#@coupling y: y'-y`, $H(\phi) = \sin(2\pi\phi/T)/\omega$ exactly; the computed H agrees to about 10^{-9}, has no even part, and shows synchrony stable. For two Morris–Lecar cells simulated directly with $\epsilon = 0.002$, started at five different phase differences, the measured rate of change of the phase difference agrees with $\epsilon\,[H(-\phi) - H(\phi)]$ to within about 5%, the difference being the next order in ϵ. The same test on the Hodgkin–Huxley result above gives the right direction of drift at all five starting phases (towards 0 or $\frac{1}{2}$, away from 0.38 and 0.62), with the size within 1% at three of them and within 13% at the other two, where the drift is smallest.

Isostable functions. For the Ginzburg–Landau oscillator the computed Z, ρ, I and B agree with the closed forms to about 10^{-9} (for B, 10^{-8}) at $q = 1$ and at $q = 1.7$, and $\kappa = -2$ to 3×10^{-9}. For a model without a closed form, van der Pol with $\mu = 0.7$, the phase response was measured directly: points displaced from the orbit by $\pm 0.02\rho$ were integrated until they had settled onto the cycle, and the gradient of the resulting phase compared with $Z + \psi B$. The derivative of the measured gradient with respect to ψ agrees with B to 10^{-5} at three places on the orbit, the size of the $O(\psi^2)$ error of the measurement itself.

Second order. For the Ginzburg–Landau model with $q = 1.7$ and $d = 0.9$, g_1 agrees with the exact $-\frac{2}{q}(1 - dq)\sin q\phi$ to 1.5×10^{-9} and $g_2 + g_3$ with $-\frac{d^2(1+q^2)}{q}\sin 2q\phi$ to 3×10^{-8}, and $\gamma = -1.57545$ equals $-\frac{1}{2}d^2(1 + q^2)$; the values from H_{24} and H_{34} agree to 10^{-13}. For two cells simulated directly, started with a small phase difference near synchrony, the difference decays as $e^{\lambda t}$, and Table 4 compares the measured λ with the first-order prediction $\epsilon g_1'(0)$ and the second-order one $\epsilon[g_1'(0) + \epsilon(g_2'(0) + g_3'(0))]$. In every case the second-order value is closer to the measured one, by a factor of between two and ten; what is left is of the size of the next term in ϵ.

model	ϵ	measured	first order	second order
van der Pol, $\mu = 0.7$	0.025	−0.02649	−0.02641	−0.02646
	0.05	−0.05303	−0.05281	−0.05301
	0.1	−0.10614	−0.10563	−0.10640
Hodgkin–Huxley, $i = 10$	0.1	−0.02464	−0.02669	−0.02572
	0.2	−0.04599	−0.05338	−0.04949

Table 4: The rate of decay λ of a small phase difference between two coupled cells near synchrony: measured in a direct simulation, and predicted at first and at second order. The van der Pol cells are coupled through `#@coupling v: v'-v`, the Hodgkin–Huxley ones through `g*(v'-v)` with $g = 1$; the unit is per unit time (per ms for Hodgkin–Huxley). The antiphase state could not be tested this way for Hodgkin–Huxley, because at these values of ϵ the coupling current of two cells in anti-phase, up to $\epsilon \times 100\,\mathrm{mV}$, is not weak.

Things to know.

- **Units.** Z is per unit of t, normalised by $Z^{\mathsf{T}}U' = 1$. To express it per unit of phase on $[0, 1)$ divide by T; on $[0, 2\pi)$ multiply by $2\pi/T$. Only the horizontal axis is scaled to t/T, so that orbits of different periods share one axis. The functions g_1, g_2, g_3 are in units of time per unit time, with the horizontal axis ϕ/T.
- **The coupling is $\epsilon\, G(X, X')$,** for a pair of identical cells, with G in the `#@coupling` lines and ϵ outside the equations. ϵ is entered only in the H_2 field. A strength written inside a line is part of G.
- **Floquet multipliers.** I, ρ, B and H_2 need a real, positive slowest multiplier ν_1 that is not too close to 1 or to zero; see the box above for the cases that are skipped. The second-order theory also assumes that the other multipliers are much smaller than ν_1.
- **Near a Hopf point** the orbit shrinks and Z grows like 1/amplitude (for the Stuart–Landau oscillator, as $1/\sqrt{\mu}$), and very close to the point it is meaningless; ν_1 is then near 1 and the isostable functions are skipped as well. Unless you give a range or values with `at=`, orbits with an amplitude below 3% of the largest in the family are left out, and the status line says so. An orbit whose Z is more than 8 times the typical size is left off the plot while the window is the automatic fit, and the key names it. Set the window with **Window**, or zoom, and every curve is drawn; **Fit** returns to the automatic fit. A range such as 100 to 200 for Morris–Lecar keeps the family away from both Hopf points.
- **Limits.** The phase reduction is for differential equations without delays or Volterra integrals, with at most 60 variables (B, and so H_2, at most 24), and for models that do not depend on t explicitly (the same restriction as for continuing orbits). The *Current periodic orbit* choice finds stable orbits only. Z, the isostable functions and the orbits are not updated when the parameters or the family change, so press **Adjoint** again after a change that alters the cycle (H and H_2, which only use them, are recomputed).

- **Recording.** With **Record** on, the chips of this view are recorded as `phasefn` lines, and the field for ϵ as `hsecond eps=`.

19 Tutorial 16: animating a model

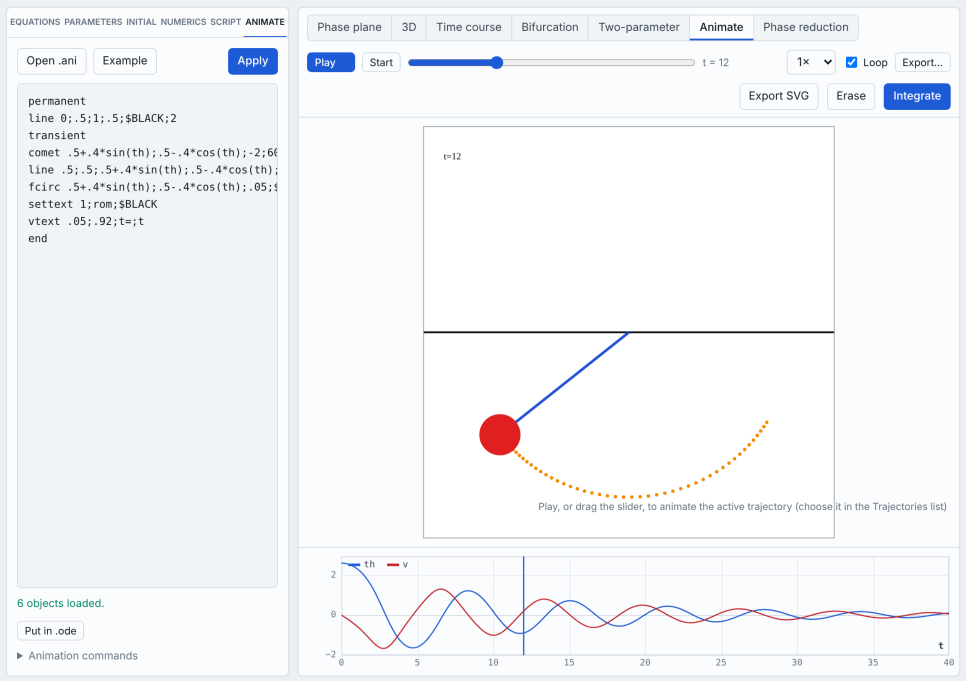


Figure 40: The Animate view, with the damped pendulum below. Left: the animation script, taken from the `#@ani` lines of the model. Centre: the pendulum at $t = 12$, with the last 60 points of its path as a trail; above it the controls for playing, and below it the time course, with a vertical line at the time shown.

An animation is a picture of the model that changes as the solution unfolds: a pendulum swinging, a wheel turning, a row of cells lighting up. XPP Studio reads the animation language of XPPAUT [1], so existing `.ani` files can be used. A script is a list of simple shapes, one per line, whose positions and colours are formulas in the variables, the parameters and t. The picture is drawn again for each moment of the active trajectory.

The script can be kept in the `.ode` file, one script line for each line that starts with `#@ani`. XPPAUT reads these as comments, so the file still works there. For example:

```
# a damped pendulum with its animation stored in the file
th'=v
v'=-mu*v-sin(th)
par mu=0.15
init th=2.6,v=0
#@range mu 0 1
@ total=40,dt=0.05
#@ani permanent
#@ani line 0;.5;1;.5;$BLACK;2
#@ani transient
#@ani comet .5+.4*sin(th);.5-.4*cos(th);-2;60;$ORANGE
#@ani line .5;.5;.5+.4*sin(th);.5-.4*cos(th);$BLUE;3
#@ani fcirc .5+.4*sin(th);.5-.4*cos(th);.05;$RED
#@ani settext 1;rom;$BLACK
#@ani vtext .05;.92;t=;t
```

```
#@ani end
done
```

The picture has its lower left corner at $(0, 0)$ and its upper right corner at $(1, 1)$, so the pivot of the pendulum is at $(0.5, 0.5)$. `permanent` objects are drawn once and `transient` ones again for every frame. `line` takes the two end points, a colour and a thickness; `fcirc` is a filled circle given by its centre and radius; `comet` keeps the last n points of a path (here 60, drawn as dots of radius 2); `vtext` writes a label followed by the value of a formula. A colour is either a name, such as `$RED`, or a formula between 0 (red) and 1 (violet).

Try it

1. Choose **Damped pendulum with animation** in the **Model** menu at the top of the window. (Or paste the model above into the **Equations** tab and press **Apply**.) The messages say that the file contains an animation.
2. Press **Animate** above the plot (or the key 7) and then **Play**. Drag the slider to go to any time: the vertical line on the time course shows where you are. The menu sets the speed, and **Loop** repeats the run.
3. Drag the slider for `mu` in the **Parameters** tab: the swing, and the trail, change as you move it.
4. In the **Animate** tab, change `$RED` in the `fcirc` line to the formula `(th+3)/6` and press **Apply**: the colour of the bob now follows the angle.
5. Press **GIF / video...** in the transport bar (or **Export...** and then *GIF*), choose GIF or WebM, and press **Make file**.

Using an existing `.ani` file. Load the model it was written for, make sure a trajectory exists (press **Integrate** for large systems, which are not run automatically), then press **Open .ani** in the **Animate** tab. The view switches to the animation at once. **Put in .ode** copies the script into the equations as `#@ani` lines, so that the next time the model is loaded the animation comes with it; loading a different model clears the script. Arrays work as in the model language, so that `fcirc .05+.04*[0..20];.5*(vv[j]+1);.02;1-w[j]` draws 21 circles. The **Animation commands** entry in the tab lists the rest of the language.

Saving the animation. **GIF / video...** in the transport bar makes a file from any number of frames chosen evenly from the whole run. A **GIF** plays everywhere, including in slides, but is limited to 256 colours and can be large. A **WebM** video is much smaller and has full colour, but it is recorded in real time, so the tab must stay in view while it is made, and PowerPoint and Keynote may not play it (`ffmpeg -i file.webm file.mp4` converts it). **Export SVG** saves the frame on the screen as a figure.

Differences from XPPAUT's animation. The picture keeps the proportions of its `dimension` box, so circles stay round, and it is drawn on a white background. Formulas may use auxiliary variables, which XPPAUT does not allow. The colour scale is a sweep of hue from red to violet. The `grab` command (dragging objects with the mouse) and the nullcline commands `xnull` and `ynull` are not supported yet: their lines are ignored, with a note.

20 Numerical settings

The **Numerics** tab (Figure 41) collects the integration and continuation parameters. Changes to the integration settings rerun every trajectory on screen (when **Auto-update** is ticked; see

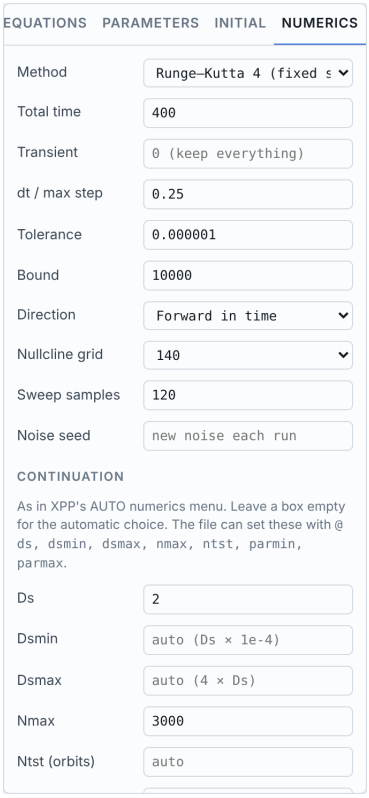


Figure 41: The Numerics tab: integration settings at the top, continuation settings below, and the continuation tolerances (Section 20) further down.

below).

Integration.

Method As in Section 3.

Total time The duration of each run; for maps, the number of iterations.

Transient As in XPPAUT, the run still lasts the total time, but only $t \geq$ the transient is kept (`@ trans=`). Use it to look at attractors without the approach to them. Manifolds and the single periods loaded from continuation are never cut.

dt The step for fixed-step methods, and the largest step for adaptive ones.

Tolerance Applies to the adaptive methods.

Bound Integration stops when any variable exceeds this value.

Direction Forward, backward or both ways in time (also next to **Integrate** in the Initial tab).

Start time The start time t_0 of each run (`@ t0=`).

Nullcline grid Sets the resolution of the nullclines.

Sweep samples Sets the number of parameter values in **Sweep max/min**.

Auto-update, Update and long computations. By default, changing a parameter, an initial value or an integration setting runs the trajectories again at once, so that the figure always matches the values shown. That is what makes sliders so useful, but it is a nuisance in two situations: when you want to change three parameters and not wait for a run after the first, and

when the system is large enough for a run to take seconds. Unticking the **Auto-update** box at the top of the window turns it off.

- **With Auto-update unticked,** a change only marks the display as out of date. A label on the plot says *Out of date: press Update* (or *Not run yet* if nothing has been computed) and **Update** is highlighted. Nullclines and the direction field still follow the parameters, because they are cheap; trajectories and equilibria wait for **Update**.
- **Update** brings everything up to date: the active curve follows the new initial values, the other curves are recomputed with the new parameters and settings, and the equilibria are found again. If nothing is out of date it does what **Integrate** does. Ticking Auto-update again while the display is out of date runs at once.
- **Integrate** (or G) still adds a new trajectory from the initial values; if the other curves are out of date, they are recomputed first, so a new curve never sits next to curves made with other parameters.
- **The default depends on the size of the system.** Auto-update is ticked for up to 40 variables, the same limit above which equilibria are no longer searched for automatically, and off above it. Loading a chain of 200 cells (400 variables) therefore computes nothing until you press **Update**, and the status line says so. You can tick it for such a model if you wish. Your choice to untick it is remembered between sessions, and **Reset all** ticks it again.

A run that takes more than a fraction of a second shows its progress in the status line, for example *Running… 34%, t = 340*, with a **Stop** button at its right-hand end, and the percentage is also shown in the title of the browser tab, so you can watch it from another tab. **Stop** ends the run where it is and keeps the part already computed. If you change something while a run is going, the old run is abandoned and a new one starts with the new values, so dragging a slider on a large system always ends with curves that match its final position. Short runs finish before any of this appears, so small systems behave exactly as before.

> Only ordinary differential equations are run in this way. Maps, delay equations, Volterra equations, models with Markov or Wiener variables, and models with global flags are always computed in one piece, as before; they have no progress line or **Stop**, although Auto-update and **Update** work for them. The time is dominated by the right-hand side: a chain of 200 FitzHugh–Nagumo cells with `meth=euler` and many thousands of steps can take several seconds, mostly in evaluating the equations. Raising `dt`, lowering the total time or plotting fewer steps helps more than any setting.

Continuation. These settings correspond to the entries of XPPAUT's AUTO numerics window and can be set in the file with `@ ds, dsmin, dsmax, nmax, ntst, parmin, parmax`. Leave a box empty to use the automatic choice. **Defaults**, next to the heading, puts them back to the model's defaults: the values its file sets, otherwise the built-in ones. A step set with `@ ds` applies to every parameter; otherwise Ds is 1/150 of the parameter's range. A Ds you set yourself is kept until you change the bifurcation parameter or press **Defaults**. The **Wilson–Cowan** example sets `@ ds=0.01`, since its branches have sharp folds that a larger step cuts across.

Setting	Effect
Ds	initial arclength step for equilibrium branches (same as **Step ds**)
Dsmin, Dsmax	smallest and largest steps; a branch stops if the step falls below Dsmin
Nmax	maximum number of points in each direction of an equilibrium branch
Ntst (orbits)	number of shooting intervals per periodic orbit (automatic: 20 for up to 3 variables, 14 up to 5, otherwise 10)
Ds, Dsmax, Nmax (orbits)	the same for periodic-orbit branches and curves of cycles
Nmax (2-par), Nmax (2-par cycles)	point limits for two-parameter curves

Continuation tolerances. Each step of a continuation predicts the next point along the tangent and corrects it with Newton's method. The correction stops when its size falls below a tolerance, and the point is then accepted only if the equations hold to within the residual tolerance; otherwise the step is halved. The section **Continuation tolerances** of the Numerics tab sets these, and its **Defaults** puts them back to the model's defaults. The file can set them with `@ eqtol, potol, tptol, restol, itmax, tgrow`, and **Write state** writes those that differ from the defaults.

Setting	Default	Effect
Newton tol (eq.)	10^{-10}	last Newton correction for branches of equilibria and fixed points, relative to the size of the solution (`eqtol`, like AUTO's EPSU)
Newton tol (orbits)	10^{-9}	the same for periodic orbits, in the scaled unknowns of the shooting method (`potol`)
Newton tol (2-par)	10^{-8}	the same for two-parameter curves of folds, Hopf points and bifurcations of cycles (`tptol`)
Residual tol	10^{-6}	largest residual of the equations at an accepted point; for orbits, relative to their size (`restol`)
Newton iterations	automatic	most Newton iterations per step before the step is halved: 15 for equilibria, 14 for orbits and two-parameter curves, 12 for curves of cycles (`itmax`, like AUTO's ITNW)
Period growth stop	40	a branch of periodic orbits, or a curve of cycles, stops when the period reaches this multiple of its starting value (`tgrow`)

Tighter tolerances give more accurate points and usually smaller steps; looser ones can carry a branch through a stretch where the corrector struggles, at some cost in accuracy, so check such a branch against a tighter run. Raise the period growth stop to follow orbits closer to a homoclinic orbit.

If a branch stops early, the status line says why: for example, it reached the point limit, the corrector stopped converging, the orbit shrank onto a Hopf point, or the period grew without bound (a homoclinic orbit is near). If it stopped because of a step-size or point limit, adjust the corresponding setting and run it again. A branch of orbits whose period grew to its end is examined (Section 17): the saddle it approaches is loaded into the homoclinic search, or, if the orbits end at a fold of the equilibria, the fold is marked as a probable SNIC.

21 Saving and exporting results

Export..., above the plot, opens a small menu with three choices (Figure 2); each says what it will save in the view you are in, and one that has nothing to save yet is greyed out. In a script, `export svg`, `export csv` and `export dat` do the same.

SVG Saves the current view as a vector figure with light-theme colours, ready for a paper or for editing in Inkscape or Illustrator. Legends and notes are included; the dashed lines (and

the dot) that mark the current values of the parameters are left out. All the plots in this document were made this way.

CSV Available in every view.

- In the phase plane, 3D, Time course and Animate views the table has every trajectory on screen, one row per stored point, with the trajectory number, its kind (trajectory or manifold), its direction (forward or backward), t and every variable and auxiliary function.
- The bifurcation table has one row per computed point, with these columns: curve, kind (equilibrium, periodic orbit, sweep or special), label (HB1, LPC2, ...), parameter name and value, stability, number of unstable directions, period, the state, and the maximum and minimum of each variable along an orbit.
- The two-parameter table has the two parameter values, the period where relevant, the state, and labelled rows for codimension-two points. Hopf curves also get columns for ℓ_1 and for criticality (subcritical, supercritical or neutral saddle).
- In the bifurcation table, special rows for Hopf points give ℓ_1, and the stability column says subcritical or supercritical.

DAT In the phase plane, 3D, Time course and Animate views, saves the active trajectory as a `.dat` file (as **Save DAT** in the trajectory list does). In the Bifurcation and Two-parameter views, saves the same continuation data as `.dat` files: numbers only, no header and space-separated columns, the form XPPAUT and gnuplot read. The bifurcation diagram gives one file each for equilibria, periodic orbits and sweeps. The status line lists the columns, and text fields become numeric codes.

GIF In the Animate view only: opens the controls that save the animation as a GIF or a WebM video (Section 19).

Copy CSV, Save DAT In the trajectory list, copy the current trajectory (time, variables and auxiliaries) to the clipboard, or save it as a `.dat` file.

Copy text Copies the `.ode` file, including any edits, so that it can be saved or used in XPPAUT.

Writing the current state into the file

After exploring a model you will often want to keep the parameter values and settings you have arrived at. **Write state**, in the Equations tab, writes them into the `.ode` text, much as XPPAUT does when it saves a file:

- parameter values and initial conditions are changed where they are defined, in the `par`, `init` and `x(0)=` lines, as are slider ranges in `#@range` lines and history formulas of delay equations;
- the integration settings (method, total time, step, transient, tolerance, bound, random seed), the plot axes and window, and the continuation settings are collected in `@` lines just before `done`.

For example, after setting $i = 150$ and a transient of 100 in the Morris–Lecar example, the end of the file reads

```
param i=150,c=20
...
# Settings written by XPP Studio, 2026-10-20 10:30
@ meth=rungekutta, total=400, dt=0.25, trans=100, bound=10000
@ xp=v, yp=w, zp=ica, xlo=-75, xhi=45, ylo=-0.1, yhi=0.55
@ ds=2, nmax=3000, parmin=0, parmax=300
```

```
done
```

Pressing **Write state** again replaces this block rather than adding another, and **Apply** on the result restores exactly the same state. The text is not reloaded, so the trajectories on screen are kept. Use **Copy text**, or your browser's own storage of the editor, to keep the file. Parameters defined through array lines are left as written.

22 Differences from XPPAUT

XPP Studio aims to read the same files and to feel familiar, but it is a reimplementation:

- **Continuation.** It is built in and written from scratch: pseudo-arclength continuation of equilibria, and multiple shooting (rather than AUTO's orthogonal collocation) for periodic orbits. Results agree with AUTO to plotting accuracy on the examples, but very long or very unstable orbits may need more shooting intervals (Ntst).
- **Delay equations** are integrated with a fixed step only, and periodic orbits of delay equations cannot be continued. XPPAUT's other delay options, such as `delshft` and `ndelay`, are not supported. The `@ delay` option is accepted but not needed, since all of the history is kept.
- **Global flags** can set state and Markov variables and `arret`. The XPP output flag `out_put` is accepted and ignored.
- **Sets.** XPPAUT's `set name {...}` lines are read and offered in the **Sets** menu (Section 3), except that the plot settings in them (axis limits and plotted variables) are ignored.
- **The adjoint** (Section 18) is found from the monodromy matrix of the orbit rather than by integrating the adjoint equation backwards over many periods as in XPPAUT. It works for unstable orbits as well, it is computed along a whole family at once, and it is limited to differential equations with at most 60 variables. The isostable response, Floquet eigenfunction and the second-order interaction functions have no counterpart in XPPAUT; they are computed for a real, positive slowest multiplier only. The interaction function H (Section 18) is computed from `#@coupling` lines in the file, which XPPAUT reads as comments, rather than from XPPAUT's averaging window.
- **Animation** (Section 19) reads XPPAUT's `.ani` files. The commands `grab`, `xnull` and `ynull` are ignored. The animation can also be stored in the model file as `#@ani` lines and saved as a GIF or WebM file.
- **Special functions.** `special` reads `conv` and `mmult` (Tutorial 8). Other kinds of `special` are skipped with a warning.
- **Not supported:** `export` declarations.
- **Two-parameter Hopf curves** are limited to 12 variables, because the bialternate product grows quadratically with dimension.
- **Homoclinic and heteroclinic orbits** (Section 17) are found by shooting along the one-dimensional unstable manifold of a saddle, not by AUTO's HomCont projection boundary conditions, so they are limited to saddles with one real unstable direction (or one real stable direction, with time running backwards).
- **Volterra models** cannot be continued, since their state is infinite-dimensional.
- **Browser files.** Files are opened and saved through the browser. Table data files are kept in the browser's local storage until you press **Forget table files**.

23 Credits and licence

XPP Studio is inspired by, and reads the model files of, XPPAUT by G. Bard Ermentrout (University of Pittsburgh), described in [1]. Its continuation methods follow the ideas of AUTO [2] by Eusebius Doedel and colleagues, and of MatCont [3]. This version (October 2026) was created by Peter Ashwin and Bard Ermentrout in collaboration with Claude Opus 5.5, an AI model made by Anthropic.

XPP Studio is free software, released under the GNU General Public License, version 3 or later. The licence text is distributed with the program (`LICENSE.txt`) and is available at `https://www.gnu.org/licenses/gpl-3.0.html`.

References

[1] G. B. Ermentrout, *Simulating, Analyzing, and Animating Dynamical Systems: A Guide to XPPAUT for Researchers and Students*, SIAM, Philadelphia, 2002.

[2] E. J. Doedel, A. R. Champneys, F. Dercole, T. F. Fairgrieve, Yu. A. Kuznetsov, B. Oldeman, R. C. Paffenroth, B. Sandstede, X. J. Wang and C. Zhang, *AUTO-07P: Continuation and bifurcation software for ordinary differential equations*, Concordia University, 2007.

[3] A. Dhooge, W. Govaerts and Yu. A. Kuznetsov, MATCONT: a MATLAB package for numerical bifurcation analysis of ODEs, *ACM Trans. Math. Software* **29** (2003) 141–164.

[4] M. J. Feigenbaum, Quantitative universality for a class of nonlinear transformations, *J. Stat. Phys.* **19** (1978) 25–52.

[5] W. J. F. Govaerts, *Numerical Methods for Bifurcations of Dynamical Equilibria*, SIAM, Philadelphia, 2000.

[6] Yu. A. Kuznetsov, *Elements of Applied Bifurcation Theory*, 3rd edn, Springer, New York, 2004.

[7] H. B. Keller, Numerical solution of bifurcation and nonlinear eigenvalue problems, in *Applications of Bifurcation Theory*, ed. P. H. Rabinowitz, Academic Press, 1977, pp. 359–384.

[8] L. F. Shampine and M. W. Reichelt, The MATLAB ODE suite, *SIAM J. Sci. Comput.* **18** (1997) 1–22.

[9] P. Linz, *Analytical and Numerical Methods for Volterra Equations*, SIAM, Philadelphia, 1985.

[10] C. Morris and H. Lecar, Voltage oscillations in the barnacle giant muscle fiber, *Biophys. J.* **35** (1981) 193–213.

[11] J. Rinzel and G. B. Ermentrout, Analysis of neural excitability and oscillations, in *Methods in Neuronal Modeling*, ed. C. Koch and I. Segev, MIT Press, 1989, pp. 135–169.

[12] D. Breda, S. Maset and R. Vermiglio, Pseudospectral differencing methods for characteristic roots of delay differential equations, *SIAM J. Sci. Comput.* **27** (2005) 482–495.

[13] G. E. Hutchinson, Circular causal systems in ecology, *Ann. N. Y. Acad. Sci.* **50** (1948) 221–246.

[14] M. C. Mackey and L. Glass, Oscillation and chaos in physiological control systems, *Science* **197** (1977) 287–289.

[15] O. E. Rössler, An equation for continuous chaos, *Phys. Lett. A* **57** (1976) 397–398.

[16] G. B. Ermentrout and D. H. Terman, *Mathematical Foundations of Neuroscience*, Springer, New York, 2010.

[17] Y. Kuramoto, *Chemical Oscillations, Waves, and Turbulence*, Springer, Berlin, 1984.

[18] D. G. Aronson, G. B. Ermentrout and N. Kopell, Amplitude response of coupled oscillators, *Physica D* **41** (1990) 403.

[19] D. Wilson and B. Ermentrout, Greater accuracy and broadened applicability of phase reduction using isostable coordinates, *J. Math. Biol.* **76** (2018) 37.

[20] D. Wilson, Isostable reduction of oscillators with piecewise smooth dynamics and complex Floquet multipliers, *Phys. Rev. E* **99** (2019) 022210.

[21] D. Wilson and B. Ermentrout, Phase models beyond weak coupling, *Phys. Rev. Lett.* **123** (2019) 164101.